

ORACLE
NetSuite

Sample SuiteCloud Customization Projects

2025.2

November 12, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

SuiteCloud Customization Tutorials	1
Hide a Column in a Sublist	1
Validate Order on Entry	8
Set Default Values in a Sublist	17
Set the Item Amount to Zero for Marketing Orders	25
Add Custom Button to Run a Suitelet	32
Calculate Commission on Sales Orders	38
Copy a Value to the Item Column	48
Set a Default Posting Period in a Custom Field	56
Track Customer Deposit Balances	65
Set Purchase Order Exchange Rate	97
Disable Tax Fields	106
Populate Item Substitution	119

SuiteCloud Customization Tutorials

Here you'll find a variety of tutorials that walk you through creating sample customization projects using SuiteCloud concepts, such as custom forms and fields, saved searches, and script parameters. All tutorials use SuiteScript 2.1 sample code. The projects are available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). Each one includes a complete project customization setup with custom objects, unit tests, package file configuration, and more. Each folder is its own project and has a README.md file with detailed descriptions of the use case and customization.

The following tutorials are available:

- [Hide a Column in a Sublist](#)
- [Validate Order on Entry](#)
- [Set Default Values in a Sublist](#)
- [Set the Item Amount to Zero for Marketing Orders](#)
- [Add Custom Button to Run a Suitelet](#)
- [Calculate Commission on Sales Orders](#)
- [Copy a Value to the Item Column](#)
- [Set a Default Posting Period in a Custom Field](#)
- [Track Customer Deposit Balances](#)
- [Set Purchase Order Exchange Rate](#)
- [Disable Tax Fields](#)
- [Populate Item Substitution](#)

Important Note

The sample code included herein is provided on an "as is" basis, without warranty of any kind, to the fullest extent permitted by law. Oracle NetSuite Inc. does not warrant or guarantee the individual success developers may have in implementing the sample code on their development platforms or in using their own Web server configurations. Oracle NetSuite Inc. does not warrant, guarantee or make any representations regarding the use, results of use, accuracy, timeliness or completeness of any data or information relating to the sample code. Oracle NetSuite Inc. disclaims all warranties, express or implied, and in particular, disclaims all warranties of merchantability, fitness for a particular purpose, and warranties related to the code, or any service or software related thereto. Oracle NetSuite Inc. shall not be liable for any direct, indirect or consequential damages or costs of any type arising out of any action taken by you or others related to the sample code.

Hide a Column in a Sublist

This sample shows you how to hide a field column in a standard sublist.

You can find this project on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes a complete project customization setup with custom objects, unit tests, package file configuration, and more.

Scenario

You're the NetSuite administrator for your organization. You use the invoice record type for your invoices. This record type has everything you need to manage your invoices, but it also has some fields you don't

need. In the Item sublist on an invoice record, you don't need to fill in the Item field. This field appears as a column in the Item sublist, and you want to use a script to hide it before an invoice is edited.

Customization Details

The customization for this use case includes:

- A user event script triggered on the beforeLoad entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Write the Script](#)
- [Step 2: Create the Script Record](#)
- [Step 3: Deploy the Script](#)
- [Step 4: Test the Solution](#)

Before You Begin

The features, permissions, and other requirements you'll need to complete this tutorial are listed in the following table.

Required Features	Ensure these features are enabled in your account: ■ Server SuiteScript – Lets you attach server scripts to records. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ A/R – Lets you create, edit, and save invoices. For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Invoices – Edit access For more information, see the help topic NetSuite Permissions Overview
Other Requirements	None

Step 1: Write the Script

This script hides the **Item** column on the **Item** sublist for an invoice.

Script Summary

The following table summarizes the script for this tutorial:

Script: Hide Sublist Column	
Script Type	SuiteScript 2.x User Event Script Type

Script: Hide Sublist Column	
Modules Used	■ N/ui/serverWidget Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ beforeLoad(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below, so that you can copy and paste it into your text editor and save the script as a .js file (for example, ue_hideSublistColumn.js).

If you'd rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the define function, which you need for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/ui/serverWidget', 'N/log'], (serverWidget, log) => {
8      function beforeLoad(context) {
9          if (context.type === context.UserEventType.EDIT) {
10             hideColumnField(context.form, 'item', 'item');
11         }
12     }
13     function hideColumnField(formObj, sublistId, fieldId) {
14         try {
15             const formSublist = formObj.getSublist({
16                 id: sublistId
17             });
18             if (formSublist) {
19                 const formField = formSublist.getField({
20                     id: fieldId
21                 });
22                 if (formField && typeof formField !== 'undefined' && formField !== null) {
23                     formField.updateDisplayType({
24                         displayType: serverWidget.FieldDisplayType.HIDDEN
25                     });
26                 }
27             }
28         } catch(error) {
29             log.error({
30                 title: 'Error occurred when hiding field',
31                 details: JSON.stringify({
32                     sublistId: sublistId,

```



```

33         fieldId: fieldId
34     });
35 }
36 }
37 }
38 return {
39     beforeLoad: beforeLoad
40 };
41 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Define the `hideColumnField()` helper function
- Add a try-catch block
- Get the Item sublist
- Hide the Item field
- Create the return statement
- Save your script file

Note: The code snippets don't include indentation. Refer to [The Complete Script](#) for suggested indentation.

Start with required opening lines

JSDoc comments and a define function are required at the top of the script file. The JSDoc comments in this script show that it's a SuiteScript 2.1 user event script. The script uses two SuiteScript modules specified in the define statement:

- `N/ui/serverWidget` – provides access to the NetSuite user interface, including loaded forms
- `N/log` – lets you log execution details

Start a new script file in any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/ui/serverWidget', 'N/log'], (serverWidget, log) => {
8  });

```

Create the entry point function

This script is triggered on the `beforeLoad` entry point when an invoice record loads. The entry point function checks if the record is being edited, and if so, it calls a helper function that is also defined in the script.

Add the following function definition at the top of the define function:

```
1 function beforeLoad(context) {
2   if (context.type === context.UserEventType.EDIT) {
3     hideColumnField(context.form, 'item', 'item');
4   }
5 }
```

Define the `hideColumnField()` helper function

This script uses a helper function to hide the **Item** column.

Add the following code after the `beforeLoad` function definition:

```
1 function hideColumnField(formObj, sublistId, fieldId) {
2 }
```

Add a try-catch block

The helper function uses a try-catch block to log any errors that might occur during script processing.

Add the following code at the top of the `hideColumnField()` helper function definition:

```
1 try {
2 } catch(error) {
3   log.error({
4     title: 'Error occurred when hiding field',
5     details: JSON.stringify({
6       sublistId: sublistId,
7       fieldId: fieldId
8     })
9   });
10 }
```

Get the **Item** sublist

To hide a field on a form, you need to get the field. Since the field you want to remove is on the **Item** sublist, you first need to get the sublist.

Add the following code inside the try section of the try-catch block:

```
1 const formSublist = formObj.getSublist({
2   id: sublistId
3 });
```

Hide the **Item** field

If the **Item** sublist exists on the invoice record, get the field from the sublist and make sure that it's defined. If it's defined, hide it.

Add the following code inside the try section of the try-catch block:

```

1  if (formSublist) {
2      const formField = formSublist.getField({
3          id: fieldId
4      });
5      if (formField && typeof formField !== 'undefined' && formField !== null) {
6          formField.updateDisplayType({
7              displayType: serverWidget.FieldDisplayType.HIDDEN
8          });
9      }
10 }

```

Create the return statement

This script associates the beforeLoad function with the beforeLoad user event entry point.

Add the following code immediately above the closing `});` in your script:

```

1  return {
2      beforeLoad: beforeLoad
3  };

```

Save your script file

You need to save your script file so you can load it to the NetSuite File Cabinet. Before saving, you may want to adjust the indentation to improve readability. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script file, save it as a .js file (for example, ue_hideSublistColumn.js).

Step 2: Create the Script Record

Now that you've completed the script, you can upload the script file to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Hide Sublist Column
ID	_ue_hide_sublist_column NetSuite prefixes the ID with customscript .
Description	This script hides the Item field on the Item sublist on an invoice record.

5. (Optional) Set any other fields on the script record as needed.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 3: Deploy the Script](#).

Step 3: Deploy the Script

After you create the script record, you can create a script deployment record for it. The script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 2: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Invoice
ID	_ue_hide_sublist_column NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's usually best to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

3. Click **Save**.

Step 4: Test the Solution

After you create the script record and deploy your script, you can test your solution by viewing and editing an existing invoice.

To test your solution:

1. Go to Transactions > Sales > Create Invoices > List.
2. Click **View** next to any existing invoice.
3. On the Invoice page, scroll down to the Item sublist and verify that the **Item** column appears.
4. Click **Edit**.
5. Scroll down to the **Item** sublist and verify that the **Item** column doesn't appear. Scroll to the right to see all columns on the sublist.

6. Click **Cancel** to close the invoice without editing it.

Validate Order on Entry

This use case shows you how to validate the number of items in an order. You do this to ensure the number of items corresponds to the number of cases that fit on a pallet. This validation ensures that you accept only orders that are fulfilled by full cases. This use case also shows you how to verify that the total weight of the order doesn't exceed the maximum allowed for a single order.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#) and includes a complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

The customization for this use case includes:

- A custom field (Cases per Pallet) to store the number of cases of the item on one pallet
- A custom field (Item Weight) to store the weight of a single item
- A client script triggered on the validateLine and saveRecord entry points
- A script parameter (Max Weight) to store the maximum weight for an order

Steps in this tutorial to complete the customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Fields](#)
- [Step 2: Write the Script](#)
- [Step 3: Create the Script Record](#)
- [Step 4: Deploy the Script](#)
- [Step 5: Create and Set the Script Parameter](#)
- [Step 6: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	Ensure the following features are enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access

	■ Script Deployments – Edit access ■ Transaction Line Fields – Create access ■ Sales Orders – Edit access For more information, see the help topic NetSuite Permissions Overview
Other Requirements	None

Step 1: Create the Custom Fields

This customization uses two custom fields: **Cases per Pallet** stores the number of cases of an item that fit on one pallet, and **Item Weight** stores the weight of a single item. Both fields are custom transaction line fields added to the Item sublist on a sales order record.

To create the Cases per Pallet field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New.
2. Enter or select values for fields as listed in the following table:

Field	Value
Label	Cases per Pallet
ID	_cases_per_pallet NetSuite prefixes the ID with custcol .
Type	Integer Number
Store Value	Checked
Applies To	Sale Item

3. Click **Save**.

To create the Item Weight field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New.
2. Enter or select values for fields as listed in the following table:

Field	Value
Label	Item Weight
ID	_item_weight NetSuite prefixes the ID with custcol .
Type	Decimal Number
Store Value	Checked
Applies To	Sale Item

3. Click **Save**.

For more information about creating custom entity fields, see the following help topics:

- [Creating a Custom Field](#)

■ Creating Custom Transaction Line Fields

Step 2: Write the Script

This script validates that the number of items in an order corresponds to the number of cases that fit on a pallet. This validation ensures that you accept only orders that are fulfilled by full cases. The script also verifies that the total weight of the order doesn't exceed the maximum allowed for a single order.

Script Summary

The following table summarizes the script:

Script: Validate Order	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/record Module ■ N/runtime Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you may if you want. This tutorial doesn't explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. You should explicitly load this module to avoid conflicts with other objects named 'log'.
Entry Points	■ saveRecord(scriptContext) ■ validateLine(scriptContext)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so you can copy and paste it into your text editor and save it as a .js file (for example, cs_validateOrder.js).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript

```

```

4  * @NModuleScope SameAccount
5  */
6
7  define(['N/runtime', 'N/log'], (, runtime, log) => {
8      function validateLine(scriptContext) {
9          const recSalesOrder = scriptContext.currentRecord;
10         if (scriptContext.sublistId === 'item') {
11             const casePerPallet = recSalesOrder.getCurrentSublistValue({
12                 sublistId: 'item',
13                 fieldId: 'custcol_cases_per_pallet'
14             });
15             const quantity = recSalesOrder.getCurrentSublistValue({
16                 sublistId: 'item',
17                 fieldId: 'quantity'
18             });
19             if (quantity % casePerPallet !== 0) {
20                 alert('Please order in multiples of ${casePerPallet} Case/Pallet for this item.');
```

return false;

```

21             } else {
22                 return true;
23             }
24         }
25     }
26     return true;
27 }
28 function saveRecord(scriptContext) {
29     const scriptObj = runtime.getCurrentScript();
30     const weightParam = scriptObj.getParameter({
31         name: 'custscript_max_weight'
32     });
33     if (NSUtil.isEmpty(weightParam)) {
34         log.error({
35             title: 'saveRecord',
36             details: 'Max weight script parameter not defined'
37         });
38         return;
39     }
40     const recSalesOrder = scriptContext.currentRecord;
41     const lineCount = recSalesOrder.getLineCount({
42         sublistId: 'item'
43     });
44     let totalWeight = 0;
45     for (let i = 0; i < lineCount; i++) { const itemWeight = recSalesOrder.getSublistValue({ sublistId: 'item', field
Id: 'custcol_item_weight', line: i }); const quantity = recSalesOrder.getSublistValue({ sublistId: 'item', fieldId: 'quanti
ty', line: i }); const lineWeight = itemWeight * quantity; totalWeight += lineWeight; } if (totalWeight > weightParam) {
46         alert('The total weight of the order is ${totalWeight}. The max weight is ${weightParam}. Please adjust the order and
submit a separate order if necessary.');
```

return false;

```

47     }
48     return true;
49 }
50 }
51 const NSUtil = {};
52 NSUtil.isEmpty = function(stValue) {
53     return(
54         stValue === "" || stValue === null || stValue === undefined ||
55         (stValue.constructor === Array && stValue.length === 0) ||
56         (stValue.constructor === Object &&
57         (function(v){
58             for (const k in v) return false;
59             return true;
60         })(stValue))
61     );
62 }
63 return {
64     validateLine: validateLine,
65     saveRecord: saveRecord
66 };
67 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point functions
- Get the current record
- Validate the cases per pallet
- Get the script parameter
- Validate the total weight of the order
- Create the isEmpty function
- Create the return statement
- Save your script file



Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments indicate that it's a SuiteScript 2.x (2.0 or 2.1) client script. The script uses three SuiteScript modules specified in the define statement:

- `N/runtime` – provides access to script parameters
- `N/log` – lets you log execution details

Start a new script file in any text editor and add the following JSDoc comments and define function at the top:



Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/log'], (runtime, log) => {
8  });

```

Create the entry point functions

This script is triggered on the `validateLine` entry point when an item is added to a sales order, and on the `saveRecord` entry point when a sales order is saved.

Add the following function definitions at the top of the define function:

```

1  function validateLine(scriptContext) {
2      return true;
3  }
4  function saveRecord(scriptContext) {
5  }

```

Get the current record

This script operates on the current sales order record. You need to get the current sales order record.

Add the following code at the top of the `validateLine` function, above the `return true;` statement:

```
1 | const recSalesOrder = scriptContext.currentRecord;
```

Validate the cases per pallet

You need to validate the value entered in the **Cases Per Pallet** column for each item on the sales order record. The **Quantity** entered for each item must correspond to the number of cases allowed per pallet for that item.

Add the following code to the `validateLine` function, above the `return true;` statement:

```
1 | if (scriptContext.sublistId === 'item') {
2 |     const casePerPallet = recSalesOrder.getCurrentSublistValue({
3 |         sublistId: 'item',
4 |         fieldId: 'custcol_cases_per_pallet'
5 |     });
6 |     const quantity = recSalesOrder.getCurrentSublistValue({
7 |         sublistId: 'item',
8 |         fieldId: 'quantity'
9 |     });
10 |     if (quantity % casePerPallet !== 0) {
11 |         alert('Please order in multiples of ${casePerPallet} Case/Pallet for this item.');
```

Get the script parameter

When the sales order record is saved, you need to get the value of the script parameter (`custscript_max_weight`, set in Step 5) that specifies the maximum weight for an order.

Add the following code at the top of the `saveRecord` function:

```
1 | const scriptObj = runtime.getCurrentScript();
2 | const weightParam = scriptObj.getParameter({
3 |     name: 'custscript_max_weight'
4 | });
5 | if (NSUtil.isEmpty(weightParam)) {
6 |     log.error({
7 |         title: 'saveRecord',
8 |         details: 'Max weight script parameter not defined'
9 |     });
10 |     return;
11 | }
```

Validate the total weight of the order

You also need to verify that the total weight of the order doesn't exceed the maximum weight allowed, as set by the script parameter (`custcol_item_weight`, set in Step 5).

Add the following code to the `saveRecord` function:

```

1 | const recSalesOrder = scriptContext.currentRecord;
2 | const lineCount = recSalesOrder.getLineCount({
3 |     sublistId: 'item'
4 | });
5 | let totalWeight = 0;
6 | for (let i = 0; i < lineCount; i++) { const itemWeight = recSalesOrder.getSublistValue({ sublistId: 'item', fieldId: 'cust
   | col_item_weight', line: i }); const quantity = recSalesOrder.getSublistValue({ sublistId: 'item', fieldId: 'quantity', line: i }); const lineWeight = itemWeight * quantity; totalWeight += lineWeight; } if (totalWeight > weightParam) {
7 |     alert('The total weight of the order is ${totalWeight}. The max weight is ${weightParam}. Please adjust the order and submit a
   | separate order if necessary. ');
8 |     return false;
9 | }
10 | return true;

```

Create the isEmpty function

This script uses a utility function to determine if the value of the script parameter is empty.

Add the following code after the end of the saveRecord function:

```

1 | const NSUtil = {};
2 | NSUtil.isEmpty = function(stValue) {
3 |     return(
4 |         stValue === "" || stValue === null || stValue === undefined ||
5 |         (stValue.constructor === Array && stValue.length === 0) ||
6 |         (stValue.constructor === Object &&
7 |         (function(v){
8 |             for (const k in v) return false;
9 |             return true;
10 |        })(stValue))
11 |     );
12 | }

```

Create the return statement

This script associates the validateLine function with the validateLine client script entry point, and the saveRecord function with the saveRecord client script entry point.

Add the following code immediately above the closing `});` in your script:

```

1 | return {
2 |     validateLine: validateLine,
3 |     saveRecord: saveRecord
4 | };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, cs_validateOrder.js).

Step 3: Create the Script Record

Now that you've completed the script, you can upload the file to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Validate Order
ID	_cs_validate_order NetSuite prefixes the ID with customscript .
Description	This script validates the number of items in an order. The script ensures the number of items in the order corresponds to the number of cases that fit on a pallet. This validation ensures that you accept only orders that are fulfilled by full cases. It also verifies that the total weight doesn't exceed the maximum allowed for a single order.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record for your script, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_cs_validate_order NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Create and Set the Script Parameter

This tutorial uses a script parameter to specify the maximum allowed weight for a sales order.

For more information about creating and setting script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

To create the script parameter:

1. Go to Customization > Scripting > Scripts.
2. Find your script in the list and click **Edit** next to the script name.
3. On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.
4. On the Script Field page, enter the following values:

Script Parameter	Label	ID	Type	Description	Preference
Maximum Weight	Maximum Weight	_max_weight NetSuite prefixes the ID with custscript , so the script uses custscript_max_weight .	Decimal Number	The maximum allowed weight for a sales order.	Leave blank

5. To save the script parameter, click **Save**. The Script page appears.
6. To save the script record with the updated script parameter, click **Save**.

After you create the script parameter, set its value on the script deployment record that you created in Step 4.

To set the script parameter:

1. Go to Customization > Scripts > Script Deployments.
2. Find your script deployment in the list and click **Edit** next to the script deployment ID.
3. On the **Parameters** subtab, enter **25** for the **Maximum Weight** script parameter.
4. Click **Save**.

Step 6: Test the Solution

After you create the script record and deploy your script, you can test your solution by creating a sales order and adding items.

To test your solution:

1. Go to Transactions > Sales > Enter Sales Orders to create a new sales order.
2. Fill in all required fields on the sales order with any values you want. You can use the Standard Sales Order form.

3. On the Items subtab, add any item. Set the **Quantity** to 17, the **Cases Per Pallet** to 10, and the **Item Weight** to 5.
4. Click **Add** to add the item. Verify that because the **Quantity** of the item (17) is not a value that corresponds to the **Cases Per Pallet** (10), an error occurs: "Please order in multiples of 10 Cases/ Pallet for this item" (which is text set in the script).
5. Close the error message and change the **Quantity** value to 10 and then click **Add** . Verify that the item was successfully added. You now have an order of 10 items each weighing 5pounds.
6. Click **Save**. Verify that the order is not saved because the weight of the order (10 x 5 lbs) is greater than the maximum weight allowed (25 as set in the script parameter). An error is displayed: "The total weight of the order is 50. The max weight is 25. Please adjust the order and submit a separate order if necessary" (which is text set in the script).
7. Edit the order so that the **Item Weight** is 1. This will make the total order weight 10 lbs.
8. Click **Save**. Verify that the order is successfully saved.

Set Default Values in a Sublist

This use case shows you how to set default values for fields in a sublist. These values are taken from custom fields on another record type.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes a complete project customization setup with custom objects, unit tests, package file configuration, and more.

Scenario

You're the NetSuite administrator for your organization. You use the vendor record type for vendors your organization works with, and the vendor bill record type to track their bills. You've added two custom fields to the vendor record type to store special department and class information for each vendor. When you edit a vendor bill record and change values in the item or expense sublists, you want the **Department** and **Class** columns in the sublist to fill in with the custom field values from the vendor record.

Customization Details

The customization for this use case includes:

- Two custom fields (Department and Class) to hold department and class information for the vendor
- A client script triggered on the fieldChanged entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Fields](#)
- [Step 2: Write the Script](#)
- [Step 3: Create the Script Record](#)
- [Step 4: Deploy the Script](#)
- [Step 5: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	The following features must be enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. For more information, see the help topic Enabling Features.
Required Permissions	You will need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Custom Entity Fields – Create access For more information, see the help topic NetSuite Permissions Overview
Other Requirements	None

Step 1: Create the Custom Fields

This customization uses two custom entity fields that are added to the vendor record. The **Custom Department** field stores department information for the vendor, and the **Custom Class** field stores class information.

To create the Custom Department field:

1. Go to Customization > Lists, Records, & Fields > Entity Fields > New.
2. Enter values for each field as listed in the following table:

Field	Value
Label	Custom Department
ID	_custom_department NetSuite prefixes the ID with custentity .
Description	Stores department information for a vendor.
Type	List/Record
List/Record	Department
Applies To	Vendor
Display	Subtab: Main

3. Click **Save**.

To create the Custom Class field:

1. Go to Customization > Lists, Records, & Fields > Entity Fields > New.
2. Enter values for each field as listed in the following table:

Field	Value
Label	Custom Class
ID	_custom_class NetSuite prefixes the ID with custentity .
Description	Stores class information for a vendor.
Type	List/Record
List/Record	Class
Applies To	Vendor
Display	Subtab: Main

3. Click **Save**.

For more information about creating custom entity fields, see the following help topics:

- [Creating a Custom Field](#)
- [Creating Custom Entity Fields](#)

Step 2: Write the Script

When a field is changed on the Expenses or Items sublist of a vendor bill, the script fills in the **Department** and **Class** columns on the sublist. If a field on a different sublist is changed, or if a field that's not on a sublist is changed, no sublist column values are populated.

Script Summary

The following table summarizes the script:

Script: Default Sublist Values	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/search Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you may if you want. This tutorial doesn't explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ fieldChanged(scriptContext)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, cs_defaultSublistValues.js).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['/N/search', '/N/log'], (search, log) => {
8      function fieldChanged(scriptContext) {
9          const sublistVal = scriptContext.sublistId;
10         if (sublistVal === 'item' || sublistVal === 'expense') {
11             const rec = scriptContext.currentRecord;
12             const vendorId = rec.getValue({
13                 fieldId: 'entity'
14             });
15             if (vendorId === '') {
16                 return;
17             }
18             const vendorLookup = search.lookupFields({
19                 type: search.Type.VENDOR,
20                 id: vendorId,
21                 columns: ['custentity_custom_department', 'custentity_custom_class']
22             });
23             let classVal = '';
24             let deptVal = '';
25             classVal = vendorLookup['custentity_custom_class'][0].value;
26             deptVal = vendorLookup['custentity_custom_department'][0].value;
27             try {
28                 rec.setCurrentSublistValue({
29                     sublistId: sublistVal,
30                     fieldId: 'department',
31                     value: deptVal,
32                     ignoreFieldChange: true
33                 });
34                 rec.setCurrentSublistValue({
35                     sublistId: sublistVal,
36                     fieldId: 'class',
37                     value: classVal,
38                     ignoreFieldChange: true
39                 });
40             } catch(e) {
41                 log.debug({
42                     title: 'Unable to set record values',
43                     details: 'Unable to set record values for department and class'
44                 });
45             }
46         }
47     }
48     return {
49         fieldChanged: fieldChanged
50     };
51 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Check the sublist type
- Get the vendor class and department values
- Set the record department and class fields
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 client script. The script uses two SuiteScript modules in the define statement:

- N/search – provides access to search functionality
- N/log – lets you log execution details

Start a new script file in any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/search', 'N/log'], (search, log) => {
8  });

```

Create the entry point function

This script is triggered on the `fieldChanged` entry point when a column in the Expenses and Items sublist is changed.

Add the following function definition at the top of the define function:

```

1  function fieldChanged(scriptContext) {
2  }

```

Check the sublist type

You're only interested in the item and expense sublists, so verify that the current sublist is one of those.

Add the following code at the top of the `fieldChanged` function:

```
1 | const sublistVal = scriptContext.sublistId;
2 | if (sublistVal === 'item' || sublistVal === 'expense') {
3 | }
```

Get the vendor class and department values

For item or expense sublists, you need to look up the vendor and retrieve the custom **Department** and **Class** fields.

Add the following code inside the `if` statement in the `fieldChanged` function:

```
1 | let rec = scriptContext.currentRecord;
2 | let vendorId = rec.getValue({
3 |   fieldId: 'entity'
4 | });
5 | if (vendorId === '') {
6 |   return;
7 | }
8 | const vendorLookup = search.lookupFields({
9 |   type: search.Type.VENDOR,
10 |   id: vendorId,
11 |   columns: ['custentity_custom_department', 'custentity_custom_class']
12 | });
13 | const classVal = '';
14 | const deptVal = '';
15 | classVal = vendorLookup['custentity_custom_class'][0].value;
16 | deptVal = vendorLookup['custentity_custom_department'][0].value;
```

Set the record department and class fields

After you retrieve the custom **Department** and **Class** values for the vendor, you want to add them to the vendor bill record. You do this in a try-catch block so you can log any errors that might occur while updating the vendor record.

Add the following code inside the `if` statement in `fieldChanged` function:

```
1 | try {
2 |   rec.setCurrentSublistValue({
3 |     sublistId: sublistVal,
4 |     fieldId: 'department',
5 |     value: deptVal,
6 |     ignoreFieldChange: true
7 |   });
8 |   rec.setCurrentSublistValue({
9 |     sublistId: sublistVal,
10 |    fieldId: 'class',
11 |    value: classVal,
12 |    ignoreFieldChange: true
13 |   });
14 | } catch(e) {
15 |   log.debug({
16 |     title: 'Unable to set record values',
17 |     details: 'Unable to set record values for department and class'
18 |   });
19 | }
```

Create the return statement

This script associates the `fieldChanged` function with the `fieldChanged` client script entry point.

Add the following code immediately above the closing `});` in your script:

```
1 | return {
2 |     fieldChanged: fieldChanged
3 | };
```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, `cs_defaultSublistValues.js`).

Step 3: Create the Script Record

Now that you've completed the script, you can upload it to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Default Sublist Values
ID	_cs_default_sublist_values NetSuite prefixes the ID with customscript .
Description	This script sets the default values for the Department and Class columns in the sublist using the custom field values from the vendor record.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Vendor Bill
ID	_cs_default_sublist_values NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Test the Solution

After you create the script record and deploy your script, you can test it by creating a new vendor record and verifying the values in the **Department** and **Class** sublist columns.

To test your solution:

1. Go to Lists > Relationships > Vendors > New to create a new vendor.
2. On the Vendor page, enter the following values:

Field	Value
Company Name	Tutorial Vendor
Category	Select any value, for example, Subcontractor
Custom Department	Select any value, for example, Service
Custom Class	Select any value, for example, Recurring Business

3. Click **Save**.
4. Go to Transactions > Payables > Enter Bills to enter a new bill from the new Tutorial Vendor vendor.
5. On the Bill page, enter the following values:

Field	Value
Custom Form	Standard Vendor Bill
Vendor	Tutorial Vendor
All remaining required fields	Accept the default values
Expenses and Items subtab: Account	Select any account
Expenses and Items subtab: Amount	Enter any amount

6. Verify that as soon as you enter the **Amount**, the **Department** and **Class** fields fill in automatically with the values you set for the Tutorial Vendor.

7. You can leave the Bill page without finishing because you only needed to test that the **Department** and **Class** fields on the vendor bill filled in automatically.

Set the Item Amount to Zero for Marketing Orders

This use case shows you how to use a checkbox to indicate a sales order is a marketing order, so the amount is automatically set to zero.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes a complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

The customization for this use case includes:

- A custom field (Marketing Order) that is used to indicate a marketing order
- A user event script triggered on the afterSubmit entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Field](#)
- [Step 2: Write the Script](#)
- [Step 3: Create the Script Record](#)
- [Step 4: Deploy the Script](#)
- [Step 5: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	The following features must be enabled in your account: ■ Server SuiteScript – Lets you attach server scripts to records. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ Sales Order – Lets you create, edit, and save sales orders (before fulfillment or payment). For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Transaction Body Fields – Create access ■ Sales Orders – Edit access

	For more information, see the help topic NetSuite Permissions Overview .
Other Requirements	None

Step 1: Create the Custom Field

This customization uses a custom field. The **Marketing Order** custom field indicates if the sales order is a marketing order. It's a transaction body field added to the sales record.

To create the custom field:

1. Go to Customization > Lists, Records, Fields > Transaction Body Fields > New.
2. Enter or select values for fields as listed in the following table:

Field	Value
Label	Marketing Order
ID	_marketing_order NetSuite prefixes the ID with custbody .
Type	Check Box
Description	Indicates whether the sales order is a marketing order.
Applies To	Sale
Display	Subtab: Main

3. Click **Save**.

For more information about creating custom fields, see the following help topics:

- [Creating a Custom Field](#)
- [Creating Custom Transaction Body Fields](#)

Step 2: Write the Script

This script determines if the sales order being saved is a marketing order. If it's a marketing order, the script sets the **Amount** field on the sales order to zero.

Script Summary

The following table summarizes the script:

Script: Marketing Order	
Script Type	SuiteScript 2.x User Event Script Type
Modules Used	■ N/record Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you may if you want. This tutorial doesn't explicitly load this module.

Script: Marketing Order	
	■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ <code>afterSubmit(context)</code>

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, ue_marketingOrder.js).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/log'], (record, log) => {
8      function afterSubmit(context) {
9          try {
10             const currentRecord = context.newRecord;
11             const recordID = currentRecord.id;
12             const recordType = currentRecord.type;
13             const objSalesOrder = record.load({
14                 type: recordType,
15                 id: recordID,
16                 isDynamic: true
17             });
18             const isMktgSalesOrder = objSalesOrder.getValue({
19                 fieldId: 'custbody_marketing_order'
20             });
21             if (isMktgSalesOrder) {
22                 const intItemLines = objSalesOrder.getLineCount({
23                     sublistId: 'item',
24                 });
25                 for (let i = 0; i < intItemLines; i++) {
26                     objSalesOrder.selectLine({
27                         sublistId: 'item',
28                         line: i
29                     });
30                     objSalesOrder.setCurrentSublistValue({
31                         sublistId: 'item',
32                         fieldId: 'amount',
33                         value: 0
34                     });
35                     objSalesOrder.commitLine({

```



```

36         sublistId: 'item'
37     });
38     }
39     objSalesOrder.save();
40     }
41     } catch(e) {
42         log.error({
43             title: 'marketing order script - afterSubmit',
44             details: e.message
45         });
46     }
47     }
48     return {
49         afterSubmit: afterSubmit
50     };
51 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Load the sales order that is being saved
- Create the return statement
- Save your script file



Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 user event script. The script uses two SuiteScript modules specified in the define statement:

- N/record – provides access to or use of NetSuite records
- N/log – allows you to log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top:



Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/log'], (record, log) => {
8  });

```

Create the entry point function

This script is triggered on the `afterSubmit` entry point when a sales order is saved. The script uses a try-catch block to log any errors that might occur during script processing. Most of the script code is placed in the try block.

Add the following function definition and initial try-catch block statements at the top of the define function:

```
1 function afterSubmit(context) {
2   try {
3   } catch(e) {
4     log.error({
5       title: 'marketing order script - afterSubmit',
6       details: e.message
7     });
8   }
9 }
```

Load the sales order that is being saved

You need to load the sales order record to determine if it's a marketing order.

Add the following code inside the try block:

```
1 const currentRecord = context.newRecord;
2 const recordID = currentRecord.id;
3 const recordType = currentRecord.type;
4 const objSalesOrder = record.load({
5   type: recordType,
6   id: recordID,
7   isDynamic: true
8 });
```

Determine if the sales order is a marketing order

You need to determine if the sales order is a marketing order by checking the value of the **Marketing Order** custom field.

Add the following code inside the try block:

```
1 const isMktgSalesOrder = objSalesOrder.getValue({
2   fieldId: 'custbody_marketing_order'
3 });
```

Set the amount for all items on the sales order to zero

If the sales order is a marketing order, you need to set the **Amount** for each item on the sales order to 0.

Add the following code inside the try block:

```
1 if (isMktgSalesOrder) {
2   const intItemLines = objSalesOrder.getLineCount({
3     sublistId: 'item',
4   });
5   for (let i = 0; i < intItemLines; i++) {
6     objSalesOrder.selectLine({
7       sublistId: 'item',
8       line: i
```

```

9      });
10     objSalesOrder.setCurrentSublistValue({
11         sublistId: 'item',
12         fieldId: 'amount',
13         value: 0
14     });
15     objSalesOrder.commitLine({
16         sublistId: 'item'
17     });
18 }
19 objSalesOrder.save();
20 }

```

Create the return statement

This script associates the `afterSubmit` function with the `afterSubmit` user event entry point.

Add the following code immediately above the closing `});` in your script:

```

1  return {
2      afterSubmit: afterSubmit
3  };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a `.js` file (for example, `ue_marketingOrder.js`).

Step 3: Create the Script Record

Now that you've completed the script, you can upload it to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Marketing Order
ID	_ue_marketing_order NetSuite prefixes the ID with customscript .
Description	This script sets the amount value to 0 for all items on a marketing (sales) order.

5. Set any other fields on the script record as required.

6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_ue_marketing_order NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It is normally best practice to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Test the Solution

After you create the script record and deploy your script, you can test it by creating a sales order with multiple items, marking it as a marketing order, and saving it. After it is saved, verify that all item values are 0.

To test your solution:

1. Go to Transaction > Sales > Enter Sales Orders.
2. Select the Standard Sales Order form and verify that the **Marketing Order** box appears on the form. This field appears on all standard and custom sales order forms, but for this tutorial, you can use the Standard Sales Order form.
3. Add multiple items to the sales order. You don't need to change any field for the item unless you want to. Ensure that the **Amount** field for each item isn't already 0.
4. Fill out all other required fields.

5. Check the **Marketing Order** box.
6. Click **Save**.
7. View the new sales order and verify that the **Marketing Order** box is checked and that the **Amount** for all items is 0.

Add Custom Button to Run a Suitelet

This use case shows you how to add a custom button on a sales order when the status is Pending Fulfillment. When you click the button, it runs a Suitelet.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Note: You can build Suitelets to do things like start a custom printing process or open a popup window to prompt users for details to create a custom record related to the order. However, creating a Suitelet isn't a part of this tutorial. You need to have your Suitelet already be developed and deployed. For more information, see the help topic [SuiteScript 2.x Suitelet Script Type](#).

Customization Details

The customization for this use case includes:

- A script parameter (Suitelet Link) to specify the Suitelet link
- A user event script triggered on the beforeLoad entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 2: Write the Script](#)
- [Step 2: Create the Script Record](#)
- [Step 3: Deploy the Script](#)
- [Step 4: Create and Set the Script Parameter](#)
- [Step 5: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	Ensure the following features are enabled in your account: ■ Server SuiteScript – Lets you attach server scripts to records. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ Sales Order – Lets you create, edit, and save sales orders (before fulfillment or payment). For more information, see the help topic Enabling Features.
--------------------------	---

Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Sales Orders – Edit access ■ Transaction Forms – Edit access For more information, see the help topic NetSuite Permissions Overview
Other Requirements	You need to have a Suitelet already developed and deployed in NetSuite. The custom button you add in this tutorial opens the Suitelet in a new window.

Step 2: Write the Script

This script adds a custom button to the sales order form if the status is Pending Fulfillment. When you click the new button, it opens a window for the Suitelet.

Script Summary

The following table summarizes the script:

Script: Add Button to Execute Suitelet	
Script Type	SuiteScript 2.x User Event Script Type
Modules Used	■ N/runtime Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ beforeLoad(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, `sl_executeSuiteletButton.js`).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

1 | /**

```

2  * @NApiVersion 2.1
3  * @NScriptType UserEventScript
4  * @NModuleScope SameAccount
5  */
6
7  define(['N/runtime', 'N/log'], (runtime, log) => {
8      function beforeLoad(scriptContext) {
9          try {
10             const recCurrent = scriptContext.newRecord;
11             const objForm = scriptContext.form;
12             const stStatus = recCurrent.getValue({
13                 fieldId: 'status'
14             });
15             const stSuiteletLinkParam = runtime.getCurrentScript().getParameter({
16                 name: 'custscript_suiteletlink'
17             });
18             const suiteletURL = '\"' + stSuiteletLinkParam + '\"';
19             if (stStatus === 'Pending Fulfillment') {
20                 objForm.addButton({
21                     id: 'custpage_suiteletbutton',
22                     label: 'Open Suitelet',
23                     functionName : 'window.open(' + suiteletURL + ')',
24                 });
25             }
26             } catch(error) {
27                 log.error({
28                     title: 'beforeLoad_addButton',
29                     details: error.message
30                 });
31             }
32         }
33         return {
34             beforeLoad: beforeLoad
35         };
36     });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Add the custom button to the sales order
- Create the return statement
- Save your script file

 **Note:** The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 user event script. The script uses two SuiteScript modules specified in the define statement:

- N/runtime - provides access to runtime parameters
- N/log – allows you to log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/log'], (runtime, log) => {
8  });

```

Create the entry point function

This script is triggered on the `beforeLoad` entry point when a sales order in the Pending Fulfillment state is loaded. The script uses a try-catch block to log any errors that might occur during script processing. Most of the script code is added to the try block.

Add the following function definition and initial try-catch block statements at the top of the `define` function:

```

1  function beforeLoad(scriptContext) {
2      try {
3      } catch(error) {
4          log.error({
5              title: 'beforeLoad_addButton',
6              details: error.message
7          });
8      }
9  }

```

Add the custom button to the sales order

You want this script to add the custom button only if the sales order is in the Pending Fulfillment state.

Add the following code inside the try block:

```

1  const recCurrent = scriptContext.newRecord;
2  const objForm = scriptContext.form;
3  const stStatus = recCurrent.getValue({
4      fieldId: 'status'
5  });
6  const stSuiteletLinkParam = runtime.getCurrentScript().getParameter({
7      name: 'custscript_suiteletlink'
8  });
9  const suiteletURL = '\"' + stSuiteletLinkParam + '\"';
10 if (stStatus === 'Pending Fulfillment') {
11     objForm.addButton({
12         id: 'custpage_suiteletbutton',
13         label: 'Open Suitelet',
14         functionName: 'window.open(' + suiteletURL + ')'
15     });
16 }

```

Create the return statement

This script associates the `beforeLoad` function with the `beforeLoad` user event entry point.

Add the following code immediately above the closing `});` in your script:

```
1 return {
2     beforeLoad: beforeLoad
3 };
```

Save your script file

You need to save your script file so you can load it to the NetSuite File Cabinet. Before you save, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested indentation.

When you are happy with your script, save it as a .js file (for example, `sl_executeSuiteletButton.js`).

Step 2: Create the Script Record

Now that you've completed the script, upload the file to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Add Button to Execute Suitelet
ID	_ue_custom_suitelet_button NetSuite prefixes the ID with customscript .
Description	This script adds a button to the sales order form. When clicked, the button opens the Suitelet URL provided on the Parameters subtab. You may want to include the button name you choose in the description to help others identify which button this scripts controls.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 3: Deploy the Script](#).

Step 3: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 2: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_ue_custom_suitelet_button NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's usually best to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

3. Click **Save**.

Step 4: Create and Set the Script Parameter

This script uses a script parameter to specify the Suitelet link. When you click the custom button, a window opens to display the Suitelet. You can add the script parameter for this script after the script is deployed on the sales order record.

For more information about creating and setting script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

To create the script parameter:

1. Go to Customization > Scripting > Scripts.
2. Find your script in the list and click **Edit** next to the script name.
3. On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.
4. On the Script Field page, enter the following values:

Label	ID	Type	Description	Preference
Suitelet Link	_suiteletlink NetSuite prefixes the ID with custscript , so the ID used in the script is custscript_suiteletlink .	Hyperlink	This is the location of the Suitelet.	Leave blank.

5. To save the script parameter, click **Save**. The Script page appears.
6. To save the script record with the updated script parameter, click **Save**.

After you create the parameter, you need to set its value on the script deployment record that you created in Step 3.

To set the script parameter:

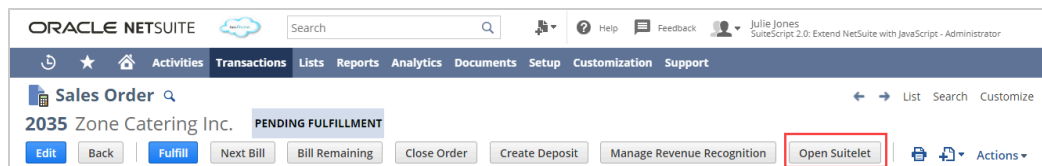
1. Go to Customization > Scripting > Script Deployments.
2. Locate your script deployment in the list of script deployments and click **Edit** next to the script deployment name.
3. On the **Parameters** subtab, enter the URL from the Suitelet deployment that you want to display when the button is clicked.
4. Click **Save**.

Step 5: Test the Solution

After you create the script record and deploy your script, you can test it by editing a sales order that is in Pending Fulfillment status.

To test your solution:

1. Go to Transactions > Sales > Enter Sales Orders > List.
2. Click **View** next to a sales order that's in Pending Fulfillment status.
3. Verify that your custom button, labeled **Open Suitelet**, appears on the sales order form. For example:



4. Click the button and make sure your Suitelet opens in a new window.

Calculate Commission on Sales Orders

This customization calculates a custom commission amount when a sales order is saved. The calculated commission amount is displayed in a custom field on the sales order record. The commission is based on each item included in the sales order: MSRP, net distributor costs ($\text{MSRP} * 0.5$), and the price sold to the customer.

NetSuite has an Employee Commissions feature, but this customization calculates and sets a simple custom commission amount in a custom field without using the feature. For more information about the Employee Commissions feature, which can calculate commission for employees with different schedules and methods of calculation, see the help topic [Commissions](#).

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

The customization for this use case includes:

- A custom field (MSRP) to store the MSRP for an item
- A custom field (Custom Commission) to store the calculated commission
- A user event script triggered on the afterSubmit entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Fields](#)
- [Step 2: Write the Script](#)
- [Step 3: Create the Script Record](#)
- [Step 4: Deploy the Script](#)
- [Step 5: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	The following features must be enabled in your account: ■ Server SuiteScript – Lets you attach server scripts to records. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ A/R – Lets you create, edit, and save invoices. ■ Sales Order – Lets you create, edit, and save sales orders (before fulfillment or payment). For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Transaction Body Fields – Create access ■ Transaction Line Fields – Create access ■ Sales Orders – Edit and Delete access For more information, see the help topic NetSuite Permissions Overview.
Other Requirements	None

Step 1: Create the Custom Fields

This customization uses two custom fields. The **Calculated Commission** field stores the calculated commission when the sales order is saved. It is a transaction body field added to the sales order record. The **MSRP** field is displayed on the **Items** sublist of a sales order and stores the MSRP value for an item. It's a transaction line field added to the sales order record.

To create the custom Calculated Commission field:

1. Go to Customization > Lists, Records, Fields > Transaction Body Fields > New and enter the following values:

Field	Value
Label	Calculated Commission
ID	_commission_amount
Type	Currency
Store Value	Checked
Applies To	Sale
Display	Subtab: Main

2. Click **Save**.

To create the custom MSRP field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New and enter the following values:

Field	Value
Label	MSRP
ID	_salesorder_msrp
Type	Currency
Store Value	Checked
Applies To	Sale Item

2. Click **Save**.

For more information about creating custom components, see the following help topics:

- [Creating a Custom Field](#)
- [Creating Custom Transaction Body Fields](#)
- [Creating Custom Transaction Line Fields](#)

Step 2: Write the Script

This script calculates a custom commission amount when a sales order is saved. The calculated commission amount is displayed in a custom field on the sales order record. The commission is based on the sum of commission amounts for each item in the sales order: MSRP, the net distributor costs (MSRP * 0.5), and the price sold to the customer.

Script Summary

The following table summarizes the script:

Script: Calculate Commission	
Script Type	SuiteScript 2.x User Event Script Type

Script: Calculate Commission	
Modules Used	■ N/record Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ afterSubmit(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, ue_calculateCommission.js).

If you'd rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/log'], (record, log) => {
8      function afterSubmit(scriptContext) {
9          const stMethodName = 'afterSubmit_calculateCommission';
10         try {
11             if (scriptContext.type !== scriptContext.UserEventType.CREATE && scriptContext.type !== scriptContext.UserEvent
Type.EDIT) {
12                 return;
13             }
14             let stItemType = null;
15             let flMSRPTotalAmt = 0.00;
16             let flMSRPAmt = 0.00;
17             let flNetDistributorCost = 0.00;
18             let flCommissionAmount = null;
19             let flQuantity = 0.00;
20             const recSalesOrder = scriptContext.newRecord;
21             let flSubtotal = parseFloat(recSalesOrder.getValue({
22                 fieldId: 'subtotal'
23             }));
24             const numItems = recSalesOrder.getLineCount({
25                 sublistId: 'item'
26             });
27             for (let intLinenum = 0; intLinenum < numItems; intLinenum++) { flMSRPAmt = parseFloat(recSalesOrder.getSublist
Value({ sublistId: 'item', fieldId: 'custcol_salesorder_msrp', line: intLinenum })); flQuantity = parseFloat(recSalesOr
der.getSublistValue({ sublistId: 'item', fieldId: 'quantity', line: intLinenum })); stItemType = recSalesOrder.getSublist
Value({ sublistId: 'item', fieldId: 'itemtype', line: intLinenum }); if (stItemType !== 'Discount' && stItemType !== 'Subto
tal' && stItemType !== 'Markup') { flMSRPTotalAmt = flMSRPTotalAmt + (flMSRPAmt * flQuantity); } } flNetDistributorCost = flMSRPTo

```

```

    talAmt * 0.5; if (flSubtotal === flNetDistributorCost) { flCommissionAmount = flSubtotal * 0.10; } else if ((flSubtotal > flNetDis
tributorCost) && (flSubtotal <= flMSRPTotalAmt)) { flCommissionAmount = flNetDistributorCost * 0.10 + (flSubtotal - flNetDistribu
torCost) * 0.75; } else { if (flSubtotal > flMSRPTotalAmt) {
28         flCommissionAmount = flNetDistributorCost * 0.10 + (flMSRPTotalAmt - flNetDistributorCost) * 0.75 + (flSubtotal
    - flMSRPTotalAmt) * 0.5;
29     }
30 }
31 const thisSalesOrderID = recSalesOrder.id;
32 const updateSalesOrder = record.load({
33     type: record.Type.SALES_ORDER,
34     id: thisSalesOrderID
35 });
36 updateSalesOrder.setValue({
37     fieldId: 'custbody_commission_amount',
38     value: flCommissionAmount
39 });
40 const updateSalesOrderID = updateSalesOrder.save();
41 } catch(e) {
42     log.debug({
43         title: stMethodName,
44         details: ' - Exit (Catch)- '
45     });
46     if (e.getDetails !== undefined) {
47         log.error({
48             title: 'Process Error',
49             details: JSON.stringify(e)
50         });
51         throw e;
52     } else {
53         log.error({
54             title: 'Unexpected Error',
55             details: JSON.stringify(e)
56         });
57         throw error.create({
58             name: 'Unexpected Error',
59             message: JSON.stringify(e)
60         });
61     }
62 }
63 }
64 return {
65     afterSubmit: afterSubmit
66 };
67 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Check the scriptContext
- Initialize variables
- Get the subtotal from the sales order record
- Process items on the sales order
- Calculate the net distributor cost
- Calculate the commission amount
- Set the commission amount on the sales order
- Save the sales order
- Create the return statement

■ Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.x (2.0 or 2.1) user event script. The script uses two SuiteScript modules specified in the define statement:

- N/record – provides access to the sales order record
- N/log – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/log'], (record, log) => {
8  });

```

Create the entry point function

This script is triggered on the afterSubmit entry point when a sales order is saved. A try-catch block logs any errors that might occur during script processing. Most of the script code is placed in the try block.

Add the following function definition and initial try-catch block statements at the top of the define function:

```

1  function afterSubmit(scriptContext) {
2      const stMethodName = 'afterSubmit_calculateCommission';
3      try {
4      } catch(e) {
5          log.debug({
6              title: stMethodName,
7              details: ' - Exit (Catch)- '
8          });
9          if (e.getDetails !== undefined) {
10             log.error({
11                 title: 'Process Error',
12                 details: JSON.stringify(e)
13             });
14             throw e;
15         } else {
16             log.error({
17                 title: 'Unexpected Error',
18                 details: JSON.stringify(e)

```



```

19         });
20         throw error.create({
21             name: 'Unexpected Error',
22             message: JSON.stringify(e)
23         });
24     }
25 }
26 }

```

Check the scriptContext

You want this script to run only when the user creates or edits a sales order.

Add the following code at the top of the try block:

```

1 if (scriptContext.type !== scriptContext.UserEventType.CREATE && scriptContext.type !== scriptContext.UserEventType.EDIT) {
2     return;
3 }

```

Initialize variables

This script uses several variables to store the type, quantity, MSRP, net distributor cost for each item, the sum of all MSRP amounts, and the calculated commission amount. You need to initialize these variables.

Add the following code into the try block:

```

1 let stItemType = null;
2 let flMSRPTotalAmt = 0.00;
3 let flMSRPAmt = 0.00;
4 let flNetDistributorCost = 0.00;
5 let flCommissionAmount = null;
6 let flQuantity = 0.00;

```

Get the subtotal from the sales order record

You want to get the subtotal from the record to calculate the commission.

Add the following code into the try block:

```

1 const recSalesOrder = scriptContext.newRecord;
2 let flSubtotal = parseFloat(recSalesOrder.getValue({
3     fieldId: 'subtotal'
4 }));

```

Process items on the sales order

You need to process each item on the sales order needs by getting the MSRP, quantity, and type for the item. Then, you make sure the it's not a discount, markup, or subtotal item since those types of items aren't used to calculate the commission. To process the items, add code for the following:

- Get the item MSRPs
- Get the item quantity and type
- Calculate the MSRP for all items that are not discount, markup, or subtotal items

To begin, add the following code into the try block:

```
1 | const numItems = recSalesOrder.getLineCount({
2 |   sublistId: 'item'
3 | });
4 | for (let intLinenum = 0; intLinenum < numItems; intLinenum++) {
5 | }
```

Get the item MSRPs

Now, you get the MSRP for each item.

Add the following code as the first statement inside the for block:

```
1 | fIMSRPAmt = parseFloat(recSalesOrder.getSublistValue({
2 |   sublistId: 'item',
3 |   fieldId: 'custcol_salesorder_msrp',
4 |   line: intLinenum
5 | }));
```

Get the item quantity and type

And you get the item quantity and the item type.

Add the following code into the for block:

```
1 | fQuantity = parseFloat(recSalesOrder.getSublistValue({
2 |   sublistId: 'item',
3 |   fieldId: 'quantity',
4 |   line: intLinenum
5 | }));
6 | stItemType = recSalesOrder.getSublistValue({
7 |   sublistId: 'item',
8 |   fieldId: 'itemtype',
9 |   line: intLinenum
10 | });
```

Calculate the MSRP for all items that are not discount, markup, or subtotal items

This script sums all item MSRPs to calculate the total MSRP, but doesn't include discount, markup, and subtotal items.

Add the following code into the for block:

```
1 | if (stItemType !== 'Discount' && stItemType !== 'Subtotal' && stItemType !== 'Markup') {
2 |   fIMSRPTotalAmt = fIMSRPTotalAmt + (fIMSRPAmt * fQuantity);
3 | }
```

Calculate the net distributor cost

Now, calculate the net distributor cost. You'll use this to calculate the commission.

Add the following code after the end of the for block, inside the try block:

```
1 | fNetDistributorCost = fIMSRPTotalAmt * 0.5;
```

Calculate the commission amount

You need to calculate the commission amount for the entire sales order. The commission is based on the sum of commission amounts for each item: MSRP, net distributor costs (MSRP * 0.1), and the price sold to the customer.

Add the following code into the try block:

```
1 | if (f1Subtotal === f1NetDistributorCost) {
2 |     f1CommissionAmount = f1Subtotal * 0.10;
3 | } else if ((f1Subtotal > f1NetDistributorCost) && (f1Subtotal <= f1MSRPTotalAmt)) { f1CommissionAmount = f1NetDistributor
Cost * 0.10 + (f1Subtotal - f1NetDistributorCost) * 0.75; } else { if (f1Subtotal > f1MSRPTotalAmt) {
4 |     f1CommissionAmount = f1NetDistributorCost * 0.10 + (f1MSRPTotalAmt - f1NetDistributorCost) * 0.75 + (f1Subtotal - f1MSRPTo
talAmt) * 0.5;
5 |     }
6 | }
```

Set the commission amount on the sales order

Finally, set the final commission amount in the custom **Commission Amount** field on the header of the sales order record.

Add the following code into the try block:

```
1 | const thisSalesOrderID = recSalesOrder.id;
2 | const updateSalesOrder = record.load({
3 |     type: record.Type.SALES_ORDER,
4 |     id: thisSalesOrderID
5 | });
6 | updateSalesOrder.setValue({
7 |     fieldId: 'custbody_commission_amount',
8 |     value: f1CommissionAmount
9 | });
```

Save the sales order

After you set the commission amount, you want to save the sales order.

Add the following code into the try block:

```
1 | const updateSalesOrderID = updateSalesOrder.save();
```

Create the return statement

This script associates the `afterSubmit` function with the `afterSubmit` user event script entry point.

Add the following code immediately above the closing `});` in your script:

```
1 | return {
2 |     afterSubmit: afterSubmit
3 | };
```

Save your script file

You need to save your script file so you can load it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you are happy with your script, save it as a .js file (for example, ue_calculateCommission.js).

Step 3: Create the Script Record

Now that you've completed the script, you can upload it to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Calculate Commission
ID	_ue_calculate_commission NetSuite prefixes the ID with customscript .
Description	This script calculates a custom commission based on each item in a sales order.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_ue_calculate_commission NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.

Field	Value
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Test the Solution

After you create the script record and deploy your script, you can test it by creating a new sales order with several items.

To test the solution:

1. Go to Transaction > Sales > Enter Sales Orders to create a new sales order.
2. In the **Custom Form** field, select the **Standard Sales Order** form. The custom fields appear on all standard and custom sales order forms, but for this tutorial you can use the standard form.
3. Add multiple items to the sales order and set a value in the **MSRP** field for each item.
4. Fill out all other required fields.
5. Save the sales order.
6. Review the sales order and verify the calculated commission in the **Calculated Commission** field.

Copy a Value to the Item Column

This use case shows you how to use a custom transaction line field and a client script to change the value of a standard column field, such as the **Item** column.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Scenario

You are a store owner and you have employees who can recognize several digits of the UPC code for your core billing items. You would like to make those UPC codes available in a small drop down list for the employee to select from. You plan to enter your items by name in NetSuite. However, the item list is long and you want employees to select a UPC code instead of scrolling through a long list of item names. You can customize the invoice form for these employees to disable the **Item** column and add a custom transaction line field where they can select billing items using the UPC code.

Customization Details

The customization for this use case includes:

- A custom field (Billing Item) to select an item by UPC code
- A custom list (Billing Items) to store products with UPC codes
- An item record to test the implementation of UPC code selection
- A custom form to be used as a custom billing item invoice
- A client script triggered on the fieldChanged entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Field, List, Record, and Form](#)
- [Step 2: Write the Script](#)
- [Step 3: Create the Script Record](#)
- [Step 4: Deploy the Script](#)
- [Step 5: Test the Solution](#)

 **Note:** This tutorial best shows how a fieldChanged client script can work for various applications. You can also limit the number of items available on a transaction using a saved search and a custom transaction form. See the help topic [Filtering the Items Dropdown List on Transactions](#).

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	The following features must be enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ A/R – Lets you create, edit and save invoices. For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Transaction Line Fields – Create access ■ Transaction Forms – Create access ■ Invoices – Create access For more information, see the help topic NetSuite Permissions Overview.
Other Requirements	None

Step 1: Create the Custom Field, List, Record, and Form

This customization uses a custom field called **Billing Item** to select an item by UPC code. This field displays as a column on transaction line items. For the custom field to find matching items, you'll create an item record with a matching UPC code.

This customization also uses a custom list called **Billing Items**, which provides values to the **Billing Item** field.

This customization also uses a custom invoice form so that users can use the custom field, list, and record. All standard forms in NetSuite remain the same. Every form you create from a standard form is a custom version of that form. For this tutorial, you'll customize the standard invoice form to add your new transaction line field as a column on the **Items** subtab.

To create the Billing Item transaction line field and Billing Items custom list:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New.
2. Enter or select values for fields as listed in the following table:

Field	Value
Label	Billing Item
ID	_billingitem
Description	This custom field is for the user to select an item that will be copied to the standard Item column.
Type	List/Record
List/Record	Select -New- to create a new list of UPC codes to choose from. For this tutorial, enter Billing Items in the Name field, and enter a single list value of 704875 in the Value field on the Values subtab. To access this custom list later, go to Customization > Lists, Records, & Fields > Lists.
Store Value	Checked
Applies To Tab	Leave all values cleared. Checking boxes here adds the custom column to standard forms. For this tutorial, the field will be added to a custom form only.

3. Click **Save**.

To create the item record:

1. Go to Lists > Accounting > Items > New.
2. Select **For Sale** under Non-Inventory Item.
3. Enter or select values for fields as listed in the following table:

Field	Value
Item Name/Number	Bulk Shampoo
UPC Code	704875 , the code you entered for the Billing Item.

4. Click **Save**.

To customize the invoice form:

1. Go to Customization > Forms > Transaction Forms.
2. Click Customize next to Standard Product Invoice.
3. Enter or select values for fields as listed in the following table:

Field	Value
Name	Billing Item Invoice

Field	Value
ID	<code>_billingitem</code>
Form is Preferred	Check this box if you want this custom form to be the default invoice form for everyone. For this tutorial, you don't need to check this box.
Sublist Fields subtab > Billing Item	Your custom field, Billing Item, may be at the bottom of the list. Click the dot grid to the left of the field, and then move the field up to the top of the list, under the standard Item field.  Note: Click the Show box to the right of the field name. If you don't check the Show box, your custom field won't appear on your custom invoice form. This box isn't checked by default. Fields are listed top to bottom as they appear right to left on the invoice form.

4. Click **Save**.

For more information about creating custom fields, lists, and forms, see the following help topics:

- [Creating a Custom Field](#)
- [Creating a Custom List](#)
- [Creating Custom Entry and Transaction Forms](#)
- [Creating Custom Transaction Line Fields](#)

Step 2: Write the Script

This script copies the value from the custom **Billing Item** field to the standard **Item** field when you select or change the **Billing Item** field.

Script Summary

The following table summarizes the script:

Script: Copy Value to Item	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your <code>define</code> or <code>require</code> statement, however, you may if you want. This tutorial does not explicitly load this module. ■ N/search Module
Entry Points	■ fieldChanged(scriptContext)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, `cs_copyValueToItem.js`).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['/N/search'], (search) => {
8      function fieldChanged (context) {
9          try {
10             const recInvoice = context.currentRecord;
11             const stCurrField = context.fieldId;
12             const stCurrSublist = context.sublistId;
13             // Get UPC code of billing item
14             if (stCurrSublist === 'item' && stCurrField === 'custcol_billingitem') {
15                 const billingitem = recInvoice.getCurrentSublistText({
16                     sublistId: 'item',
17                     fieldId: 'custcol_billingitem'
18                 });
19                 // Search for Item with Billing Item's UPC code
20                 const itemSearch = search.create({
21                     type: 'noninventoryitem',
22                     filters: [
23                         ['upccode', 'is', billingitem]
24                     ],
25                     columns: [
26                         'upccode', 'itemid'
27                     ]
28                 }).run();
29                 // Set the UPC code text to the invoice
30                 const result = itemSearch.getRange(0, 1);
31                 const itemName = result[0].getValue(itemSearch.columns[1]);
32                 recInvoice.setCurrentSublistText({
33                     sublistId: 'item',
34                     fieldId: 'item',
35                     text: itemName
36                 });
37             }
38             catch (e) {
39                 alert(e);
40             }
41         }
42         return {
43             fieldChanged: fieldChanged
44         };
45     });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- [Start with required opening lines](#)
- [Create the entry point function](#)

- Get the field and sublist that changed
- Create a search to find the Billing Item UPC Code
- Set the UPC Code Text to the Invoice
- Create the return statement
- Save your script file

 **Note:** The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need a JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 client script. The script doesn't use any SuiteScript modules except for the globally available `N/currentRecord` module, which you don't need to include in the define statement.

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

 **Note:** This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define([], function() {
8  });

```

Create the entry point function

This script is triggered on the `fieldChanged` entry point when the value for the custom **Billing Item** line field on the **Item** subtab changes. A try-catch block logs errors that might occur when the script runs. Most of the script code goes in the try block.

Add the following function definition and initial try-catch block statements at the top of the define function:

```

1  function fieldChanged(context) {
2      try {
3      } catch(e) {
4          alert(e.name + ': ' + e.message);
5      }
6  }

```

Get the field and sublist that changed

You only need to set the standard Item line field when someone changes the custom Billing Item field. You need to get the current field and sublist IDs to determine which field was changed.

Add the following code into the try block:

```
1 | const recInvoice = context.currentRecord;
2 | const stCurrField = context.fieldId;
3 | const stCurrSublist = context.sublistId;
4 | if (stCurrSublist === 'item' && stCurrField === 'custcol_billingitem') {
5 | }
```

Create a search to find the Billing Item UPC Code

Create and run a saved search to find the Billing Item by its UPC Code.

Add the following code into the try block:

```
1 | const itemSearch = search.create({
2 |     type: 'noninventoryitem',
3 |     filters: [
4 |         ['upccode', 'is', billingitem]
5 |     ],
6 |     columns: [
7 |         'upccode', 'itemid'
8 |     ]
9 | }).run();
```

Set the UPC Code Text to the Invoice

Use the results from the saved search to set the UPC code text values on the invoice.

Add the following code in the if statement in the try block:

```
1 | const result = itemSearch.getRange(0, 1);
2 | const itemName = result[0].getValue(itemSearch.columns[1]);
3 | recInvoice.setCurrentSublistText({
4 |     sublistId: 'item',
5 |     fieldId: 'item',
6 |     text: itemName
7 | });
```

Create the return statement

This script associates the fieldChanged function with the fieldChanged client entry point.

Add the following code immediately above the closing `});` in your script:

```
1 | return {
2 |     fieldChanged: fieldChanged
3 | };
```

Save your script file

You need to save your script file so you can load it to the NetSuite File Cabinet. Before you save, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you are happy with your script, save it as a .js file (for example, `cs_copyValueToItem.js`).

Step 3: Create the Script Record

Now that you've completed the script, you can upload the file to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Copy Value to Item
ID	_cs_copy_value_to_item NetSuite prefixes the ID with customscript .
Description	This script copies the value entered into a custom Billing Item field into the standard Item field.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Invoice
ID	_cs_copy_value_to_item NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug

Field	Value
	The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Test the Solution

After you create the script record and deploy your script, you can test. Select a value in the custom **Billing Item** field (on your custom form) and verify that it is copied to the standard **Item** field.

To test your solution:

1. Go to Transactions > Sales > Create Invoices to create a new invoice.
2. In the **Custom Form** field, select your custom **Billing Item Invoice** form.
3. In the **Customer** field, select any customer to test with.
4. On the **Items** subtab, in the **Billing Item** field, select **704875**. The script copies 704875 to the standard **Item** field. Because 704875 is in the **UPC Code** field on the Bulk Shampoo item record, the system automatically selects **Bulk Shampoo** in the standard **Item** field.

Set a Default Posting Period in a Custom Field

This use case shows you how to set a default posting period using a custom transaction body field and then copy that value to the standard Posting Period field. This helps limit the posting period options users see and avoids potential errors when selecting a period.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

This customization for this use case includes:

- A custom field (Preferred Posting Period) to select an open accounting period
- A custom form that disables the standard Posting Period and includes the custom Preferred Posting Period field
- A saved search used to search for open accounting periods
- A client script triggered on the pageInit and fieldChanged entry points

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Field, Form, and Saved Search](#)
- [Step 2: Write the Script](#)

- [Step 3: Create the Script Record](#)
- [Step 4: Deploy the Script](#)
- [Step 5: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	Ensure the following features are enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ A/R – Lets you create, edit and save invoices. For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Transaction Body Fields – Create access ■ Saved Searches – Edit access ■ Invoices – Create access For more information, see the help topic NetSuite Permissions Overview.
Other Requirements	None

Step 1: Create the Custom Field, Form, and Saved Search

This customization uses a custom field. The **Preferred Posting Period** field lets you select a preferred posting period. This custom field is a list/record type transaction body field with values sourced from the accounting period record.

This customization also uses a custom form. The custom invoice form disables the standard **Posting Period** field and moves the custom **Preferred Posting Period** field to the required position.

This customization also uses a saved search. The **Preferred Posting Periods** saved search finds open accounting periods that you want users can select in the custom **Preferred Posting Period** field. This search lets you filter out posting periods used only for organizational purposes or corrections, not for new transactions.

To create the custom field:

1. Go to Customization > Lists, Records, Fields > Transaction Body Fields > New.
2. Enter or select values for fields as listed in the following table:

Field	Value
Label	Preferred Posting Period

Field	Value
ID	_preferred_posting_period
Description	This custom field lets users select a preferred posting period that is copied to the standard posting period field.
Type	List/Record
List/Record	Accounting Period
Store Value	Checked
Applies To	Sale
Display	Subtab: Main

3. Click **Save**.

To customize the invoice form:

1. Go to Customization > Forms > Transaction Forms.
2. Click Customize next to the Standard Product Invoice form.
3. Enter or select values for fields as listed in the following table:

Field	Value
Name	Preferred Posting Invoice
ID	_posting_invoice NetSuite prefixes the ID with custform .
Screen Fields subtab > Main	Check Show next to Preferred Posting Period. In the Field Group column, select Primary Information . Doing so automatically moves the field up in the list. To disable the standard Posting Period field, set its Display Type field to Inline Text or Disabled .
Form is Preferred	Check this box if you want this custom form to be the default invoice form for everyone. For this tutorial, you don't need to check this box.

4. Click **Save**.

To create the saved search:

1. Go to Reports > Saved Searches > All Saved Searches > New.
2. Select Accounting Period.
3. On the Saved Accounting Period Search page, enter a **Search Title**. If others might use this search, indicate that the search is used in a script and should not be edited. For example, enter **Preferred Posting Periods***USED IN SCRIPT*****.
4. Enter an ID, such as **_open_accounting_periods**. NetSuite automatically adds **customsearch** as a prefix.
5. On the **Criteria** subtab, enter the following:

Filter	Description
Closed	No
Period Name	does not start with – Q

Filter	Description
Period Name	does not start with – FY

6. On the **Results** subtab, in the **Sort By** field, select **Start Date**.
7. Click **Save & Run** to confirm that your search runs and returns the correct results.

For more information about creating custom fields, forms, and searches, see the following help topics:

- [Creating a Custom Field](#)
- [Creating a Custom List](#)
- [Creating Custom Entry and Transaction Forms](#)
- [Creating Custom Transaction Line Fields](#)
- [Saved Searches](#)

Step 2: Write the Script

This script sets the custom **Preferred Posting Period** field to the most recent posting period from your saved search results. It then copies the value you select to the standard **Posting Period** field.

Script Summary

The following table summarizes the script:

Script: Preferred Posting Period	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/runtime Module ■ N/search Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial doesn't explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects named 'log'.
Entry Points	■ fieldChanged(scriptContext) ■ pageInit(scriptContext)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, cs_preferredPostingPeriod.js).

If you'd rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/search', 'N/log'], (runtime, search, log) => {
8      function pageInit(context) {
9          try {
10             if (context.mode === 'copy' || context.mode === 'create') {
11                 const record = context.currentRecord;
12                 const searchOpenAccountingPeriods = search.load({
13                     id: 'customsearch_open_accounting_periods'
14                 });
15                 const resultsFromSearch = searchOpenAccountingPeriods.run().getRange({
16                     start: 0,
17                     end: 1
18                 });
19                 record.setValue({
20                     fieldId: 'custbody_preferred_posting_period',
21                     value: resultsFromSearch[0].id
22                 });
23                 record.setValue({
24                     fieldId: 'postingperiod',
25                     value: resultsFromSearch[0].id
26                 });
27             }
28         } catch(e) {
29             log.error({
30                 title: 'ERROR',
31                 details: e
32             });
33         }
34     }
35     function fieldChanged(context) {
36         try {
37             if (context.fieldId === 'custbody_preferred_posting_period') {
38                 const record = context.currentRecord;
39                 const prefPostPeriod = record.getValue({
40                     fieldId: 'custbody_preferred_posting_period'
41                 });
42                 record.setValue({
43                     fieldId: 'postingperiod',
44                     value: prefPostPeriod
45                 });
46             } else {
47                 return;
48             }
49         } catch(e) {
50             log.error({
51                 title: 'ERROR',
52                 details: e
53             });
54         }
55     }
56     return {
57         pageInit: pageInit,
58         fieldChanged: fieldChanged
59     }
60 }

```

```

59 | };
60 | });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point functions
- Set up field defaults
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments indicate that it's a SuiteScript 2.1 client script. The script uses three SuiteScript modules specified in the define statement:

- `N/runtime` – provides access to runtime parameters
- `N/search` – lets you create and run on-demand or saved searches, and work with the results
- `N/log` – lets you log execution details

Start a new script file using any text editor and place the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1 | /**
2 |  * @ApiVersion 2.1
3 |  * @NScriptType ClientScript
4 |  * @NModuleScope SameAccount
5 |  */
6 |
7 | define(['N/runtime', 'N/search', 'N/log'], (runtime, search, log) => {
8 | });

```

Create the entry point functions

This script is triggered on the `pageInit` entry point when you create an invoice, and on the `fieldChanged` entry point when you change the **Preferred Posting Period** field. Both entry point functions use a try-catch block to log any errors that occur during script processing. Most of the script code goes in the try blocks.

Add the following function definitions and initial try-catch block statements at the top of the define function:

```

1 function pageInit(context) {
2   try {
3   } catch(e) {
4     log.error({
5       title: 'ERROR',
6       details: JSON.stringify(e)
7     });
8   }
9 }
10 function fieldChanged(context) {
11   try {
12   } catch(e) {
13     log.error({
14       title: 'ERROR',
15       details: JSON.stringify(e)
16     });
17   }
18 }

```

Update the standard posting period field

If someone changes the value in the custom **Preferred Posting Period** field, we want the standard **Posting Period** field to update as well for reporting consistency, even if they can't edit the standard field.

Add the following code into the try block of the fieldChanged function:

```

1 if (context.fieldId === 'custbody_preferred_posting_period') {
2   const record = context.currentRecord;
3   const prefPostPeriod = record.getValue({
4     fieldId: 'custbody_preferred_posting_period'
5   });
6   record.setValue({
7     fieldId: 'postingperiod',
8     value: prefPostPeriod
9   });
10 } else {
11   return;
12 }

```

Set up field defaults

When the user initiates an invoice, we need to know if the transaction is being copied or created. If it is, run the saved search to get the list of open accounting periods for the custom field.

Add the following code into the try block of the pageInit function:

```

1 if (context.mode === 'copy' || context.mode === 'create') {
2   const record = context.currentRecord;
3   const searchOpenAccountingPeriods = search.load({
4     id: customsearch_open_accounting_periods
5   });
6   const resultsFromSearch = searchOpenAccountingPeriods.run().getRange({
7     start: 0,
8     end: 1
9   });
10  record.setValue({
11    fieldId: 'custbody_preferred_posting_period',
12    value: resultsFromSearch[0].id
13  });

```

```

14 |     record.setValue({
15 |         fieldId: 'postingperiod',
16 |         value: resultsFromSearch[0].id
17 |     });
18 | }

```

Create the return statement

This script associates the `pageInit` function with the `pageInit` client script entry point and the `functionChanged` function with the `functionChanged` client script entry point.

Add the following code immediately above the closing `});` in your script:

```

1 | return {
2 |     pageInit: pageInit,
3 |     fieldChanged: fieldChanged
4 | };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested indentation.

When you are happy with your script, save it as a `.js` file (for example, `cs_preferredPostingPeriod.js`).

Step 3: Create the Script Record

Now that you've completed the script, you can upload the file to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Preferred Posting Period
ID	_cs_preferred_posting_period NetSuite prefixes the ID with customscript .
Description	This script lets the user select a preferred posting period that is then copied into the standard posting period field.

5. Set any other fields on the script record as required.

6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Invoice
ID	_cs_preferred_posting_period NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Test the Solution

After you create the script record and deploy your script, you can test it by creating an invoice and confirming that the current posting period in the **Preferred Posting Period** field is copied to the standard **Posting Period** field.

To test your solution:

1. Go to Transactions > Sales > Create Invoices to create a new invoice.
2. In the **Custom Form** field, select **Preferred Posting Invoice**.
3. Confirm the following results:
 - The **Preferred Posting Period** field appears at the top of the form
 - The most recent open period from your saved search displays in the **Preferred Posting Period** field
 - The most recent open period from your saved search results is also copied into the standard **Posting Period** field (if you left it visible on the form)

For example:

The screenshot shows the Oracle NetSuite interface for creating an invoice. The 'Primary Information' section contains several fields. A red box highlights the 'PREFERRED POSTING PERIOD' dropdown menu, which is currently set to 'Jun 2020'. Other visible fields include 'CUSTOM FORM' (set to 'Preferred Posting Invoice'), 'INVOICE #' (1061), 'CUSTOMER/JOB' (Bailey's Training Services), 'DATE' (6/3/2020), 'START DATE', 'END DATE', 'DUE DATE' (7/3/2020), 'PO #', and 'MEMO'. A 'Summary' table on the right shows various costs totaling 0.00.

Summary	
SUBTOTAL	0.00
DISCOUNT ITEM	0.00
TAX	0.00
SHIPPING COST	
HANDLING COST	
GIFT CERTIFICATE	0.00
TOTAL	0.00

Note: Your Invoice page may look different from the preceding example, depending on how you created your customized Preferred Posting Invoice. You may have set fields differently, so your form may look different.

Track Customer Deposit Balances

This use case shows you how to track deposit and refund balances on sales orders, which you could use to track layaways. You could also build other customizations that require the customer deposits, deposit refunds, and gift certificates applied to a sales order before it's invoiced.

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

This customization for this use case includes:

- A custom field (Amount Deposited) to store the calculated total amount of deposits made on the sales order
- A custom field (Balance Remaining) to store the balance remaining on the sales order
- A saved search (Deposit Balance on Sales Order) to determine the balance of customer deposits
- Four user event scripts triggered on the beforeSubmit and afterSubmit entry points

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Fields and Saved Search](#)
- [Step 2: Write the Set Custom Fields User Event Script](#)
- [Step 3: Write the Update Custom Fields User Event Script](#)
- [Step 4: Write the Handle Refund User Event Script](#)

- [Step 5: Write the Handle Deletion of a Deposit Application User Event Script](#)
- [Step 6: Create the Script Records](#)
- [Step 7: Deploy the Scripts](#)
- [Step 8: Test the Solution](#)

This tutorial is also available as a [Complete SuiteCloud Project](#).

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	The following features must be enabled in your account: ■ Server SuiteScript – Lets you attach server scripts to records. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ A/R – Lets you create, edit, and save invoices. ■ Sales Order – Lets you create, edit, and save sales orders (before fulfillment or payment). ■ Advanced Shipping – This feature separates the fulfillments from invoices. For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Transaction Body Fields – Create access ■ Saved Searches – Edit access ■ Sales Orders – Edit and Delete access ■ Customer Deposits – Edit and Delete access ■ Customer Refunds – Edit and Delete access ■ Deposit Applications – Delete access For more information, see the help topic NetSuite Permissions Overview.
Other Requirements	None

Step 1: Create the Custom Fields and Saved Search

This customization uses two custom fields. The **Amount Deposited** field stores the calculated total deposits made on the sales order. The **Balance Remaining** field stores the balance remaining on the sales order.

This customization uses a transaction saved search with deposit balances on sales orders. When calculating the balance of customer deposits, you need to include related deposit applications in case any deposits were refunded. This saved search helps you do that. This saved search also considers if gift certificates or discounts have been applied to the sales order.

To create the Amount Deposited transaction body field:

1. Go to Customization > Lists, Records, Fields > Transaction Body Fields > New.

2. Enter values for each field as listed in the following table:

Field	Value
Label	Amount Deposited
ID	_total_deposit_paid
Type	Currency
Store Value	Checked
Applies To	Sale , unless you want to hide the field by default and edit a custom form to show the field
Display > Subtab	Main
Display > Display Type	Inline Text
Display > Help	This field is set by a script to show the total amount deposited on the sales order.
Validation & Defaulting > Default Value	0.00

3. Click **Save**.

To create the Balance Remaining transaction body field:

1. Go to Customization > Lists, Records, Fields > Transaction Body Fields > New.
2. Enter values for each field as listed in the following table:

Field	Value
Label	Balance Remaining
ID	_balance_remaining
Type	Currency
Store Value	Checked
Applies To	Sale , unless you want to hide the field by default and edit a custom form to show the field.
Display > Subtab	Main
Display > Display Type	Inline Text
Display > Help	This field is set by a script to show the remaining balance on the sales order after deposits, refunds, and gift certificates.
Validation & Defaulting > Default Value	0.00

3. Click **Save**.

The saved search includes criteria and results settings for creating a transaction saved search with deposit balances on sales orders. The saved search is complex because it uses expressions and formulas.

To create the saved search:

1. Go to Reports > Saved Searches > All Saved Searches > New.
2. Select Transaction.
3. On the Saved Transaction Search page, enter or select the following values:

Field	Value																																																						
Search Title	***USED IN SCRIPT*** Deposit Balance on Sales Orders																																																						
ID	_sobalancedue																																																						
Criteria > Use Expressions	Standard • Summary <table><tr><th>NOT</th><th>PARENS</th><th>FILTER *</th><th>DESCRIPTION *</th><th>FORMULA</th><th>PARENS</th><th>AND/OR</th></tr><tr><td></td><td>((</td><td>Main Line</td><td>is true</td><td></td><td></td><td>And</td></tr><tr><td></td><td></td><td>Type</td><td>is any of Customer Deposit, Deposit Application</td><td></td><td></td><td>And</td></tr><tr><td></td><td>(</td><td>Applied To Transaction</td><td>is not none</td><td></td><td></td><td>Or</td></tr><tr><td></td><td></td><td>Applying Transaction</td><td>is not none</td><td></td><td>)</td><td>Or</td></tr><tr><td></td><td>(</td><td>Type</td><td>is Sales Order</td><td></td><td></td><td>And</td></tr><tr><td></td><td></td><td>Account</td><td>is 62400 Miscellaneous</td><td></td><td>)</td><td></td></tr></table> In the Description for the Account filter, you need to set the value for the correct account in your COA.	NOT	PARENS	FILTER *	DESCRIPTION *	FORMULA	PARENS	AND/OR		((	Main Line	is true			And			Type	is any of Customer Deposit, Deposit Application			And		(	Applied To Transaction	is not none			Or			Applying Transaction	is not none		)	Or		(	Type	is Sales Order			And			Account	is 62400 Miscellaneous		)						
NOT	PARENS	FILTER *	DESCRIPTION *	FORMULA	PARENS	AND/OR																																																	
	((	Main Line	is true			And																																																	
		Type	is any of Customer Deposit, Deposit Application			And																																																	
	(	Applied To Transaction	is not none			Or																																																	
		Applying Transaction	is not none		)	Or																																																	
	(	Type	is Sales Order			And																																																	
		Account	is 62400 Miscellaneous		)																																																		
Results	<table><tr><th>FIELD *</th><th>SUMMARY TYPE</th><th>FUNCTION</th><th>FORMULA</th><th>WHEN ORDERED BY FIELD</th><th>CUSTOM LABEL</th><th>CUSTOM LABEL TRANSLATION</th><th>SUMMARY LABEL</th><th>SUMMARY LABEL TRANSLATION</th></tr><tr><td>Name</td><td>Group</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>Formula (Text)</td><td>Group</td><td></td><td>case when {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {createdfrom.salesorder} when {type}='Sales Order' then 'Sales Order #' {number} end</td><td></td><td>Sales Order</td><td></td><td></td><td></td></tr><tr><td>Formula (Currency)</td><td>Sum</td><td></td><td>case when {type}='Sales Order' then abs({amount}) else ({amount}) end</td><td></td><td></td><td></td><td>Deposits and Gift Cards Less Refunds</td><td></td></tr><tr><td>Created From : Amount</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>Formula (Currency)</td><td></td><td></td><td>max({createdfrom.totalamount})-sum({amount})</td><td></td><td></td><td></td><td></td><td></td></tr></table> Here are the formulas you can copy and paste into your search: - case when {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {createdfrom.salesorder} when {type}='Sales Order' then 'Sales Order #' {number} end - case when {type}='Sales Order' then abs({amount}) else ({amount}) end - max({createdfrom.totalamount})-sum({amount})	FIELD *	SUMMARY TYPE	FUNCTION	FORMULA	WHEN ORDERED BY FIELD	CUSTOM LABEL	CUSTOM LABEL TRANSLATION	SUMMARY LABEL	SUMMARY LABEL TRANSLATION	Name	Group								Formula (Text)	Group		case when {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {createdfrom.salesorder} when {type}='Sales Order' then 'Sales Order #' {number} end		Sales Order				Formula (Currency)	Sum		case when {type}='Sales Order' then abs({amount}) else ({amount}) end				Deposits and Gift Cards Less Refunds		Created From : Amount									Formula (Currency)			max({createdfrom.totalamount})-sum({amount})					
FIELD *	SUMMARY TYPE	FUNCTION	FORMULA	WHEN ORDERED BY FIELD	CUSTOM LABEL	CUSTOM LABEL TRANSLATION	SUMMARY LABEL	SUMMARY LABEL TRANSLATION																																															
Name	Group																																																						
Formula (Text)	Group		case when {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {createdfrom.salesorder} when {type}='Sales Order' then 'Sales Order #' {number} end		Sales Order																																																		
Formula (Currency)	Sum		case when {type}='Sales Order' then abs({amount}) else ({amount}) end				Deposits and Gift Cards Less Refunds																																																
Created From : Amount																																																							
Formula (Currency)			max({createdfrom.totalamount})-sum({amount})																																																				

4. Click **Save & Run** to ensure your search runs and returns the correct results. You should see a list of sales orders with customer deposits and refund balances. Click a sales order number to see the amounts that make up the total balance.

For more information about creating custom fields and saved searches, see the following help topics:

- Creating a Custom Field
- Creating Custom Transaction Body Fields
- Saved Searches

Step 2: Write the Set Custom Fields User Event Script

This script sets the custom **Amount Deposited** and **Balance Remaining** fields on a sales order record when the user saves it.

Script Summary

The following table summarizes this script:

Script: Deposit Balance Set Custom Fields	
Script Type	SuiteScript 2.x User Event Script Type

Script: Deposit Balance Set Custom Fields	
Modules Used	■ N/record Module ■ N/search Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ afterSubmit(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, ue_set_custom_fields.js).

If you would rather create this script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8      function afterSubmit(scriptContext) {
9          const contextOrder = scriptContext.newRecord;
10         const soID = contextOrder.id;
11         const salesorder = record.load({
12             type: record.Type.SALES_ORDER,
13             id: soID
14         });
15         const soTotal = salesorder.getValue({
16             fieldId: 'total'
17         });
18         const soEntity = salesorder.getValue({
19             fieldId: 'entity'
20         });
21         const soTranId = salesorder.getValue({
22             fieldId: 'tranid'
23         });
24         const soFullTextTranID = 'Sales Order #' + soTranId;
25         const mySearch = search.load({
26             id: 'customsearch_sobalancedue'
27         });
28         const entityFilter = search.createFilter({
29             name: 'entity',
30             operator: search.Operator.ANYOF,
31             values: soEntity
32         });

```

```

33     const soIdFilter = search.createFilter({
34         name: 'formulatext',
35         operator: search.Operator.IS,
36         formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {creat
edfrom.salesorder} when {type}='Sales Order' then 'Sales Order #' || {number} end",
37         values: soFullTextTranID
38     });
39     mySearch.filters.push(entityFilter, soIdFilter);
40     const soresults = mySearch.run();
41     mySearch.run().each(function(soresults) {
42         let soTextID = soresults.getValue({
43             name: 'formulatext',
44             summary: search.Summary.GROUP
45         });
46         if (soFullTextTranID === soTextID) {
47             let totalPaid = soresults.getValue({
48                 name: 'formulacurrency',
49                 summary: search.Summary.SUM
50             });
51             let remainingBalanceOnOrder = parseFloat(parseFloat(soTotal))-parseFloat(totalPaid);
52             salesorder.setValue({
53                 fieldId: 'custbody_total_deposit_paid',
54                 value: totalPaid
55             });
56             salesorder.setValue({
57                 fieldId: 'custbody_balance_remaining',
58                 value: remainingBalanceOnOrder
59             });
60             let id = salesorder.save({
61                 enableSourcing: true,
62                 ignoreMandatoryFields: true
63             });
64         }
65     });
66 }
67 return{
68     afterSubmit: afterSubmit
69 };
70 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Load the new sales order record
- Get values from the new sales order record
- Create and apply filters for your saved search
- Run the saved search and process the results
- Create the return statement
- Save your script file



Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 user event script. The script uses three SuiteScript modules specified in the define statement:

- N/record – allows you to work with NetSuite records
- N/search – allows you to create and run on-demand or saved searches and analyze and iterate through the search results
- N/log – allows you to log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```
1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8  });
```

Create the entry point function

This script is triggered on the afterSubmit entry point when you save a sales order.

Add the following function definition at the top of the define function:

```
1  function afterSubmit(scriptContext) {
2  }
```

Load the new sales order record

This section of the script loads the new sales order record.

Add the following code at the top of the afterSubmit entry point function:

```
1  const contextOrder = scriptContext.newRecord;
2  const soID = contextOrder.id;
3  const salesorder = record.load({
4    type: record.Type.SALES_ORDER,
5    id: soID
6  });
```

Get values from the new sales order record

This section of the script gets values from the sales order and loads the saved search.

Add the following code within the afterSubmit entry point function:

```
1  const soTotal = salesorder.getValue({
2    fieldId: 'total'
3  });
4  const soEntity = salesorder.getValue({
```

```

5     fieldId: 'entity'
6   });
7   const soTranId = salesorder.getValue({
8     fieldId: 'tranid'
9   });
10  const soFullTextTranID = 'Sales Order #' + soTranId;
11  const mySearch = search.load({
12    id: 'customsearch_sobalancedue'
13  });

```

Create and apply filters for your saved search

This section of the script creates filters and applies them to your saved search.

Add the following code within the `afterSubmit` entry point function:

```

1  const entityFilter = search.createFilter({
2    name: 'entity',
3    operator: search.Operator.ANYOF,
4    values: soEntity
5  });
6  const soIdFilter = search.createFilter({
7    name: 'formulatext',
8    operator: search.Operator.IS,
9    formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {created from.salesorder} when {type}='Sales Order' then 'Sales Order #'||{number} end",
10   values: soFullTextTranID
11  });
12  mySearch.filters.push(entityFilter, soIdFilter);

```

Run the saved search and process the results

The last section of the script calculates the remaining balance using the total current value of the sales order and the total paid value from the saved search. It then sets the custom fields and saves the updated sales order. When the user sees the saved sales order, the values are populated in the **Total Deposit Paid** and **Remaining Balance** fields.

Add the following code into the `afterSubmit` entry point function:

```

1  const soresults = mySearch.run();
2  mySearch.run().each(function(soresults) {
3    let soTextID = soresults.getValue({
4      name: 'formulatext',
5      summary: search.Summary.GROUP
6    });
7    if (soFullTextTranID === soTextID) {
8      let totalPaid = soresults.getValue({
9        name: 'formulacurrency',
10       summary: search.Summary.SUM
11     });
12     let remainingBalanceOnOrder = parseFloat(parseFloat(soTotal)) - parseFloat(totalPaid);
13     salesorder.setValue({
14       fieldId: 'custbody_total_deposit_paid',
15       value: totalPaid
16     });
17     salesorder.setValue({
18       fieldId: 'custbody_balance_remaining',
19       value: remainingBalanceOnOrder
20     });
21     let id = salesorder.save({
22       enableSourcing: true,
23       ignoreMandatoryFields: true
24     });
25   }

```

```
26 | });
```

Create the return statement

This section of the script associates the `afterSubmit` function with the `afterSubmit` user event script entry point.

Add the following code immediately above the closing `});` in your script:

```
1 | return {
2 |     afterSubmit: afterSubmit
3 | };
```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, `ue_set_custom_fields.js`).

Step 3: Write the Update Custom Fields User Event Script

This script updates the custom **Amount Deposited** and **Balance Remaining** fields when the user loads or saves a sales order.

Script Summary

The following table summarizes this script:

Script: Deposit Balance Update Custom Fields	
Script Type	SuiteScript 2.x User Event Script Type
Modules Used	■ N/record Module ■ N/search Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ beforeSubmit(context) ■ afterSubmit(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, `ue_update_custom_fields.js`).

If you would rather create this script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8      function beforeSubmit(scriptContext) {
9          const contextDep = scriptContext.newRecord;
10         const soID = contextDep.getValue({
11             fieldId: 'salesorder'
12         });
13         if ((soID !== null) && (scriptContext.type === scriptContext.UserEventType.DELETE)) {
14             const depAmt = contextDep.getValue({
15                 fieldId: 'payment'
16             });
17             const salesorder = record.load({
18                 type: record.Type.SALES_ORDER,
19                 id: soID
20             });
21             const status = salesorder.getValue({
22                 fieldId: 'status'
23             });
24             if (status !== 'Billed') {
25                 const soTotalPaid = salesorder.getValue({
26                     fieldId: 'custbody_total_deposit_paid'
27                 });
28                 const soRemainingBalance = salesorder.getValue({
29                     fieldId: 'custbody_balance_remaining'
30                 });
31                 salesorder.setValue({
32                     fieldId: 'custbody_total_deposit_paid',
33                     value: soTotalPaid - depAmt
34                 });
35                 salesorder.setValue({
36                     fieldId: 'custbody_balance_remaining',
37                     value: (soRemainingBalance + depAmt)
38                 });
39                 const id = salesorder.save({
40                     enableSourcing: true,
41                     ignoreMandatoryFields: true
42                 });
43             }
44         }
45     }
46     function afterSubmit(scriptContext) {
47         const contextDep = scriptContext.newRecord;
48         const soID = contextDep.getValue({
49             fieldId: 'salesorder'
50         });
51         if ((soID !== null) && ((scriptContext.type === scriptContext.UserEventType.CREATE) || (scriptContext.type === scriptContext
52             .UserEventType.EDIT))) {
53             const salesorder = record.load({
54                 type: record.Type.SALES_ORDER,
55                 id: soID
56             });
57             const status = salesorder.getValue({

```

```

57         fieldId: 'status'
58     });
59     if (status !== 'Billed') {
60         const soEntity = salesorder.getValue({
61             fieldId: 'entity'
62         });
63         const soTranId = salesorder.getValue({
64             fieldId: 'tranid'
65         });
66         const soFullTextTranID = 'Sales Order #' + soTranId;
67         const mySearch = search.load({
68             id: 'customsearch_sobalancedue'
69         });
70         const entityFilter = search.createFilter({
71             name: 'name',
72             operator: search.Operator.IS,
73             values: soEntity
74         });
75         const soIdFilter = search.createFilter({
76             name: 'formulatext',
77             operator: search.Operator.IS,
78             summary: search.Summary.MAX,
79             formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application'
then {createdfrom.salesorder} when {type}='Sales Order' then 'Order #' || {number} end",
80             values: soFullTextTranID
81         });
82         mySearch.filters.push(entityFilter, soIdFilter);
83         const soresults = mySearch.run();
84         mySearch.run().each(function(soresults) {
85             let soTextID = soresults.getValue({
86                 name: 'formulatext',
87                 summary: search.Summary.GROUP
88             });
89             if (soFullTextTranID === soTextID) {
90                 let totalPaid = soresults.getValue({
91                     name: 'formulacurrency',
92                     summary: search.Summary.SUM
93                 });
94                 let soTotal = salesorder.getValue({
95                     fieldId: 'total'
96                 });
97                 let remainingBalanceOnOrder = parseFloat(soTotal) - parseFloat(totalPaid);
98                 salesorder.setValue({
99                     fieldId: 'custbody_total_deposit_paid',
100                    value: totalPaid
101                });
102                salesorder.setValue({
103                    fieldId: 'custbody_balance_remaining',
104                    value: remainingBalanceOnOrder
105                });
106                let id = salesorder.save({
107                    enableSourcing: true,
108                    ignoreMandatoryFields: true
109                });
110            }
111        });
112    }
113 }
114 }
115 return {
116     beforeSubmit: beforeSubmit,
117     afterSubmit: afterSubmit
118 };
119 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines

- Create the entry point functions
- Get the new deposit record and check the event type
- Get values from the sales order if it isn't in Billed status
- Make sure the sales order ID is valid and the user is creating or editing the record
- Get the sales order status
- Create and apply filters for your saved search
- Run the search and process the results
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 user event script. The script uses three SuiteScript modules specified in the define statement:

- `N/record` – lets you work with NetSuite records
- `N/search` – lets you create and run on-demand or saved searches and analyze and iterate through the search results
- `N/log` – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8  });

```

Create the entry point functions

This script is triggered on the `beforeSubmit` and `afterSubmit` entry points when a sales order is saved.

Add the following function definitions at the top of the define function:

```

1  function beforeSubmit(scriptContext) {
2  }
3  function afterSubmit(scriptContext) {
4  }

```

Get the new deposit record and check the event type

This section of the script gets the value of the sales order the customer deposit is created from. If the customer deposit was created from a sales order and is being deleted, it updates the amount deposited and amount remaining on that sales order.

This section of the script runs after the customer deposit is saved but before it's submitted to the database, using the `beforeSubmit` entry point. Add the following code at the top of the `beforeSubmit` entry point function:

```
1 | const contextDep = scriptContext.newRecord;
2 | const soID = contextDep.getValue({
3 |   fieldId: 'salesorder'
4 | });
5 | if ((soID !== null) && (scriptContext.type === scriptContext.UserEventType.DELETE)) {
6 | }
```

Get values from the sales order if it isn't in Billed status

This section of the script loads the sales order, checks if it's in Billed status. If the sales order is not in Billed status, the script sets the custom fields based on the deposit and calculated balance remaining.

Add the following code within the `if` statement of the `beforeSubmit` entry point function:

```
1 | const depAmt = contextDep.getValue({
2 |   fieldId: 'payment'
3 | });
4 | const salesorder = record.load({
5 |   type: record.Type.SALES_ORDER,
6 |   id: soID
7 | });
8 | const status = salesorder.getValue({
9 |   fieldId: 'status'
10 | });
11 | if (status !== 'Billed') {
12 |   const soTotalPaid = salesorder.getValue({
13 |     fieldId: 'custbody_total_deposit_paid'
14 |   });
15 |   const soRemainingBalance = salesorder.getValue({
16 |     fieldId: 'custbody_balance_remaining'
17 |   });
18 |   salesorder.setValue({
19 |     fieldId: 'custbody_total_deposit_paid',
20 |     value: soTotalPaid - depAmt
21 |   });
22 |   salesorder.setValue({
23 |     fieldId: 'custbody_balance_remaining',
24 |     value: (soRemainingBalance + depAmt)
25 |   });
26 |   const id = salesorder.save({
27 |     enableSourcing: true,
28 |     ignoreMandatoryFields: true
29 |   });
30 | }
```

Make sure the sales order ID is valid and the the user is creating or editing the record

This section of the script makes sure the record is valid and the user is creating or editing the order, not viewing it.

Add the following code at the top of the `afterSubmit` entry point function:

```
1 | const contextDep = scriptContext.newRecord;
```

```

2 | const soID = contextDep.getValue({
3 |     fieldId: 'salesorder'
4 | });
5 | if ((soID !== null) && ((scriptContext.type === scriptContext.UserEventType.CREATE) || (scriptContext.type === scriptContext
6 |     .UserEventType.EDIT))) {

```

Get the sales order status

This section of the script loads the sales order and checks to see if it's in Billed status (which determines whether the saved search runs).

Add the following code within the if statement:

```

1 | const salesorder = record.load({
2 |     type: record.Type.SALES_ORDER,
3 |     id: soID
4 | });
5 | const status = salesorder.getValue({
6 |     fieldId: 'status'
7 | });
8 | if (status !== 'Billed') {
9 | }

```

Create and apply filters for your saved search

This section of the script loads the saved search to match the customer deposit order to the order in the search results. The saved search uses formulas because the link between a customer deposit or customer refund and a sales order can take different paths.

Add the following code within the nested if statement:

```

1 | const soEntity = salesorder.getValue({
2 |     fieldId: 'entity'
3 | });
4 | const soTranId = salesorder.getValue({
5 |     fieldId: 'tranid'
6 | });
7 | const soFullTextTranID = 'Sales Order #' + soTranId;
8 | const mySearch = search.load({
9 |     id: 'customsearch_sobalancedue'
10 | });
11 | const entityFilter = search.createFilter({
12 |     name: 'name',
13 |     operator: search.Operator.IS,
14 |     values: soEntity
15 | });
16 | const soIdFilter = search.createFilter({
17 |     name: 'formulatext',
18 |     operator: search.Operator.IS,
19 |     summary: search.Summary.MAX,
20 |     formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {created
from.salesorder} when {type}='Sales Order' then 'Order #'||{number} end",
21 |     values: soFullTextTranID
22 | });
23 | mySearch.filters.push(entityFilter, soIdFilter);

```

Run the search and process the results

This section of the script gets the balances from the saved search and sets the custom fields on the sales order that the deposit came from.

Add the following code within the nested if statement:

```

1 | const soresults = mySearch.run();

```

```

2 mySearch.run().each(function(soresults) {
3   let soTextID = soresults.getValue({
4     name: 'formulatext',
5     summary: search.Summary.GROUP
6   });
7   if (soFullTextTranID === soTextID) {
8     let totalPaid = soresults.getValue({
9       name: 'formulacurrency',
10      summary: search.Summary.SUM
11    });
12    let soTotal = salesorder.getValue({
13      fieldId: 'total'
14    });
15    let remainingBalanceOnOrder = parseFloat(soTotal) - parseFloat(totalPaid);
16    salesorder.setValue({
17      fieldId: 'custbody_total_deposit_paid',
18      value: totalPaid
19    });
20    salesorder.setValue({
21      fieldId: 'custbody_balance_remaining',
22      value: remainingBalanceOnOrder
23    });
24    let id = salesorder.save({
25      enableSourcing: true,
26      ignoreMandatoryFields: true
27    });
28  }
29 });

```

Create the return statement

This part of the script associates the `beforeSubmit` function with the `beforeSubmit` user event script entry point and the `afterSubmit` function with the `afterSubmit` user event script entry point.

Add the following code immediately above the closing `});` in your script:

```

1 return {
2   beforeSubmit: beforeSubmit,
3   afterSubmit: afterSubmit
4 };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested indentation.

When you're happy with your script, save it as a .js file (for example, `ue_update_custom_fields.js`).

Step 4: Write the Handle Refund User Event Script

This script updates the sales order if a refund has been issued.

Script Summary

The following table summarizes this script:

Script: Deposit Balance Handle Refund	
Script Type	SuiteScript 2.x User Event Script Type
Modules Used	N/record Module

Script: Deposit Balance Handle Refund	
	■ N/search Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ afterSubmit(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, ue_handle_refund.js).

If you would rather create this script step by step, follow the steps in [Build the Script](#).



Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8      return {
9          afterSubmit: function(scriptContext) {
10             const contextRef = scriptContext.newRecord;
11             const refId = contextRef.id;
12             UpdateSalesOrder(refId);
13         }
14     };
15     function UpdateSalesOrder(refId) {
16         const refund = record.load({
17             type: record.Type.CUSTOMER_REFUND,
18             id: refId,
19             isDynamic: true
20         });
21         const soEntity = refund.getValue({
22             fieldId: 'customer'
23         });
24         const lines = refund.getLineCount({
25             sublistId: 'apply'
26         });
27         for (let i = 0; i < lines; i++) {
28             let depositnum = refund.getSublistText({
29                 sublistId: 'apply',
30                 fieldId: 'internalid',
31                 line: i
32             });
33             let refundamt = refund.getSublistValue({
34                 sublistId: 'apply',
35                 fieldId: 'amount',
36                 line: i

```

```

37     });
38     let order = search.lookupFields({
39         type: search.Type.DEPOSIT_APPLICATION,
40         id: depositnum,
41         columns: 'createdfrom.salesorder'
42     });
43     let soFullTextTranID = order['createdfrom.salesorder'][0].text;
44     let orderId = order['createdfrom.salesorder'][0].value;
45     let soTotalPaid = search.lookupFields({
46         type: search.Type.SALES_ORDER,
47         id: orderId,
48         columns: ['total']
49     });
50     let soTotal = soTotalPaid['total'];
51     let mySearch = search.load({
52         id: 'customsearch_sobalancedue'
53     });
54     let entityFilter = search.createFilter({
55         name: 'internalid',
56         join: 'customer',
57         operator: search.Operator.EQUALTO,
58         summary: search.Summary.MAX,
59         values: soEntity
60     });
61     let soIdFilter = search.createFilter({
62         name: 'formulatext',
63         operator: search.Operator.IS,
64         formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then
{createdfrom.salesorder} when {type}='Sales Order' then 'Sales Order #'||{number} end",
65         values: soFullTextTranID
66     });
67     mySearch.filters.push(entityFilter, soIdFilter);
68     let soresults = mySearch.run();
69     mySearch.run().each(function(soresults) {
70         let soTextID = soresults.getValue({
71             name: 'formulatext',
72             summary: search.Summary.GROUP
73         });
74         if (soFullTextTranID === soTextID) {
75             let totalPaid = soresults.getValue({
76                 name: 'formulacurrency',
77                 summary: search.Summary.SUM
78             });
79             let remainingBalanceOnOrder = parseFloat(soTotal);
80             remainingBalanceOnOrder = parseFloat(remainingBalanceOnOrder)-parseFloat(totalPaid);
81             let salesorder = record.load({
82                 type: record.Type.SALES_ORDER,
83                 id: orderId,
84                 isDynamic: true
85             });
86             salesorder.setValue({
87                 fieldId: 'custbody_total_deposit_paid',
88                 value: totalPaid
89             });
90             salesorder.setValue({
91                 fieldId: 'custbody_balance_remaining',
92                 value: remainingBalanceOnOrder
93             });
94             let id = salesorder.save({
95                 enableSourcing: true,
96                 ignoreMandatoryFields: true
97             });
98         }
99     });
100 }
101 }
102 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Create the UpdateSalesOrder function
- Load the customer refund
- Add a for loop
- Get the deposit number and the refund amount
- Find the linked sales order
- Load the saved search
- Create and apply search filters for you saved search
- Run the search and process the results
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 user event script. The script uses three SuiteScript modules specified in the define statement:

- N/record – lets you work with NetSuite records
- N/search – lets you create and run on-demand or saved searches and analyze and iterate through the search results
- N/log – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8  });

```

Create the entry point function

This script is triggered on the afterSubmit entry points when a sales order is saved.

Add the following function definition at the top of the define function:

```

1  return {
2    afterSubmit: function(scriptContext) {

```

```

3      const contextRef = scriptContext.newRecord;
4      const refId = contextRef.id;
5      UpdateSalesOrder(refId);
6    }
7  };

```

Create the UpdateSalesOrder function

This section of the script uses a function to update the sales order. This function is called from the `afterSubmit` entry point function.

Add the following code after the end of the `return` statement:

```

1  function UpdateSalesOrder(refId) {
2  }

```

Load the customer refund

This section of the script loads the customer refund and gets the total refund amount and the number of lines or transactions it's applied to.

Add the following code at the top of the `UpdateSales Order` function:

```

1  const refund = record.load({
2    type: record.Type.CUSTOMER_REFUND,
3    id: refId,
4    isDynamic: true
5  });
6  const soEntity = refund.getValue({
7    fieldId: 'customer'
8  });
9  const lines = refund.getLineCount({
10   sublistId: 'apply'
11 });

```

Add a for loop

This section of the script uses a `for` loop to iterate through each deposit application associated with the customer refund and gets the amount applied.

Add the following code within the `UpdateSalesOrder` function:

```

1  for (let i = 0; i < lines; i++) {
2  }

```

Get the deposit number and the refund amount

For each deposit application associated with the customer refund, this script gets the deposit number and the refund amount.

Add the following code at the top of the `for` loop:

```

1  let depositnum = refund.getSublistText({
2    sublistId: 'apply',
3    fieldId: 'internalid',
4    line: i
5  });
6  let refundamt = refund.getSublistValue({
7    sublistId: 'apply',
8    fieldId: 'amount',

```



```

9 |   line: i
10 | });

```

Find the linked sales order

This section of the script looks up each deposit application to find the linked sales order and its total.

Add the following code within the for statement:

```

1 | let order = search.lookupFields({
2 |   type: search.Type.DEPOSIT_APPLICATION,
3 |   id: depositnum,
4 |   columns: 'createdfrom.salesorder'
5 | });
6 | let soFullTextTranID = order['createdfrom.salesorder'][0].text;
7 | let orderId = order['createdfrom.salesorder'][0].value;
8 | let soTotalPaid = search.lookupFields({
9 |   type: search.Type.SALES_ORDER,
10 |   id: orderId,
11 |   columns: ['total']
12 | });
13 | let soTotal = soTotalPaid['total'];

```

Load the saved search

This section of the script loads the saved search to get the current balance of the deposits and gift certificates paid toward the sales order, minus any refunds.

Add the following code within the for statement:

```

1 | let mySearch = search.load({
2 |   id: 'customsearch_sobalancedue'
3 | });

```

Create and apply search filters for you saved search

This section of the script creates and applies two search filters for your saved search.

Add the following code within the for statement:

```

1 | let entityFilter = search.createFilter({
2 |   name: 'internalid',
3 |   join: 'customer',
4 |   operator: search.Operator.EQUALTO,
5 |   summary: search.Summary.MAX,
6 |   values: soEntity
7 | });
8 | let soIdFilter = search.createFilter({
9 |   name: 'formulatext',
10 |   operator: search.Operator.IS,
11 |   formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {created from.salesorder} when {type}='Sales Order' then 'Sales Order #'||{number} end",
12 |   values: soFullTextTranID
13 | });
14 | mySearch.filters.push(entityFilter, soIdFilter);

```

Run the search and process the results

This section of the script loads each related sales order and sets the new values for total paid and remaining balance.

Add the following code within the for statement:

```

1 let soresults = mySearch.run();
2 mySearch.run().each(function(soresults) {
3     let soTextID = soresults.getValue({
4         name: 'formulatext',
5         summary: search.Summary.GROUP
6     });
7     if (soFullTextTranID === soTextID) {
8         let totalPaid = soresults.getValue({
9             name: 'formulacurrency',
10            summary: search.Summary.SUM
11        });
12        let remainingBalanceOnOrder = parseFloat(soTotal);
13        remainingBalanceOnOrder = parseFloat(remainingBalanceOnOrder) - parseFloat(totalPaid);
14        let salesorder = record.load({
15            type: record.Type.SALES_ORDER,
16            id: orderId,
17            isDynamic: true
18        });
19        salesorder.setValue({
20            fieldId: 'custbody_total_deposit_paid',
21            value: totalPaid
22        });
23        salesorder.setValue({
24            fieldId: 'custbody_balance_remaining',
25            value: remainingBalanceOnOrder
26        });
27        let id = salesorder.save({
28            enableSourcing: true,
29            ignoreMandatoryFields: true
30        });
31    }
32 });

```

Create the return statement

In this script, you already added the return statement when you added the entry point function at the top of the define function (see [Create the entry point function](#)).

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, ue_handle_refund.js).

Step 5: Write the Handle Deletion of a Deposit Application User Event Script

This script handles the deletion of a deposit application that's associated with a customer refund or customer deposit when a user saves a sales order.

Script Summary

The following table summarizes this script:

Script: Deposit Balance Handle Deletion of Deposit Application	
Script Type	SuiteScript 2.x User Event Script Type

Script: Deposit Balance Handle Deletion of Deposit Application	
Modules Used	■ N/record Module ■ N/search Module ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ afterSubmit(context)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, ue_handle_deposit.js).

If you would rather create this script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8      function afterSubmit(scriptContext) {
9          const contextDepApp = scriptContext.oldRecord;
10         const depAppId = contextDepApp.id;
11         const soEntity = contextDepApp.getValue({
12             fieldId: 'customer'
13         });
14         const createdFrom = contextDepApp.getValue({
15             fieldId: 'deposit'
16         });
17         const cusDeposit = record.load({
18             type: record.Type.CUSTOMER_DEPOSIT,
19             id: createdFrom,
20             isDynamic: true
21         });
22         const orderId = cusDeposit.getValue({
23             fieldId: 'salesorder'
24         });
25         const soFullTextTranID = cusDeposit.getText({
26             fieldId: 'salesorder',
27         });
28         const mySearch = search.load({
29             id: 'customsearch_sobalancedue'
30         });
31         const entityFilter = search.createFilter({
32             name: 'internalidnumber',

```

```

33         operator: search.Operator.IS,
34         values: soEntity
35     });
36     const soIdFilter = search.createFilter({
37         name: 'formulatext',
38         operator: search.Operator.IS,
39         formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {createdfrom.salesorder} when {type}='Sales Order' then 'Sales Order #'||{number} end",
40         values: soFullTextTranID
41     });
42     mySearch.filters.push(entityFilter, soIdFilter);
43     const soresults = mySearch.run();
44     mySearch.run().each(function(soresults) {
45         let soTextID = soresults.getValue({
46             name: 'formulatext',
47             summary: search.Summary.GROUP
48         });
49         if (soFullTextTranID === soTextID) {
50             let totalPaid = soresults.getValue({
51                 name: 'formulacurrency',
52                 summary: search.Summary.SUM
53             });
54             let salesorder = record.load({
55                 type: record.Type.SALES_ORDER,
56                 id: orderId,
57                 isDynamic: true
58             });
59             let soTotal = salesorder.getValue({
60                 fieldId: 'total'
61             });
62             let remainingBalanceOnOrder = parseFloat(soTotal);
63             remainingBalanceOnOrder = parseFloat(remainingBalanceOnOrder) - parseFloat(totalPaid);
64             salesorder.setValue({
65                 fieldId: 'custbody_total_deposit_paid',
66                 value: totalPaid
67             });
68             salesorder.setValue({
69                 fieldId: 'custbody_balance_remaining',
70                 value: remainingBalanceOnOrder
71             });
72             let id = salesorder.save({
73                 enableSourcing: true,
74                 ignoreMandatoryFields: true
75             });
76         }
77     });
78 }
79 return {
80     afterSubmit: afterSubmit
81 };
82 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Get values for the related customer and customer deposit
- Load the saved search and create and apply search filters
- Run the search and process the results
- Load the sales order and set values
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 user event script. The script uses three SuiteScript modules specified in the define statement:

- `N/record` – lets you work with NetSuite records
- `N/search` – lets you create and run on-demand or saved searches and analyze and iterate through the search results
- `N/log` – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```
1  /**
2   * @ApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/log'], (record, search, log) => {
8  });
```

Create the entry point function

This script is triggered on the `beforeSubmit` and `afterSubmit` entry points when a sales order is saved.

Add the following function definition at the top of the define function:

```
1  function afterSubmit(scriptContext) {
2  }
```

Get values for the related customer and customer deposit

This section of the script loads the previous version of the deposit application in case was deleted so that it can get the related customer and customer deposit values.

Add the following code within the `afterSubmit` entry point script:

```
1  const contextDepApp = scriptContext.oldRecord;
2  const depAppId = contextDepApp.id;
3  const soEntity = contextDepApp.getValue({
4    fieldId: 'customer'
5  });
6  const createdFrom = contextDepApp.getValue({
7    fieldId: 'deposit'
8  });
9  const cusDeposit = record.load({
10   type: record.Type.CUSTOMER_DEPOSIT,
```

```

11     id: createdFrom,
12     isDynamic: true
13   });
14   const orderId = cusDeposit.getValue({
15     fieldId: 'salesorder'
16   });
17   const soFullTextTranID = cusDeposit.getText({
18     fieldId: 'salesorder',
19   });

```

Load the saved search and create and apply search filters

This section of script loads the saved search you created and adds filters to find the sales order related to the deposit application, along with its current total deposited.

Add the following code into the `afterSubmit` entry point script:

```

1  const mySearch = search.load({
2    id: 'customsearch_sobalancedue'
3  });
4  const entityFilter = search.createFilter({
5    name: 'internalidnumber',
6    operator: search.Operator.IS,
7    values: soEntity
8  });
9  const soIdFilter = search.createFilter({
10   name: 'formulatext',
11   operator: search.Operator.IS,
12   formula: "CASE WHEN {type}='Customer Deposit' then {appliedtotransaction} when {type}='Deposit Application' then {created from.salesorder} when {type}='Sales Order' then 'Sales Order #' || {number} end",
13   values: soFullTextTranID
14 });
15 mySearch.filters.push(entityFilter, soIdFilter);

```

Run the search and process the results

This section of the script runs the saved search you created to find the sales order related to the deposit application and its current total deposited.

Add the following code within the `afterSubmit` entry point script:

```

1  const soresults = mySearch.run();
2  mySearch.run().each(function(soresults) {
3    let soTextID = soresults.getValue({
4      name: 'formulatext',
5      summary: search.Summary.GROUP
6    });
7    if (soFullTextTranID === soTextID) {
8    }
9  });

```

Load the sales order and set values

This section of the script loads the sales order related to the deposit application and sets the deposit paid and new remaining balance based on the saved search results.

Add the following code within the `if` statement:

```

1  let totalPaid = soresults.getValue({
2    name: 'formulacurrency',
3    summary: search.Summary.SUM
4  });
5  let salesorder = record.load({

```

```

6      type: record.Type.SALES_ORDER,
7      id: orderId,
8      isDynamic: true
9    });
10    let soTotal = salesorder.getValue({
11      fieldId: 'total'
12    });
13    let remainingBalanceOnOrder = parseFloat(soTotal);
14    remainingBalanceOnOrder = parseFloat(remainingBalanceOnOrder) - parseFloat(totalPaid);
15    salesorder.setValue({
16      fieldId: 'custbody_total_deposit_paid',
17      value: totalPaid
18    });
19    salesorder.setValue({
20      fieldId: 'custbody_balance_remaining',
21      value: remainingBalanceOnOrder
22    });
23    let id = salesorder.save({
24      enableSourcing: true,
25      ignoreMandatoryFields: true
26    });

```

Create the return statement

This script associates the `afterSubmit` function with the `afterSubmit` user event script entry point.

Add the following code immediately above the closing `});` in your script:

```

1  return {
2    afterSubmit: afterSubmit
3  };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested indentation.

When you're happy with your script, save it as a .js file (for example, `ue_handle_deposit.js`).

Step 6: Create the Script Records

After you create your scripts, you need to create script records for each one:

- [Create the script record for the Set Custom Fields user event script](#)
- [Create the script record for the Update Custom Fields user event script](#)
- [Create the script record for the Handle Refund user event script](#)
- [Create the script record for the Handle Deletion of Deposit Application user event script](#)

For more information about creating script records, see the help topic [Creating a Script Record](#).

Create the script record for the Set Custom Fields user event script

To create the script record for the Set Custom Fields user event script:

1. Upload your script to the NetSuite File Cabinet.

2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Deposit Balance Set Custom Fields
ID	_ue_depBalance_set_fields NetSuite prefixes the ID with customscript .
Description	This script sets the custom fields on a sales order record when the user saves the sales order.

5. Set any other fields on the script record as required.
6. Click **Save**.

Create the script record for the Update Custom Fields user event script

To create the script record for the Update Custom Fields user event script:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Deposit Balance Update Custom Fields
ID	_ue_depBalance_update_fields NetSuite prefixes the ID with customscript .
Description	This script updates the custom fields on a sales order record when a user loads or saves a sales order.

5. Set any other fields on the script record as required.
6. Click **Save**.

Create the script record for the Handle Refund user event script

To create the script record for the Handle Refund user event script:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Deposit Balance Handle Refund
ID	_ue_depBalance_refund NetSuite prefixes the ID with customscript .
Description	This script updates the sales order if a refund has been issued.

5. Set any other fields on the script record as required.
6. Click **Save**.

Create the script record for the Handle Deletion of Deposit Application user event script

To create the script record for the Handle Deletion of Deposit Application user event script:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Deposit Balance Deletion of Deposit App
ID	_ue_depBalance_delete_depApp NetSuite prefixes the ID with customscript .
Description	This script handles the deletion of a deposit application that is associated with a customer refund or customer deposit when a user saves a sales order.

5. Set any other fields on the script record as required.
6. Click **Save**.

Step 7: Deploy the Scripts

After you create the script record for each script, you can create script deployment records for them. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

- [Deploy the Set Custom Fields user event script](#)
- [Deploy the Update Custom Fields user event script](#)
- [Deploy the Handle Refund user event script](#)
- [Deploy the Handle Deletion of Deposit Application user event script](#)

Deploy the Set Custom Fields user event script

To deploy the Set Custom Fields user event script:

1. Complete the steps in [Step 6: Create the Script Records](#) for your user event script.
2. Go to Customization > Scripting > Scripts.
3. Find your user event script in the list of scripts and click Deployments. The Script Deployments page appears.
4. Click **New Deployment**. The Script Deployment page appears.
5. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_ue_deposit_track_so NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's best practice to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

6. Click **Save**.

Deploy the Update Custom Fields user event script

To deploy the Update Custom Fields user event script:

1. Complete the steps in [Step 6: Create the Script Records](#) for your user event script.
2. Go to Customization > Scripting > Scripts.
3. Find your user event script in the list of scripts and click Deployments. The Script Deployments page appears.
4. Click **New Deployment**. The Script Deployment page appears.
5. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Customer Deposit
ID	_ue_customer_deposit NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.

Field	Value
Execute As Role	Current Role It's best to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

- Click **Save**.

Deploy the Handle Refund user event script

To deploy the Handle Refund user event script:

- Complete the steps in [Step 6: Create the Script Records](#) for your user event script.
- Go to Customization > Scripting > Scripts.
- Find your user event script in the list of scripts and click Deployments. The Script Deployments page appears.
- Click **New Deployment**. The Script Deployment page appears.
- On the Script Deployment page, enter the following values:

Field	Value
Applies To	Customer Refund
ID	_ue_customer_refund NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's best to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

- Click **Save**.

Deploy the Handle Deletion of Deposit Application user event script

To deploy the Handle Deletion of Deposit Application user event script:

- Complete the steps in [Step 6: Create the Script Records](#) for your user event script.
- Go to Customization > Scripting > Scripts.
- Find your user event script in the list of scripts and click Deployments. The Script Deployments page appears.

- Click **New Deployment**. The Script Deployment page appears.
- On the Script Deployment page, enter the following values:

Field	Value
Applies To	Deposit Application
ID	_ue_deposit_application NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's best to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

- Click **Save**.

Step 8: Test the Solution

After you create the script records and deploy your scripts, you can test your solution.

When these scripts run on sales orders, customer deposits, customer refunds, or deposit applications, the two custom fields, **Amount Deposited** and **Balance Remaining**, are updated on the sales order as shown below.

The screenshot shows the NetSuite interface for a Sales Order (126) for customer Garncarz, Lani. The order status is 'PENDING FULFILLMENT'. The 'Primary Information' section displays the order details. A red box highlights the custom fields 'AMOUNT DEPOSITED' (650.00) and 'BALANCE REMAINING' (20,396.93) in the 'MEMO' section. The 'Summary' section shows the following values:

Summary	
SUBTOTAL	19,627.00
DISCOUNT ITEM	
TAX	1,519.93
SHIPPING COST	
HANDLING COST	
GIFT CERTIFICATE	-100.00
TOTAL	21,046.93

You can test this scenario in your sandbox, release preview, or by deleting your test data after you're done testing.

To test the deposit:

- Go to Transactions > Sales > Enter Sales Orders > List.

2. Click **View** next to an order that is Pending Fulfillment and needs a customer deposit recorded against it. You can create a test sales order if you do not have any existing orders.
3. Click **Create Deposit**. If you do not see the **Create Deposit** button, you may need to edit the sales order and change the Custom Form selection to **Standard Sales Order – Invoice**. Or, select the custom form you chose when you created the **Amount Deposited** and **Balance Remaining** fields.
4. On the Customer Deposit page, change the **Payment Amount** to an amount that is less than the total of the sales order. Set the **Payment Method** to cash or check, and click **Save**.
5. Return to the sales order. You should see the deposit value in the **Amount Deposited** field, and the sales order total minus the deposit amount in the **Balance Remaining** field.
6. Edit the sales order to change the total, and the **Balance Remaining** should update when you save.

To test a refund:

1. Go to Transactions > Customers > Issue Customer Refund.
2. Select the same customer as you did for the sales order and deposit, and click the **Apply** subtab and the **Deposits** subtab.
3. Check the **Apply** box next to the customer deposit and enter an amount in the Payment field.
4. On the **Refund Method** subtab, in the **Refund Method** field, select **Cash** or **Check**.
5. Click **Save**.
6. View the sales order. You should see the **Amount Deposited** decrease and the **Balance Remaining** increase.
7. To test the deletion of a deposit application, open the customer refund created in steps 7–11.
8. On the **Apply** subtab, click the date to open the customer deposit the refund is applied to open the deposit.
9. On the customer deposit, on the **Applied To** subtab, click the date in the **Date Applied** column for the customer refund. This opens the deposit application.
10. Click **Edit** and then click **Delete**.
11. View the sales order again and verify that the refund is no longer applied toward the **Amount Deposited**.

Complete SuiteCloud Project

Project file: [com.example.SalesOrderDepositTracking.zip](https://www.oracle.com/netSuite/sample-projects/com.example.SalesOrderDepositTracking.zip)

You can download this complete project as an SDF account customization project using the preceding ZIP file download link. After you download the project, open it in an IDE with the NetSuite SuiteCloud IDE Plug-In or Extension, such as WebStorm or Visual Studio Code. In the IDE, you'll see each customization objects (scripts, custom fields, and saved search) represented as an XML file. Using the SuiteCloud IDE Plug-In or Extension, you can deploy the entire solution to your account for testing.

While the script deployments in the project default to Testing mode, the custom fields and saved search may be visible to others in your account. Use caution when choosing the NetSuite account and account type for testing.

To open the project in WebStorm or Visual Studio Code:

1. Extract the contents of the .zip file to your preferred project folder location.

2. In WebStorm, click **File > Open**. In Visual Studio Code, click **File > Open Folder**.
 3. Go to the location of the extracted project folder, and select the folder.
- The project and its components appear in the Projects list.

For information about deploying the project from the IDE to your account, see:

- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud IDE Plug-in for WebStorm](#)
- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#)

For more information about SuiteCloud IDE Plug-Ins and Extensions, including installation and setup, see:

- [SuiteCloud IDE Plug-in for WebStorm Overview](#)
- [SuiteCloud Extension for Visual Studio Code Overview](#)

Set Purchase Order Exchange Rate

This use case shows you how to convert the transaction total on a purchase order to a user-specific currency rather than the vendor's currency. The converted purchase order total is stored in a custom field on the purchase order record so that you can use it for reporting without changing the standard **Total** field.

 **Note:** This use case solution updates the custom fields every time the purchase order is edited. If you'd rather have this field only set when the purchase order is created, you need to use a user event script.

For more information about currencies and exchange rates in NetSuite, see the help topic [Currency Management](#).

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

This customization for this use case includes:

- A custom field (Custom Currency PO Amount) to store the purchase amount in the currency of the person who created the purchase order.
- A custom field (Custom Currency Exchange Rate) to store the exchange rate between the transaction currency and the custom currency.
- A script parameter (Custom Currency) to store the currency set by the user.
- A client script triggered on the saveRecord entry point.

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Fields](#)
- [Step 2: Write the Script](#)
- [Step 3: Create the Script Record](#)

- [Step 4: Deploy the Script](#)
- [Step 5: Create and Set the Script Parameter](#)
- [Step 6: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	Ensure the following features are enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ Multiple Currencies – Lets you select the currency on transactions and use exchange rates. ■ Purchase Orders – Lets you create and track the orders for items you buy from vendors. ■ A/P – Lets you create and track bills based on vendor terms. For more information, see the help topic Enabling Features.
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access ■ Transaction Body Fields – Create access ■ Purchase Orders – Edit and Delete access ■ Currencies – View access ■ Currency Exchange Rates – View access For more information, see the help topic NetSuite Permissions Overview.
Other Requirements	None

Step 1: Create the Custom Fields

This customization uses two custom fields. The **Custom Currency PO Amount** field stores the purchase amount in the currency of the person who created the order. The **Custom Currency Exchange Rate** field stores the exchange rate between the transaction currency and the custom currency. Both fields are custom transaction body fields added to the purchase order.

To create the Custom Currency PO Amount field:

1. Go to Customization > Lists, Records & Forms > Transaction Body Fields > New
2. Enter values for each field as listed in the following table:

Field	Value
Label	Custom Currency PO Amount
ID	_currency_po_amount NetSuite prefixes the ID with custbody .

Field	Value
Description	This custom field stores the purchase amount in the currency for the user who is creating or editing the order.
Type	Currency
Store Value	Checked
Applies To	Purchase
Display	Subtab: Main

3. Click **Save**.

To create the Custom Currency Exchange Rate field:

1. Go to Customization > Lists, Records & Forms > Transaction Body Fields > New.
2. Enter values for each field as listed in the following table:

Field	Value
Label	Custom Currency Exchange Rate
ID	_currency_exchange_rate NetSuite prefixes the ID with custbody .
Description	This custom field stores the exchange rate for the custom currency set by the user.
Type	Currency
Store Value	Checked
Applies To	Purchase
Display	Subtab: Main

3. Click **Save**.

For more information about creating custom transaction body fields, see the following help topics:

- [Creating a Custom Field](#)
- [Creating Custom Transaction Body Fields](#)

Step 2: Write the Script

This script converts the transaction total on a purchase order to the user's currency instead of using the vendor's currency. The converted purchase order total is stored in a custom field on the purchase order record so that you can use it for reporting without changing the standard **Total** field.

Script Summary

The following table summarizes the script:

Script: Set Purchase Order Exchange Rate	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/runtime Module

Script: Set Purchase Order Exchange Rate	
	■ N/currency Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial explicitly loads this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ saveRecord(scriptContext)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, cs_poExchangeRate.js).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/currentRecord', 'N/currency', 'N/log'], (runtime, currentRecord, currency, log) => {
8      function saveRecord(context) {
9          try {
10             const stUserCurrency = runtime.getCurrentScript().getParameter({
11                 name: 'custscript_custom_currency'
12             });
13             if (stUserCurrency === " " || stUserCurrency === null || stUserCurrency === undefined) {
14                 throw "Please enter a value for Custom Currency at Home > User Preferences > Custom.";
15             }
16             const purchaseOrder = context.currentRecord;
17             const stTranCurrency = purchaseOrder.getValue({
18                 fieldId: 'currency'
19             });
20             const stTranDate = purchaseOrder.getValue({
21                 fieldId: 'trandate'
22             });
23             const stTotal = purchaseOrder.getValue({
24                 fieldId: 'total'
25             });
26             let flTotalAmount = parseFloat(stTotal);
27             let exchangeRate = currency.exchangeRate({
28                 source: stTranCurrency,
29                 target: stUserCurrency,
30                 date: stTranDate

```

```

31     });
32     const flExchangeRate = parseFloat(exchangeRate);
33     const flAmountInUserCurrency = parseFloat(flTotalAmount * flExchangeRate);
34     purchaseOrder.setValue({
35         fieldId: 'custbody_currency_exchange_rate',
36         value: flExchangeRate
37     });
38     purchaseOrder.setValue({
39         fieldId: 'custbody_currency_po_amount',
40         value: flAmountInUserCurrency
41     });
42     } catch(e) {
43         if (e.getDetails() !== undefined) {
44             log.error({
45                 title: 'Process Error',
46                 details: JSON.stringify(e)
47             });
48         } else {
49             log.error({
50                 title: 'Unexpected Error',
51                 details: JSON.stringify(e)
52             });
53         }
54         throw (e);
55     }
56     return true;
57 }
58 return {
59     saveRecord: saveRecord
60 };
61 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Get the currency script parameter
- Get the purchase order data
- Convert the purchase order total to a decimal value
- Get the exchange rate between the transaction currency and the user's custom currency
- Calculate the custom currency amount
- Set the custom currency exchange rate and custom currency PO amount custom fields
- Create the return statement
- Save your script file



Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.x (2.0 or 2.1) client script. The script uses four SuiteScript modules specified in the define statement:

- N/runtime – provides access to script parameters
- N/currentRecord – provides access to the purchase order record

- N/currency – provides a SuiteScript method to calculate an exchange rate
- N/log – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```
1  /**
2   * @ApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/currentRecord', 'N/currency', 'N/log'], (runtime, currentRecord, currency, log) => {
8  });
```

Create the entry point function

This script is triggered on the saveRecord entry point when the purchase order record is saved. A try-catch block is logs any errors that might occur during script execution. Most of the script code goes in the try block.

Add the following function definition and initial try-catch block statements at the top of the define function:

```
1  function saveRecord(context) {
2      try {
3      } catch(e) {
4          if (e.getDetails !== undefined) {
5              log.error({
6                  title: 'Process Error',
7                  details: JSON.stringify(e)
8              });
9          } else {
10             log.error({
11                 title: 'Unexpected Error',
12                 details: JSON.stringify(e)
13             });
14         }
15         throw (e);
16     }
17     return true;
18 }
```

Get the currency script parameter

This script uses a script parameter to determine the currency set by the user. If this script parameter is not specified, the script ends.

Add the following code within the try block:

```
1  const stUserCurrency = runtime.getCurrentScript().getParameter({
2      name: 'custscript_custom_currency'
3  });
4  if (stUserCurrency === " " || stUserCurrency === null || stUserCurrency === undefined) {
5      throw "Please enter a value for Custom Currency at Home > User Preferences > Custom.";
6  }
```

Get the purchase order data

You need to get the currency value and the purchase order date from the purchase order record.

Add the following code within the try block:

```
1 | const purchaseOrder = context.currentRecord;
2 | const stTranCurrency = purchaseOrder.getValue({
3 |   fieldId: 'currency'
4 | });
5 | const stTranDate = purchaseOrder.getValue({
6 |   fieldId: 'trandate'
7 | });
```

Convert the purchase order total to a decimal value

Now you need to convert the purchase order total to a decimal number. A decimal number is required to calculate the amount total in the user's custom currency.

Add the following code into the try block:

```
1 | const stTotal = purchaseOrder.getValue({
2 |   fieldId: 'total'
3 | });
4 | let flTotalAmount = parseFloat(stTotal);
```

Get the exchange rate between the transaction currency and the user's custom currency

You need to convert the exchange rate between the transaction currency and the currency for the user into a decimal value using the `exchangeRate` method provided by the `N/currency` module.

Add the following code into the try block:

```
1 | let exchangeRate = currency.exchangeRate({
2 |   source: stTranCurrency,
3 |   target: stUserCurrency,
4 |   date: stTranDate
5 | });
6 | const flExchangeRate = parseFloat(exchangeRate);
```

Calculate the custom currency amount

Now you need to calculate the custom currency amount using the exchange rate and the purchase order total.

Add the following code within the try block:

```
1 | const flAmountInUserCurrency = parseFloat(flTotalAmount * flExchangeRate);
```

Set the custom currency exchange rate and custom currency PO amount custom fields

Then, set the custom fields on the purchase order to display the custom currency's exchange rate and the calculated purchase amount.

Add the following code into the try block:

```

1 purchaseOrder.setValue({
2   fieldId: 'custbody_currency_exchange_rate',
3   value: flExchangeRate
4 });
5 purchaseOrder.setValue({
6   fieldId: 'custbody_currency_po_amount',
7   value: flAmountInUserCurrency
8 });

```

Create the return statement

This script associates the saveRecord function with the saveRecord client script entry point.

Add the following code immediately above the closing `});` in your script:

```

1 return {
2   saveRecord: saveRecord
3 };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested indentation.

When you're happy with your script, save it as a .js file (for example, cs_poExchangeRate.js).

Step 3: Create the Script Record

Now that you've completed the script, you can upload it to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Set Purchase Order Exchange Rate
ID	_cs_po_set_exchange_rate NetSuite prefixes the ID with customscript .
Description	This script converts the transaction total on a purchase order to a user-specific currency rather than the currency associated with the vendor.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 4: Deploy the Script](#).

Step 4: Deploy the Script

After you create the script record, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

To deploy the script:

1. Complete the steps in [Step 3: Create the Script Record](#).
2. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Purchase Order
ID	_cs_po_set_exchange_rate NetSuite prefixes the ID with customdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

3. Click **Save**.

Step 5: Create and Set the Script Parameter

This tutorial uses a script parameter to specify the user's custom currency and the corresponding exchange rate.

For more information about creating and setting script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

To create the script parameter:

1. Go to Customization > Scripting > Scripts.
2. Find your script in the list and click **Edit** next to the script name.
3. On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.
4. On the Script Field page, enter the following values to create the two script parameters. Click **Save** after you enter each value.

Label	ID	Type	List/Record	Description	Preference
Custom Currency	_custom_currency NetSuite prefixes the ID with custscript , using custscript_custom_	List/Record	Currency	This parameter specifies the user's custom currency.	User This setting lets each user set their own value for the script parameter on the Set Preferences page. The user preference

Label	ID	Type	List/Record	Description	Preference
	currency in the script.				overrides the value set on the script deployment page.

5. To save the script parameter, click **Save**. The Script page appears.
6. To save the script record with the updated script parameter, click **Save**.

After you create the script parameter, you must set the value of the parameter.

To set the script parameter:

1. Go to Home > Set Preferences to set your parameter value as a user preference.
2. On the **Custom Preferences** subtab, in the **Custom Currency** field, select the currency to use for the custom exchange rate.
3. Click **Save**.

Step 6: Test the Solution

After you create the script record and deploy your script, you can test it by creating a new purchase order and reviewing it to verify the currency and exchange rate are correct.

To test the solution:

1. Go to Transactions > Purchases > Enter Purchase Orders to create a new purchase order.
2. In the **Custom Form** field, select **Standard Purchase Order**.
3. Fill in all required fields on the purchase order.
4. Click **Save**.
5. View the newly created purchase order and verify that the custom fields, **Custom Currency PO Amount** and **Custom Currency Exchange Rate**, have the correct values.

Disable Tax Fields

Note: This customization is for accounts that use Legacy Tax, where SuiteTax feature is **not** enabled.

The script in this tutorial disables tax fields on U.S. sales transactions only. The script assumes that the Per Line Taxes on Transaction preference isn't enabled for U.S., and that the tax fields appear as transaction body fields on the Accounting subtab of the transaction record. For information about U.S. tax preferences, see the help topic [Setting U.S. Tax Preferences](#).

This customization shows how to control who can edit tax fields on the following sales transactions: Cash Refund, Cash Sale, Credit Memo, Invoice, Sales Order, and Return Authorization.

Only users with the Administrator role and users with other specific roles have permission to edit tax fields. Sales transactions are created as usual, and taxes depend on how the customer and items are set up.

This customization uses a script parameter to determine which roles can edit tax fields. If the user is not logged in with an Administrator role or a role that is specified in the parameter, the script disables the tax fields. This occurs whether the sale is taxable or not.

The script in this tutorial disables the following tax fields for U.S. sales transactions:

Field Name	Field Type	Internal ID	Location
Taxable	Checkbox	istaxable	Accounting subtab > Tax information field group
Tax Item	Select	taxitem	Accounting subtab > Tax information field group
Tax Rate	Float	taxrate	Accounting subtab > Tax information field group

This project is available on the [SuiteCloud Project Repository on Oracle Samples GitHub](#). It includes the complete project customization setup with custom objects, unit tests, package file configuration, and more.

Customization Details

The customization for this use case includes:

- A script parameter (Tax Allowlist) to specify which roles are allowed to modify the tax fields on the transaction record
- A client script triggered on the pageInit entry point

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Write the Script](#)
- [Step 2: Create the Script Record](#)
- [Step 3: Deploy the Script](#)
- [Step 4: Create and Set the Script Parameters](#)
- [Step 5: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	Ensure the following features are enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ Multiple Currencies – Lets you select currencies on transactions and set exchange rates. For more information, see the help topic Enabling Features.  Note: This tutorial deals with subsidiaries. You need a NetSuite OneWorld account, which automatically supports subsidiaries.
Required Permissions	You need a role with access to the following:

	■ Scripts – Edit access ■ Script Deployments – Edit access ■ Sales Orders – Edit access For more information, see the help topic NetSuite Permissions Overview.
Other Requirements	None

Step 1: Write the Script

This script checks a script parameter to determine if the user can edit tax fields on a U.S. sales transaction.

Script Summary

The following table summarizes the script:

Script: Disable Tax Fields	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/runtime Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial doesn't explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ pageInit(scriptContext)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, cs_disableTaxFields.js).

If you would rather create your script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */

```

```

6
7 define(['N/runtime', 'N/log'], (runtime, log) => {
8   function pageInit (scriptContext) {
9     try {
10      const objRecord = scriptContext.currentRecord;
11      const stRecordType = objRecord.type;
12      const stFuncName = '_DisableTaxField_' + stRecordType;
13      if (scriptContext.mode !== 'copy') {
14        const objContext = runtime.getCurrentScript();
15        const stAllowList = objContext.getParameter({
16          name: 'custscript_tax_allowlist'
17        });
18        const stUserRole = runtime.getCurrentUser().role;
19        const arrExclusionRoleList = stAllowList.split(',');
20        if (NSUtil.isEmpty(stAllowList)) {
21          log.error({
22            title: stFuncName,
23            details: 'Script parameter is empty. Script will now exit'
24          });
25          return
26        }
27        const taxitem = objRecord.getField({
28          fieldId: 'taxitem'
29        });
30        if (taxitem && !NSUtil.inArray(stUserRole, arrExclusionRoleList)) {
31          const istaxable = objRecord.getField({
32            fieldId: 'istaxable'
33          });
34          const taxrate = objRecord.getField({
35            fieldId: 'taxrate'
36          });
37          istaxable.isDisabled = true;
38          taxitem.isDisabled = true;
39          taxrate.isDisabled = true;
40        }
41      }
42    } catch (error) {
43      if (error !== undefined) {
44        log.error({
45          title: 'Process Error in pageInit',
46          details: error
47        });
48        throw error;
49      }
50    }
51  }
52  return {
53    pageInit: pageInit
54  };
55 });
56 const NSUtil = {
57   isEmpty: function (stValue) {
58     if ((stValue === '') || (stValue === null) || (stValue === undefined)) {
59       return true;
60     }
61   },
62   inArray: function (stValue, arrValue) {
63     let bIsValueFound = false;
64     for (let i = arrValue.length; i >= 0; i--) {
65       if (stValue == arrValue[i]) {
66         bIsValueFound = true
67         break;
68       }
69     }
70     return bIsValueFound;
71   }
72 }

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point functions
- Complete the pageInit function
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.x (2.0 or 2.1) client script. The script uses two SuiteScript modules specified in the define statement:

- `N/runtime` – provides access to runtime information, including script parameters
- `N/log` – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/log'], (runtime, log) => {
8  });

```

Create the entry point functions

This script is triggered on the `pageInit` entry point when a sales transaction is created or edited. Sales transactions include cash refunds, cash sales, cash memos, invoices, sales orders, and return authorizations. A try-catch block logs any errors that might occur during script execution in the entry point function. Most of the script code goes in one of the try blocks.

Add the following entry point function definitions and initial try-catch block statements at the top of the `define` function:

```

1  function pageInit(scriptContext) {
2      try {
3      } catch(error) {
4          if (error !== undefined) {
5              log.error({
6                  title: 'Process Error in pageInit,
7                  details: error

```

```

8         });
9         throw error;
10    }
11 }
12 }

```

Complete the pageInit function

Add the following code at the top of the try block of the pageInit function:

```

1  const objRecord = scriptContext.currentRecord;
2  const stRecordType = objRecord.type;
3  const stFuncName = 'pageInit_DisableTaxFields_' + stRecordType;

```

If the transaction is in create or edit mode, we want to get the script parameter and user role information.

Add the following code into the try block of the pageInit function:

```

1  if (scriptContext.mode !== 'copy') {
2      const objContext = runtime.getCurrentScript();
3      const stAllowList = objContext.getParameter({
4          name: 'custscript_tax_allowlist'
5      });
6      const stUserRole = runtime.getCurrentUser().role;
7      const arrExclusionRoleList = stAllowList.split(',');

```

If the script parameter allow list parameter is empty, we need to log an error.

Add the following code within the try block of the pageInit function:

```

1  if (NSUtil.isEmpty(stAllowList)) {
2      log.error({
3          title: stFuncName,
4          details: 'Script parameter is empty. Script will now exit'
5      });
6      return
7  }

```

We'll disable fields according to the U.S. subsidiary of the customer on the transaction record. To do this, we'll check if the tax item field exists when a customer is selected. If the tax item field exists and the user role doesn't match the allow list parameter, we'll disable the **Taxable**, **Tax Item**, and **Tax Rate** fields.

Add the following code into the try block of the pageInit function:

```

1  const taxitem = objRecord.getField({
2      fieldId: 'taxitem'
3  });
4  if (taxitem && !NSUtil.inArray(stUserRole, arrExclusionRoleList)) {
5      const istaxable = objRecord.getField({
6          fieldId: 'istaxable'
7      });
8      const taxrate = objRecord.getField({
9          fieldId: 'taxrate'
10     });
11     istaxable.isDisabled = true;
12     taxitem.isDisabled = true;
13     taxrate.isDisabled = true;
14 }

```

The pageInit function should now be complete.

Create the return statement

This script associates the `pageInit` function to the `pageInit` entry point.

Add the following code immediately above the closing `});` in your script:

```
1 | return {
2 |     pageInit: pageInit
3 | };
```

Add the utility functions

A utility function checks the script parameters and compares them to the user's role and the subsidiary of the transaction.

Add the following code at the end of your script following the end of the `define` function.

```
1 | const NSUtil = {
2 |     isEmpty : function(stValue) {
3 |         if ((stValue === '') || (stValue === null) || (stValue === undefined)) {
4 |             return true;
5 |         }
6 |     },
7 |     isArray : function(stValue, arrValue) {
8 |         let bIsValueFound = false;
9 |         for (let i = arrValue.length; i >= 0; i--) {
10 |             if (stValue == arrValue[i]) {
11 |                 bIsValueFound = true;
12 |                 break;
13 |             }
14 |         }
15 |         return bIsValueFound;
16 |     }
17 | }
```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a `.js` file (for example, `cs_disableTaxFields.js`).

Step 2: Create the Script Record

Now that you've completed the script, you can upload it to the File Cabinet and create a script record for it.

For more information about creating script records, see the help topic [Creating a Script Record](#).

To create the script record:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.

3. Select your script from the **Script File** list and click **Create Script Record**. The Script page appears.
4. On the Script page, enter the following values:

Field	Value
Name	Disable Tax Fields
ID	_cs_disable_tax_fields NetSuite prefixes the ID with customscript .
Description	This script disables tax fields on a transaction record. It also ensures that only users with an Administrator role or one of the roles specified in the script parameter can edit tax fields.

5. Set any other fields on the script record as required.
6. Click **Save and Deploy**. The Script Deployment page appears. Continue with [Step 3: Deploy the Script](#).

Step 3: Deploy the Script

After you create the script record for your script, you can create a script deployment record for it. A script deployment record determines how, when, and for whom the script runs. For this tutorial, we'll create multiple deployments, one for each of the following sales transactions:

- [Cash Refund](#)
- [Cash Sale](#)
- [Credit Memo](#)
- [Invoice](#)
- [Sales Order](#)
- [Return Authorization](#)

For more information about script deployment records, see the help topic [Script Deployment](#).

Cash Refund

Follow this procedure to deploy the script on cash refund transactions.

To deploy the script on cash refund transactions:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#).
2. Click **New Deployment**.
3. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Cash Refund
ID	_cs_disable_tax_fields_cr NetSuite prefixes the ID with custdeploy .
Status	Testing

Field	Value
	The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Internal Roles	Check Select All
Audience > External Roles	Select all external roles.

4. Click **Save**.

Cash Sale

Follow this procedure to deploy the script on cash sale transactions.

To deploy the script on cash sale transactions:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#).
2. Click **New Deployment**.
3. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Cash Sale
ID	_cs_disable_tax_fields_cs NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Internal Roles	Check Select All
Audience > External Roles	Select all external roles.

4. Click **Save**.

Credit Memo

Follow this procedure to deploy the script on credit memo transactions.

To deploy the script on credit memo transactions:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#)).
2. Click **New Deployment**.
3. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Credit Memo
ID	_cs_disable_tax_fields_cm NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors in the script to the Execution Log subtab of the script deployment record.
Audience > Internal Roles	Check Select All
Audience > External Roles	Select all external roles.

4. Click **Save**.

Invoice

Follow this procedure to deploy the script on invoice transactions.

To deploy the script on invoice transactions:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#)).
2. Click **New Deployment**.
3. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Invoice
ID	_cs_disable_tax_fields_in NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status let the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Internal Roles	Check Select All
Audience > External Roles	Select all external roles.

4. Click **Save**.

Sales Order

Follow this procedure to deploy the script on sales order transactions.

To deploy the script on sales order transactions:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#).
2. Click **New Deployment**.
3. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_cs_disable_tax_fields_so NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Internal Roles	Check Select All
Audience > External Roles	Select all external roles.

4. Click **Save**.

Return Authorization

Follow this procedure to deploy the script on return authorization transactions.

To deploy the script on return authorization transactions:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#).
2. Click **New Deployment**.
3. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Return Authorization
ID	_cs_disable_tax_fields_ra NetSuite prefixes the ID with custdeploy .

Field	Value
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Internal Roles	Check Select All
Audience > External Roles	Select all external roles.

4. Click **Save**.

To verify that you have six deployments:

1. Go to Customization > Scripting > Scripts and click **Deployments** for the script record you created in [Step 2: Create the Script Record](#).
2. Verify all six script deployments are listed:
 - custdeploy_cs_disable_tax_fields_cr
 - custdeploy_cs_disable_tax_fields_cs
 - custdeploy_cs_disable_tax_fields_cm
 - custdeploy_cs_disable_tax_fields_in
 - custdeploy_cs_disable_tax_fields_so
 - custdeploy_cs_disable_tax_fields_ra

Step 4: Create and Set the Script Parameters

This script uses a script parameter to specify which roles can edit tax fields on a transaction record.

For more information about creating and setting script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

To create the script parameters:

1. Go to Customization > Scripting > Scripts.
2. Find your script in the list and click **Edit** next to the script name.
3. On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.
4. On the Script Field page, enter the following values:

Label	ID	Type	Description	Preference
Tax Allowlist	_tax_allowlist NetSuite prefixes the ID with custscript , so the	Free-Form Text	This parameter determines which roles can edit tax fields on sales transactions.	Leave blank.

Label	ID	Type	Description	Preference
	script uses custscript_tax_allowlist .			

5. To save the script record with the updated script parameter, click **Save**.

After you create the script parameters, you must set the value for each parameter on each script deployment record you created in Step 3.

To set the script parameter:

1. Go to Customization > Scripts > Script Deployments.
2. For each script deployment:
 1. Click **Edit** next to the deployment ID.
 2. Click the **Parameters** subtab.
 3. Enter the value for the script parameter on each script deployment record:

Deployment Record	Tax Allowlist script parameter
custdeploy_cs_disable_tax_fields_cr	3 (this is the internal ID of the Administrator role) Add the internal IDs of other roles that can edit tax fields on sales transactions. Separate each ID with a comma, for example, 3,1,1001
custdeploy_cs_disable_tax_fields_cs	
custdeploy_cs_disable_tax_fields_cm	
custdeploy_cs_disable_tax_fields_in	
custdeploy_cs_disable_tax_fields_so	
custdeploy_cs_disable_tax_fields_ra	

3. Click **Save**. Repeat these steps for each script deployment record.

For more information about creating script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

Step 5: Test the Solution

After you create the script record and deploy your script, you can test it for two main cases:

- Create a sales transaction with a role that's allowed to edit tax fields
- Create a sales transaction with any other role that has Edit access to sales transactions

To test the solution for a role that is allowed to edit tax fields:

1. Log in with a role specified in the script parameter.
2. Go to Transaction > Sales > Enter Sales Orders to create a new sales order.
3. In the **Customer** field, select a U.S. customer.
4. On the **Items** subtab, select an item.
5. On the **Accounting** subtab, ensure that the following tax fields are editable and set values for these fields:
 - Taxable

- Tax Item
 - Tax Rate
6. Click **Save**.
 7. Edit the sales order that you created in the previous step. Verify that the tax fields are still editable in edit mode.

To test the solution for a role that is not allowed to edit tax fields:

1. Log in with a role that isn't specified in the script parameter. Make sure that the role has permission to edit sales transactions.
2. Go to Transaction > Sales > Enter Sales Orders to create a new sales order.
3. In the **Customer** field, select a U.S. customer.
4. On the **Items** subtab, select an item.
5. On the **Accounting** subtab, ensure that the following tax fields aren't editable:
 - Taxable
 - Tax Item
 - Tax Rate
6. If the subsidiary or customer has default tax values set up, you should be able to save the sales order. Click **Save**.
7. Edit the sales order that you created in the previous step. Verify that tax fields are still editable in edit mode.

Populate Item Substitution

This use case shows you how to let a user select pre-defined substitute items for a sales order when there's not enough of a selected item.

The use case adds a custom button to the sales order that the user can click when they've physically picked a substitute item. They can then enter information such as quantity and serial number to keep inventory records accurate.

Customization Details

The customization for this use case includes:

- A custom field (Primary Substitute) to store the preferred substitute item if the original item is unavailable
- A custom field (Substitute Item ID) to store the ID of the primary substitute item
- A custom field (Substitute Available Qty) to store the available quantity for the primary substitute item
- A custom field (Substitute Processed) that provides an indication of when the substitute item processing is complete
- A script parameter (Client Script Path) to store the path to the script that executes a Suitelet
- A script parameter (Helper Client Script Path) to store the path to the helper script that populates fields on the Suitelet

- A script parameter (Max Lines) to store the maximum number of lines on the Suitelet
- A script parameter (Inventory Numbers Search) to store the ID of the Inventory Numbers saved search
- A script parameter (Inventory Adjustment Account) to store the ID of an Inventory Adjustment account
- A script parameter (Item ID from Name Saved Search) to store the ID of the From Name saved search
- A script parameter (Substitute Available Qty Empty) to store an alert message that is displayed when a substitute item has 0 quantity
- A saved search (Inventory Numbers) that returns serial numbers for inventory items
- A saved search (Item ID from Name) that returns item identifiers associated with items when a name is specified
- A user event script triggered on the beforeLoad entry point
- A client script triggered on the validateLine entry point
- A helper client script triggered on the executeSuitelet and saveRecord entry points
- A Suitelet triggered on the OnRequest entry point when the user clicks the custom button

Steps in this tutorial to complete this customization:

- [Before You Begin](#)
- [Step 1: Create the Custom Fields and Saved Searches](#)
- [Step 2: Write the Add Fulfill with Substitutes Button Script](#)
- [Step 3: Write the Item Substitution Button Helper Script](#)
- [Step 4: Write the Item Substitution Script](#)
- [Step 5: Write the Validate Item Substitution Script](#)
- [Step 6: Create the Script Records](#)
- [Step 7: Deploy the Scripts](#)
- [Step 8: Create and Set the Script Parameters](#)
- [Step 9: Test the Solution](#)

Before You Begin

The following table lists features, permissions, and other requirements necessary for performing this tutorial and implementing the solution:

Required Features	Ensure the following features are enabled in your account: ■ Client SuiteScript – Lets you apply client scripts to records and forms. ■ Server SuiteScript – Lets you attach server scripts to records. ■ File Cabinet – Lets you store your script files in the NetSuite File Cabinet. ■ Serialized Inventory or Lot Tracking – These features allow purchase and sale of inventory using unique serial numbers or lot numbers, respectively. For more information, see the help topic Enabling Features
Required Permissions	You need a role with access to the following: ■ Scripts – Edit access ■ Script Deployments – Edit access

	■ Transaction Body Fields – Create access ■ Transaction Line Fields – Create access ■ Saved Searches – Create access ■ Sales Orders – Edit and Delete access
Other Requirements	You must already have pre-defined substitute items. This tutorial doesn't cover assigning substitute items.

Step 1: Create the Custom Fields and Saved Searches

This customization uses four custom fields. The **Primary Substitute** field stores the preferred substitute item if the original item isn't available. The **Substitute Item ID** field stores the ID of the primary substitute item. The **Substitute Available Qty** field stores the available quantity of the primary substitute item. The **Substitute Processed** field indicates whether the processing for the substitute item is complete. Each custom field is a custom transaction line field added to the sales order record.

This customization also uses two saved searches. The **Inventory Numbers** saved search returns serial numbers for inventory items, sorted by number. The **Item ID from Name** saved search returns item identifiers when an item name is specified, sorted by name.

To create the Primary Substitute field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New and enter the following values:

Field	Value
Label	Primary Substitute Item
ID	_item_substitute
Type	List/Record
List/Record	Item
Store Value	Checked
Applies To	Sale Item

2. Click **Save**.

To create the Substitute Item ID field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New and enter the following values:

Field	Value
Label	Substitute Item ID
ID	_sub_item_id
Type	Integer Number
Store Value	Checked
Applies To	Sale Item

2. Click **Save**.

To create the Substitute Available Qty field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New and enter the following values:

Field	Value
Label	Substitute Available Qty
ID	_sub_avail_qty
Type	Integer Number
Store Value	Checked
Applies To	Sale Item

2. Click **Save**.

To create the Substitute Processed field:

1. Go to Customization > Lists, Records, Fields > Transaction Line Fields > New and enter the following values:

Field	Value
Label	Substitute Processed
ID	_line_processed_substitute
Type	Check Box
Store Value	Checked
Applies To	Sale Item

2. Click **Save**.

To create the Inventory Numbers search:

1. Go to Reports > Saved Searches > All Saved Searches > New.
2. Click Inventory Numbers.
3. On the Inventory Numbers page, enter or select the following values:

Field	Value
Search Title	Inventory Numbers
ID	_inventory_numbers
Public	Checked
Results > Sort By	Number
Results > Output Type	Normal
Columns > Fields	Number Remove all other fields that may have been preselected.

4. Click **Save & Run** to ensure that your search runs and returns the correct results.

To create the Item ID from Name search:

1. Go to Reports > Saved Searches > All Saved Searches > New.
2. Click Item.
3. On the Saved Item Search page, enter or select the following values:

Field	Value
Search Title	Item ID from Name
ID	_itemid_from_name
Public	Checked
Criteria > Filter	Type
Saved Item Search Pop-up Window > Type	none of Description, Discount, Markup, Non-Inventory Item, Other Charge, Payment, Service, Subtotal
Results > Sort By	Name
Results > Output Type	Normal
Columns > Fields	Name and Internal ID Remove all other fields that may have been preselected.

4. Click **Save & Run** to confirm that your search will run and return the proper results.

For more information about creating custom fields and searches, see the following help topics:

- [Creating a Custom Field](#)
- [Creating Custom Transaction Line Fields](#)
- [Saved Searches](#)

Step 2: Write the Add Fulfill with Substitutes Button Script

This script adds the Fulfill With Substitutes button on the sales order record.

Script Summary

The following table summarizes this script:

Script: Add Fulfill with Substitutes Button	
Script Type	SuiteScript 2.x User Event Script Type
Modules Used	■ N/runtime Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You do not need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial doesn't explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.

Script: Add Fulfill with Substitutes Button

Entry Points	■ <code>beforeLoad(context)</code>
--------------	------------------------------------

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, ue_ItemSubstitution.js).

If you would rather create this script step by step, follow the steps in [Build the Script](#).



Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#)

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/log'], (runtime, log) => {
8      function beforeLoad(scriptContext) {
9          const stLogTitle = 'beforeLoad_addButton';
10         try {
11             if (scriptContext.type === scriptContext.UserEventType.EDIT) {
12                 const clientScriptPath = runtime.getCurrentScript().getParameter({
13                     name: 'custscript_client_script_path'
14                 });
15                 if (isEmpty(clientScriptPath)) {
16                     log.error({
17                         title: stLogTitle,
18                         details: 'You must specify the location of the client script path in a script parameter (custscript_client_script_path).'
19                     });
20                     return;
21                 }
22                 const currentRecord = scriptContext.newRecord;
23                 const form = scriptContext.form;
24                 form.clientScriptModulePath = clientScriptPath;
25                 form.addButton({
26                     id: 'custpage_execute_suitelet_button',
27                     label: 'Fulfill with Substitutes',
28                     functionName: 'executeSuitelet'
29                 });
30             }
31             return true;
32         } catch(error) {
33             log.error({
34                 title: stLogTitle,
35                 details: error
36             });
37         }
38     }
39 }

```

```

38     }
39     function isEmpty(stValue) {
40         return stValue === '' || stValue === null || stValue === undefined;
41     }
42     return {
43         beforeLoad: beforeLoad
44     };
45 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Check the scriptContext
- Check the script parameter
- Get the current sales order record and form
- Add a button to the sales order form
- Create the isEmpty function
- Create the return statement
- Save your script file



Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it is a SuiteScript 2.1 user event script. The script uses two SuiteScript modules specified in the define statement:

- N/runtime – provides access to runtime settings for company, script, sessions, system, user, and version
- N/log – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:



Note: This tutorial script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the require function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/log'], (runtime, log) => {

```

```
8 | });
```

Create the entry point function

This script is triggered on the `beforeLoad` entry point when the user creates a new sales order. A try-catch block logs any errors that might occur during script execution. Most of the script code goes in the try block.

Add the following function definition and initial try-catch block statements at the top of the define function:

```
1 function beforeLoad(scriptContext) {
2     const stLogTitle = 'beforeLoad_addButton';
3     try {
4         return true;
5     } catch(error) {
6         log.error({
7             title: stLogTitle,
8             details: error
9         });
10    }
11 }
```

Check the scriptContext

This function adds the custom button only when the user edits an existing sales order.

Add the following code as the first statement in the try block above the `return true;` statement:

```
1 if (scriptContext.type === scriptContext.UserEventType.EDIT) {
2 }
```

Check the script parameter

This function relies on a script parameter to specify the location of the client script that executes the Suitelet.

Add the following code within the if statement that checks the `scriptContext`:

```
1 const clientScriptPath = runtime.getCurrentScript().getParameter({
2     name: 'custscript_client_script_path'
3 });
4 if (isEmpty(clientScriptPath)) {
5     log.error({
6         title: stLogTitle,
7         details: 'You must specify the location of the client script path in a script parameter (custscript_client_script_path).'
8     });
9     return;
10 }
```

Get the current sales order record and form

This function needs access to the sales order record and form.

Add the following code within the if statement that checks the script context:

```
1 | const currentRecord = scriptContext.newRecord;
```

```

2 | const form = scriptContext.form;
3 | form.clientScriptModulePath = clientScriptPath;

```

Add a button to the sales order form

This function adds a button to the sales order form that runs the Suitelet when the user clicks it.

Add the following code within the if statement that checks the script context:

```

1 | form.addButton({
2 |   id: 'custpage_execute_suitelet_button',
3 |   label: 'Fulfill with Substitutes',
4 |   functionName: 'executeSuitelet'
5 | });

```

Create the isEmpty function

This user event script uses a support function to determine if the script parameter is empty, null, or undefined.

Add the following code after the end of the beforeLoad function:

```

1 | function isEmpty(stValue) {
2 |   return stValue === '' || stValue === null || stValue === undefined;
3 | }

```

Create the return statement

This script associates the beforeLoad function with the beforeLoad user event entry point.

Add the following code immediately above the closing `});` in your script:

```

1 | return {
2 |   beforeLoad: beforeLoad
3 | };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, ue_ItemSubstitution.js).

Step 3: Write the Item Substitution Button Helper Script

This script runs the Suitelet when the Fulfill With Substitutes button is clicked.

Script Summary

The following table summarizes this script:

Script: Item Substitution Button Helper	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/ui/dialog Module ■ N/url Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You do not need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial explicitly loads this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ <code>executeSuitelet</code> ■ <code>saveRecord(scriptContext)</code>

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, `cs_itemSubstitution_helper.js`).

If you would rather create this script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/url', 'N/currentRecord', 'N/ui/dialog', 'N/log'], (url, currentRecord, dialog, log) => {
8      function executeSuitelet() {
9          try {
10             const record = currentRecord.get();
11             const recordID = record.id;
12             console.log('record ID in executeSuitelet', recordID);
13             const suiteUrl = url.resolveScript({
14                 scriptId: 'customscript_sl_item_substitution',
15                 deploymentId: 'customdeploy_sl_item_substitution',
16                 params: {
17                     record_id: recordID
18                 },
19                 returnExternalUrl: false
20             });
21             window.open(suiteUrl, 'Item Substitution', "width=1300,height=700");
22         } catch(error) {

```

```

23     alert(error.toString());
24 }
25 return true;
26 }
27 function saveRecord(scriptContext) {
28     try {
29         const currentRecord = scriptContext.currentRecord;
30         const countLines = currentRecord.getLineCount({
31             sublistId: 'so_lines_sublist'
32         });
33         for (let i = 0; i < countLines; i++) {
34             let substQty = currentRecord.getSublistValue({
35                 sublistId: 'so_lines_sublist',
36                 fieldId: 'item_substitute_qty',
37                 line: i
38             });
39             let serialNumbers = currentRecord.getSublistValue({
40                 sublistId: 'so_lines_sublist',
41                 fieldId: 'item_substitute_serial_nums',
42                 line: i
43             });
44             if (isEmpty(substQty) || isEmpty(serialNumbers)) {
45                 dialog.alert({
46                     title: 'Alert',
47                     message: 'Substitution Quantity and Serial Numbers can not be empty'
48                 });
49                 return false;
50             } else {
51                 let serialNumbersSplit = serialNumbers.split(',');
52                 let serialNumbersQty = serialNumbersSplit.length;
53                 if (serialNumbersQty !== substQty) {
54                     dialog.alert({
55                         title: 'Alert',
56                         message: 'Substitution Quantity and Serial Numbers Quantity needs to match.'
57                     });
58                     return false;
59                 }
60             }
61         }
62         const salesOrderURL = url.resolveRecord({
63             recordType: 'salesorder',
64             recordId: currentRecord.getValue({
65                 fieldId: 'sales_order_id'
66             }),
67             isEditMode: false
68         });
69         window.ischanged = false;
70         window.open(salesOrderURL);
71         window.close();
72     } catch(error) {
73         alert(error.toString());
74     }
75 }
76 function isEmpty(stValue) {
77     return stValue === '' || stValue === null || stValue === undefined;
78 }
79 return {
80     executeSuitelet: executeSuitelet,
81     saveRecord: saveRecord
82 };
83 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point functions
- Complete the executeSuitelet function

- Complete the `saveRecord` function
- Create the `isEmpty` function
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a `define` function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 client script. The script uses four SuiteScript modules specified in the `define` statement:

- `N/url` – lets you determine URL navigation paths within NetSuite or to format URL strings
- `N/currentRecord` – provides access to the record instance that you are currently working on.
- `N/ui/dialog` – lets you create a modal dialog that is displayed until a button on the dialog is pressed
- `N/log` – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and `define` function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```
1  /**
2   * @ApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/url', 'N/currentRecord', 'N/ui/dialog', 'N/log'], (url, currentRecord, dialog, log) => {
8  });
```

Create the entry point functions

This script is triggered on the `saveRecord` entry point when the user saves a new sales order with substitute item information from the Suitelet form. It is also triggered on the `executeSuitelet` entry point when the user clicks the Open Suitelet button. A try-catch block logs any errors that might occur during script execution. Most of the script code goes in the try block.

Add the following function definitions and initial try-catch block statements at the top of the `define` function:

```
1  function executeSuitelet() {
2    try {
3    } catch(error) {
4      alert(error.toString());
5    }
6    return true;
7  }
```

```

8 function saveRecord(scriptContext) {
9     try {
10    } catch(error) {
11        alert(error.toString());
12    }
13 }

```

Complete the executeSuitelet function

This helper client script defines an `executeSuitelet` function. Within this function, you will add code for the following:

- [Get the current record and ID](#)
- [Resolve the script URL](#)
- [Display the Suitelet](#)

Get the current record and ID

This function uses the current sales order record ID.

Add the following code at the top of the try block of the `executeSuitelet` function:

```

1 const record = currentRecord.get();
2 const recordID = record.id;
3 console.log('record ID in executeSuitelet', recordID);

```

Resolve the script URL

This function needs to know where the Suitelet is located.

Add the following code within the try block of the `executeSuitelet` function:

```

1 const suiteUrl = url.resolveScript({
2     scriptId: 'customscript_sl_item_substitution',
3     deploymentId: 'customdeploy_sl_item_substitution',
4     params: {
5         record_id: recordID
6     },
7     returnExternalUrl: false
8 });

```

Display the Suitelet

This function displays the Suitelet in its own window, with a specific size and title.

Add the following code within the try block of the `executeSuitelet` function:

```

1 window.open(suiteUrl, 'Item Substitution', "width=1300,height=700");

```

Complete the saveRecord function

This helper client script defines a `saveRecord` function to validate values entered by the user on the Suitelet. This function is called when the user clicks the **Save** button on the Suitelet. Within this function, you will add code for the following:

- Get the sales order and number of sublist lines
- Verify the sublist items
- Get the URL to the sales order
- Return to the sales order and close the Suitelet

Get the sales order and number of sublist lines

This function processes sublist lines on the sales order.

Add the following code at the top of the try block of the saveRecord function:

```
1 | const currentRecord = scriptContext.currentRecord;
2 | const countLines = currentRecord.getLineCount({
3 |   sublistId: 'so_lines_sublist'
4 | });
```

Verify the sublist items

This function iterates through all sublist items to verify the item quantity and serial numbers, using a for loop.

Add the following code within the try block of the saveRecord function:

```
1 | for (let i = 0; i < countLines; i++) {
2 | }
```

Get the quantity of the substitute item and serial numbers

This function gets the quantity of the substitute item ordered and the serial numbers for each one.

Add the following code within the for loop:

```
1 | let substQty = currentRecord.getSublistValue({
2 |   sublistId: 'so_lines_sublist',
3 |   fieldId: 'item_substitute_qty',
4 |   line: i
5 | });
6 | let serialNumbers = currentRecord.getSublistValue({
7 |   sublistId: 'so_lines_sublist',
8 |   fieldId: 'item_substitute_serial_nums',
9 |   line: i
10 | });
```

Verify the quantity and serial numbers

This function requires the **Substitute Item Quantity** and **Serial Number** fields, and there has to be a serial number for each item.

Add the following code within the for loop:

```
1 | if (isEmpty(substQty) || isEmpty(serialNumbers)) {
2 |   dialog.alert({
3 |     title: 'Alert',
4 |     message: 'Substitution Quantity and Serial Numbers can not be empty'
5 |   });
6 |   return false;
```

```

7 | } else {
8 |   let serialNumbersSplit = serialNumbers.split(',');
9 |   let serialNumbersQty = serialNumbersSplit.length;
10 |   if (serialNumbersQty !== substQty) {
11 |     dialog.alert({
12 |       title: 'Alert',
13 |       message: 'Substitution Quantity and Serial Numbers Quantity needs to match.'
14 |     });
15 |     return false;
16 |   }
17 | }

```

Get the URL to the sales order

This function will return to the sales order when the user saves the Suitelet and all values are verified. To do this, it needs the URL of the sales order.

Add the following code after the end of the for loop, within the try block of the saveRecord function.

```

1 | const salesOrderURL = url.resolveRecord({
2 |   recordType: 'salesorder',
3 |   recordId: currentRecord.getValue({
4 |     fieldId: 'sales_order_id'
5 |   }),
6 |   isEditMode: false
7 | });

```

Return to the sales order and close the Suitelet

This function displays the sales order when the Suitelet is closed, if the values entered are valid.

Add the following code within the try block of the saveRecord function:

```

1 | window.ischanged = false;
2 | window.open(salesOrderURL);
3 | window.close();

```

Create the isEmpty function

This helper client script uses a support function to determine if a value is empty, null, or undefined.

Add the following code after the end of the saveRecord function:

```

1 | function isEmpty(stValue) {
2 |   return stValue === '' || stValue === null || stValue === undefined;
3 | }

```

Create the return statement

This script associates the validateLine function with the validateLine client script entry point.

Add the following code immediately above the closing `});` in your script:

```

1 | return {
2 |   executeSuitelet: executeSuitelet,
3 |   saveRecord: saveRecord
4 | };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, cs_itemSubstitution_helper.js).

Step 4: Write the Item Substitution Script

This script reads the names, quantities, and primary substitutes from the items sublists and creates a Substitute Items sublist. Sales order line items are only populated in the Substitute Items sublist if the ordered quantity is greater than what's available for the original item. Two custom columns (Substitution Quantity and Serial Numbers) are also added to the sales order. When the order is saved, the Substitution Quantity and Serial Numbers values are used to create an inventory adjustment record. The inventory adjustment removes the ordered quantity from the substitute item and adds it to the original line item. The sales order line items' inventory detail subrecords are updated with the serial numbers entered.

The script handles errors if any serial numbers are missing for a serialized line item.

Script Summary

The following table summarizes this script:

Script: Item Substitution Suitelet	
Script Type	SuiteScript 2.x Suitelet Script Type
Modules Used	■ N/ui/serverWidget Module ■ N/runtime Module ■ N/record Module ■ N/search Module ■ N/http Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial does not explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ onRequest(params)

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, sl_itemSubstitution.js).

If you would rather create this script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#)

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/ui/serverWidget', 'N/runtime', 'N/record', 'N/search', 'N/http', 'N/log'], (widget, runtime, record, search, http, log)
8  => {
9      function onRequest(scriptContext) {
10         const stLogTitle = 'onRequest';
11         try {
12             if (scriptContext.request.method === http.Method.GET) {
13                 const maxLinesParam = runtime.getCurrentScript().getParameter({
14                     name: 'custscript_max_lines'
15                 });
16                 const recordID = scriptContext.request.parameters.record_id;
17                 let itemSubForm = widget.createForm({
18                     title: 'Item Substitution'
19                 });
20                 const headerTab = itemSubForm.addTab({
21                     id: 'headertab',
22                     label: 'Tab'
23                 });
24                 addSublistAndFields(itemSubForm, recordID, maxLinesParam);
25                 itemSubForm.addSubmitButton({
26                     label: 'Save'
27                 });
28                 itemSubForm.clientScriptModulePath = runtime.getCurrentScript().getParameter({
29                     name: 'custscript_sl_client_script_path'
30                 });
31                 scriptContext.response.writePage(itemSubForm);
32             } else if (scriptContext.request.method === http.Method.POST) {
33                 const recordID = scriptContext.request.parameters.sales_order_id;
34                 const subsidiary = scriptContext.request.parameters.sales_order_subsidary;
35                 const location = scriptContext.request.parameters.sales_order_location;
36                 const inventoryNumbersSearch = runtime.getCurrentScript().getParameter({
37                     name: 'custscript_inv_nums_search'
38                 });
39                 const account = runtime.getCurrentScript().getParameter({
40                     name: 'custscript_inv_adj_account'
41                 });
42                 const sublistLength = scriptContext.request.getLineCount({
43                     group: 'so_lines_sublist'
44                 });
45                 let arrItemsInfo = [];
46                 let arrSQLines = [];
47                 const objSavedSearch = search.load({
48                     id: inventoryNumbersSearch
49                 });
50                 for (let i = 0; i < sublistLength; i++) {
51                     let originalItem = scriptContext.request.getSublistValue({
52                         group: 'so_lines_sublist',
53                         name: 'item_id',
54                         line: i
55                     });
56                     let substituteItem = scriptContext.request.getSublistValue({
57                         group: 'so_lines_sublist',
58                         name: 'item_substitute_id',
59                         line: i
60                     });
61                 }
62             }
63         } catch (e) {
64             log.error(stLogTitle, e);
65         }
66     }
67 }

```

```

60         let substituteItemQty = scriptContext.request.getSublistValue({
61             group: 'so_lines_sublist',
62             name: 'item_substitute_qty',
63             line: i
64         });
65         let serialNumberField = scriptContext.request.getSublistValue({
66             group: 'so_lines_sublist',
67             name: 'item_substitute_serial_nums',
68             line: i
69         });
70         let salesOrderLine = scriptContext.request.getSublistValue({
71             group: 'so_lines_sublist',
72             name: 'sales_order_line',
73             line: i
74         });
75         arrSOLines.push(salesOrderLine);
76         let arrSerialNumbersIDs = [];
77         if (!isEmpty(serialNumberField)) {
78             let serialNumbersSplit = serialNumberField.split(',');
79             arrSerialNumbersIDs = getSerialNumbersIDs(serialNumbersSplit, objSavedSearch, arrSerialNumbersIDs);
80         }
81         let objectItemInfo = {};
82         objectItemInfo.originalItem = originalItem;
83         objectItemInfo.substituteItem = substituteItem;
84         objectItemInfo.substituteItemQty = substituteItemQty;
85         objectItemInfo.arrSerialNumbersIDs = arrSerialNumbersIDs;
86         arrItemsInfo.push(objectItemInfo);
87     }
88     const remainingUnitsBeforeCreation = runtime.getCurrentScript().getRemainingUsage();
89     log.debug({
90         title: stLogTitle,
91         details: 'remainingUnitsBeforeCreation: ' + remainingUnitsBeforeCreation
92     });
93     createInventoryAdjustments(subsidiary, location, account, arrItemsInfo);
94     const salesOrderRec = record.load({
95         type: record.Type.SALES_ORDER,
96         id: recordID,
97         isDynamic: true
98     });
99     for (let i = 0; i < arrSOLines.length; i++) {
100         salesOrderRec.selectLine({
101             sublistId: 'item',
102             line: i
103         });
104         salesOrderRec.setCurrentSublistValue({
105             sublistId: 'item',
106             fieldId: 'custcol_line_processed_substitute',
107             value: true
108         });
109         salesOrderRec.commitLine({
110             sublistId: 'item'
111         });
112     }
113     const salesOrderUpdated = salesOrderRec.save({
114         enableSourcing: true,
115         ignoreMandatoryFields: true
116     });
117     log.audit({
118         title: stLogTitle,
119         details: 'Sales Order Updated: ' + salesOrderUpdated
120     });
121     const remainingUnitsEND = runtime.getCurrentScript().getRemainingUsage();
122     log.debug({
123         title: stLogTitle,
124         details: 'remainingUnitsEND: ' + remainingUnitsEND
125     });
126 }
127 } catch(error) {
128     log.error({
129         title: stLogTitle,
130         details: error
131     });
132 }

```

```

133     }
134     function addSublistAndFields(form, recordID, maxLines) {
135         const stLogTitle = 'addSublistAndFields';
136         try {
137             const salesOrderID = form.addField({
138                 id: 'sales_order_id',
139                 type: ui.FieldType.TEXT,
140                 label: 'Sales Order ID'
141             });
142             const salesOrderSubsidiary = form.addField({
143                 id: 'sales_order_subsidiary',
144                 type: ui.FieldType.TEXT,
145                 label: 'Sales Order Subsidiary'
146             });
147             const salesOrderLocation = form.addField({
148                 id: 'sales_order_location',
149                 type: ui.FieldType.TEXT,
150                 label: 'Sales Order Location'
151             });
152             salesOrderID.updateDisplayType({
153                 displayType: ui.FieldDisplayType.HIDDEN
154             });
155             salesOrderSubsidiary.updateDisplayType({
156                 displayType: ui.FieldDisplayType.HIDDEN
157             });
158             salesOrderLocation.updateDisplayType({
159                 displayType: ui.FieldDisplayType.HIDDEN
160             });
161             const substituteItemsSublist = form.addSublist({
162                 id: 'so_lines_sublist',
163                 type: ui.SublistType.LIST,
164                 tab: 'headertab',
165                 label: 'Substitute Items'
166             });
167             const itemName = substituteItemsSublist.addField({
168                 id: 'item_name',
169                 type: ui.FieldType.TEXT,
170                 label: 'Item'
171             });
172             const itemID = substituteItemsSublist.addField({
173                 id: 'item_id',
174                 type: ui.FieldType.TEXT,
175                 label: 'Item ID'
176             });
177             const itemQty = substituteItemsSublist.addField({
178                 id: 'item_qty',
179                 type: ui.FieldType.FLOAT,
180                 label: 'Item Qty'
181             });
182             const itemSubstitute = substituteItemsSublist.addField({
183                 id: 'item_substitute',
184                 type: ui.FieldType.TEXT,
185                 label: 'Item Substitute'
186             });
187             const itemSubstituteID = substituteItemsSublist.addField({
188                 id: 'item_substitute_id',
189                 type: ui.FieldType.TEXT,
190                 label: 'Item Substitute ID'
191             });
192             const itemSubstituteQty = substituteItemsSublist.addField({
193                 id: 'item_substitute_qty',
194                 type: ui.FieldType.FLOAT,
195                 label: 'Substitute Quantity'
196             });
197             itemSubstituteQty.updateDisplayType({
198                 displayType: ui.FieldDisplayType.ENTRY
199             });
200             const itemSubstituteSerialNumbers = substituteItemsSublist.addField({
201                 id: 'item_substitute_serial_nums',
202                 type: ui.FieldType.TEXT,
203                 label: 'Substitute Serial Numbers'
204             });
205             itemSubstituteSerialNumbers.updateDisplayType({

```

```

206         displayType: ui.FieldDisplayType.ENTRY
207     });
208     const salesOrderLine = substituteItemsSublist.addField({
209         id: 'sales_order_line',
210         type: ui.FieldType.TEXT,
211         label: 'Sales Order Line'
212     });
213     populateSublistAndFields(salesOrderSubsidiary, salesOrderLocation, salesOrderID, substituteItemsSublist, recordID,
maxLines);
214 } catch(error) {
215     log.error({
216         title: stLogTitle,
217         details: error
218     });
219 }
220 }
221 function populateSublistAndFields(subsidiaryField, locationField, salesOrderIDField, sublist, recordID, maxLines) {
222     const stLogTitle = 'populateSublistAndFields';
223     try {
224         const soRecord = record.load({
225             type: record.Type.SALES_ORDER,
226             id: recordID,
227             isDynamic: true
228         });
229         salesOrderIDField.defaultValue = recordID;
230         subsidiaryField.defaultValue = soRecord.getValue('subsidiary');
231         locationField.defaultValue = soRecord.getValue('location');
232         const countLines = soRecord.getLineCount({
233             sublistId: 'item'
234         });
235         let line = 0;
236         for (let i = 0; i < countLines; i++) {
237             if (line < maxLines) {
238                 soRecord.selectLine({
239                     sublistId: 'item',
240                     line: i
241                 });
242                 let lineAlreadyProcessed = soRecord.getCurrentSublistValue({
243                     sublistId: 'item',
244                     fieldId: 'custcol_line_processed_substitute'
245                 });
246                 if (lineAlreadyProcessed === 'F' || lineAlreadyProcessed === false) {
247                     let itemQty = soRecord.getCurrentSublistValue({
248                         sublistId: 'item',
249                         fieldId: 'quantityavailable'
250                     });
251                     let orderQty = soRecord.getCurrentSublistValue({
252                         sublistId: 'item',
253                         fieldId: 'quantity'
254                     });
255                     if (itemQty < orderQty) {
256                         let item = soRecord.getCurrentSublistText({
257                             sublistId: 'item',
258                             fieldId: 'item'
259                         });
260                         sublist.setSublistValue({
261                             id: 'item_name',
262                             line: line,
263                             value: item
264                         });
265                         let itemID = soRecord.getCurrentSublistValue({
266                             sublistId: 'item',
267                             fieldId: 'item'
268                         });
269                         sublist.setSublistValue({
270                             id: 'item_id',
271                             line: line,
272                             value: itemID
273                         });
274                         sublist.setSublistValue({
275                             id: 'item_qty',
276                             line: line,
277                             value: itemQty

```

```

278     });
279     let itemSubstitute = soRecord.getCurrentSublistValue({
280         sublistId: 'item',
281         fieldId: 'custcol_item_substitute'
282     });
283     if (!isEmpty(itemSubstitute)) {
284         sublist.setSublistValue({
285             id: 'item_substitute',
286             line: line,
287             value: itemSubstitute
288         });
289         let itemSubstituteID = soRecord.getCurrentSublistValue({
290             sublistId: 'item',
291             fieldId: 'custcol_sub_item_id'
292         });
293         sublist.setSublistValue({
294             id: 'item_substitute_id',
295             line: line,
296             value: itemSubstituteID
297         });
298     }
299     sublist.setSublistValue({
300         id: 'sales_order_line',
301         line: line,
302         value: i
303     });
304     line++;
305 }
306 }
307 }
308 }
309 } catch(error) {
310     log.error({
311         title: stLogTitle,
312         details: error
313     });
314 }
315 }
316 function getSerialNumbersIDs(serialNumbersSplit, objSavedSearch, arrSerialNumbersIDs) {
317     const stLogTitle = 'getSerialNumbersIDs';
318     try {
319         let namesFilter = [];
320         let index = 0;
321         for (let i = 0; i < serialNumbersSplit.length; i++) {
322             namesFilter.push(['inventorynumber', 'IS', serialNumbersSplit[i]]);
323             if (index !== (serialNumbersSplit.length - 1)) {
324                 namesFilter.push('OR');
325             }
326             index++;
327         }
328         const results = extendedSearch('inventorynumber', objSavedSearch, namesFilter);
329         for (let i = 0; i < results.length; i++) { let obj = new Object(); obj.internalID = results[i].id; obj.serialName =
330             results[i].getValue('inventorynumber'); arrSerialNumbersIDs.push(obj); } return arrSerialNumbersIDs; } catch(error) { log.er
331             ror({ title: stLogTitle, details: error }); } } function extendedSearch(stRecordType, objSavedSearch, arrSearchFilter, arrSearchCol
332             umn) { if (stRecordType === null && objSavedSearch === null) { error.create({ name: 'SSS_MISSING_REQD_ARGUMENT', message: 'search:
333             Missing a required argument. Either stRecordType or objSavedSearch should be provided.', notifyOff: false }); } let arrRe
334             turnSearchResults = new Array(); const maxResults = 1000; if (objSavedSearch !== null) { objSavedSearch.filters = []; if (ar
335             rSearchFilter !== null) { if (arrSearchFilter[0] instanceof Array || (typeof arrSearchFilter[0] === 'string')) { objSavedSearch.fil
336             terExpression = objSavedSearch.filterExpression.concat(arrSearchFilter); } else { objSavedSearch.filters = objSavedSearch.fil
337             ters.concat(arrSearchFilter); } } if (arrSearchColumn !== null) { objSavedSearch.columns = objSavedSearch.columns.concat(ar
338             rSearchColumn); } } else { let objSavedSearch = search.create({ type: stRecordType }); if (arrSearchFilter !== null) { if (ar
339             rSearchFilter[0] instanceof Array || (typeof arrSearchFilter[0] === 'string')) { objSavedSearch.filterExpression = arrSearchFil
340             ter; } else { objSavedSearch.filters = arrSearchFilter; } } if (arrSearchColumn !== null) { objSavedSearch.columns = arrSearchCol
341             umn; } } const objResultset = objSavedSearch.run(); let intSearchIndex = 0; let arrResultSlice = null; do { arrResultSlice = objRe
342             sultset.getRange(intSearchIndex, intSearchIndex + maxResults); if (arrResultSlice === null) { break; } arrReturnSearchResults =
343             arrReturnSearchResults.concat(arrResultSlice); intSearchIndex = arrReturnSearchResults.length; } while (arrResultSlice.length >=
344             maxResults);
345     return arrReturnSearchResults;
346 }
347 }
348 function createInventoryAdjustments(subsidiary, location, account, arrItemsInfo) {
349     const stLogTitle = 'createInventoryAdjustments';
350     try {
351         let inventoryAdjustmentDeletion = record.create({

```



```

336         type: record.Type.INVENTORY_ADJUSTMENT,
337         isDynamic: true
338     });
339     inventoryAdjustmentDeletion.setValue({
340         fieldId: 'subsidiary',
341         value: subsidiary
342     });
343     inventoryAdjustmentDeletion.setValue({
344         fieldId: 'adjlocation',
345         value: location
346     });
347     inventoryAdjustmentDeletion.setValue({
348         fieldId: 'account',
349         value: account
350     });
351     for (let j = 0; j < arrItemsInfo.length; j++) { let objectItemInfo = arrItemsInfo[j]; inventoryAdjustmentDeletion.s
electNewLine({ sublistId: 'inventory' }); inventoryAdjustmentDeletion.setCurrentSublistValue({ sublistId: 'inventory', field
Id: 'item', value: objectItemInfo.substituteItem }); inventoryAdjustmentDeletion.setCurrentSublistValue({ sublistId: 'in
ventory', fieldId: 'location', value: location }); inventoryAdjustmentDeletion.setCurrentSublistValue({ sublistId: 'inven
tory', fieldId: 'adjustqtyby', value: (objectItemInfo.substituteItemQty * (-1)) }); let itemTypeLookup = search.lookupField
s({ type: search.Type.ITEM, id: objectItemInfo.substituteItem, columns: 'recordtype' }); if (!isEmpty(itemTypeLookup)) { let item
Type = itemTypeLookup.recordtype; if (itemType === 'serializedinventoryitem') { if (objectItemInfo.arrSerialNumbersIDs.length > 0)
{
352         let inventoryDetailIA = inventoryAdjustmentDeletion.getCurrentSublistSubrecord({
353             sublistId: 'inventory',
354             fieldId: 'inventorydetail'
355         });
356         for (let index = 0; index < objectItemInfo.arrSerialNumbersIDs.length; index++) {
357             inventoryDetailIA.selectNewLine({
358                 sublistId: 'inventoryassignment',
359             });
360             let line = inventoryDetailIA.getCurrentSublistIndex({
361                 sublistId: 'inventoryassignment'
362             });
363             inventoryDetailIA.setCurrentSublistValue({
364                 sublistId: 'inventoryassignment',
365                 fieldId: 'issueinventorynumber',
366                 value: JSON.parse(objectItemInfo.arrSerialNumbersIDs[index]).internalID
367             });
368             inventoryDetailIA.commitLine({
369                 sublistId: 'inventoryassignment'
370             });
371         }
372     }
373 }
374 }
375     inventoryAdjustmentDeletion.commitLine({
376         sublistId: 'inventory'
377     });
378 }
379     const invAdjIDDeletion = inventoryAdjustmentDeletion.save({
380         enableSourcing: true,
381         ignoreMandatoryFields: true
382     });
383     log.audit({
384         title: 'Inventory Adjustment deletion: ',
385         details: invAdjIDDeletion
386     });
387     let inventoryAdjustmentAddition = record.create({
388         type: record.Type.INVENTORY_ADJUSTMENT,
389         isDynamic: true
390     });
391     inventoryAdjustmentAddition.setValue({
392         fieldId: 'subsidiary',
393         value: subsidiary
394     });
395     inventoryAdjustmentAddition.setValue({
396         fieldId: 'adjlocation',
397         value: location
398     });
399     inventoryAdjustmentAddition.setValue({
400         fieldId: 'account',
401         value: account

```

```

402     });
403     for (let k = 0; k < arrItemsInfo.length; k++) { let objectItemInfoAdd = arrItemsInfo[k]; inventoryAdjustmentAdd
    dition.selectNewLine({ sublistId: 'inventory' }); inventoryAdjustmentAddition.setCurrentSublistValue({ sublistId: 'inven
    tory', fieldId: 'item', value: objectItemInfoAdd.originalItem }); inventoryAdjustmentAddition.setCurrentSublistValue({ sub
    listId: 'inventory', fieldId: 'location', value: location }); inventoryAdjustmentAddition.setCurrentSublistValue({ subli
    stId: 'inventory', fieldId: 'adjustqtyby', value: objectItemInfoAdd.substituteItemQty }); let itemTypeLookup = search.lookupField
    s({ type: search.Type.ITEM, id: objectItemInfoAdd.originalItem, columns: 'recordtype' }); if (!isEmpty(itemTypeLookup)) { let item
    Type = itemTypeLookup.recordtype; if (itemType === 'serializedinventoryitem') { if (objectItemInfoAdd.arrSerialNumbersID
    s.length > 0) {
404         let inventoryDetailIAAddition = inventoryAdjustmentAddition.getCurrentSublistSubrecord({
405             sublistId: 'inventory',
406             fieldId: 'inventorydetail'
407         });
408         for (let index1 = 0; index1 < objectItemInfoAdd.arrSerialNumbersIDs.length; index1++) {
409             inventoryDetailIAAddition.selectNewLine({
410                 sublistId: 'inventoryassignment',
411             });
412             inventoryDetailIAAddition.setCurrentSublistValue({
413                 sublistId: 'inventoryassignment',
414                 fieldId: 'receiptinventorynumber',
415                 value: objectItemInfoAdd.arrSerialNumbersIDs[index1].serialName
416             });
417             inventoryDetailIAAddition.commitLine({
418                 sublistId: 'inventoryassignment'
419             });
420         }
421     }
422 }
423 }
424 inventoryAdjustmentAddition.commitLine({
425     sublistId: 'inventory'
426 });
427 }
428 const invAdjIDAddition = inventoryAdjustmentAddition.save({
429     enableSourcing: true,
430     ignoreMandatoryFields: true
431 });
432 log.audit({
433     title: 'Inventory Adjustment addition: ',
434     details: invAdjIDAddition
435 });
436 } catch(error) {
437     log.error({
438         title: stLogTitle,
439         details: error
440     });
441 }
442 }
443 function isEmpty(stValue) {
444     return stValue === '' || stValue === null || stValue === undefined;
445 }
446 return {
447     onRequest: onRequest
448 };
449 });

```

Build the Script

You can write the user event script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Create the Item Substitution form on a GET request
- Create the inventory adjustment record on a POST request
- Create the addSublistAndFields function
- Create the populateSublistAndFields function

- Create the `getSerialNumbersIDs` function
- Create the `extendedSearch` function
- Create the `createInventoryAdjustments` function
- Create the `isEmpty` function
- Create the return statement
- Save your script file

Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 Suitelet script. The script six five SuiteScript modules specified in the define statement:

- `N/ui/serverWidget` – lets you work with the user interface within NetSuite
- `N/runtime` – provides access to runtime settings for company, script, session, system, user, and version
- `N/record` – lets you work with NetSuite records
- `N/search` – lets you create and run on-demand or saved searches and analyze and iterate through the search results.
- `N/http` – lets you make http calls
- `N/log` – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/ui/serverWidget', 'N/runtime', 'N/record', 'N/search', 'N/http', 'N/log'], (widget, runtime, record, search, http, log)
8  => {
9  });

```

Create the entry point function

This script is triggered on the `onRequest` entry point function when the Suitelet gets an HTTP request. A try-catch block logs any errors that might occur during script execution. Most of the script code goes in the try block.

Add the following function definition and initial try-catch block statements at the top of the define function:

```

1  function onRequest(scriptContext) {
2      const stLogTitle = 'onRequest';
3      try {

```

```

4      } catch(error) {
5          log.error({
6              title: stLogTitle,
7              details: error
8          });
9      }
10 }

```

Create the Item Substitution form on a GET request

The Suitelet form lets the user select a substitute item when there is insufficient quantity of an item in their order. The Suitelet receives a GET request from the helper client script (see [Step 3: Write the Item Substitution Button Helper Script](#)) when the user clicks the Fulfill With Substitutes button on the sales order.

Add the following code at the top of the try block:

```

1  if (scriptContext.request.method === http.Method.GET) {
2      log.debug({
3          title: stLogTitle,
4          details: 'get'
5      });
6  }

```

Get script parameters

This Suitelet uses a script parameter to specify the maximum number of items on a sales order. It also uses the record ID from the GET request.

Add the following code within the if block that is processing the GET request:

```

1  const maxLinesParam = runtime.getCurrentScript().getParameter({
2      name: 'custscript_max_lines'
3  });
4  const recordID = scriptContext.request.parameters.record_id;

```

Create and display the Suitelet form

The form created in this Suitelet has a header tab, a submit button, and several sublists and fields.

Add the following code within the if block that is processing the GET request:

```

1  let itemSubForm = widget.createForm({
2      title: 'Item Substitution'
3  });
4  const headerTab = itemSubForm.addTab({
5      id: 'headertab',
6      label: 'Tab'
7  });
8  addSublistAndFields(itemSubForm, recordID, maxLinesParam);
9  itemSubForm.addSubmitButton({
10     label: 'Save'
11 });
12 itemSubForm.clientScriptModulePath = runtime.getCurrentScript().getParameter({
13     name: 'custscript_sl_client_script_path'
14 });
15 scriptContext.response.writePage(itemSubForm);

```

Create the inventory adjustment record on a POST request

This Suitelet creates inventory adjustment records when it receives a POST request, which happens when the user clicks the **Save** button on the Suitelet. The form data is validated before the POST request is sent.

As part of the processing of the POST request, you'll add code for the following:

- Get the record ID and script parameters
- Get the number of lines on the sublist
- Initialize variable arrays to store sales order data
- Load the saved search
- Process sublist data
- Log remaining governance units
- Create inventory adjustments
- Update and save the sales order
- Log remaining governance units

Add the following code as the else clause of the if statement added to process the GET request:

```
1 | else if (scriptContext.request.method === http.Method.POST) {
2 | }
```

Get the record ID and script parameters

This Suitelet uses the data passed as parameters in the POST request.

Add the following code at the top of the else if block that is processing the POST request:

```
1 | const recordID = scriptContext.request.parameters.sales_order_id;
2 | const subsidiary = scriptContext.request.parameters.sales_order_subsidary;
3 | const location = scriptContext.request.parameters.sales_order_location;
4 | const inventoryNumbersSearch = runtime.getCurrentScript().getParameter({
5 |   name: 'custscript_inv_nums_search'
6 | });
7 | const account = runtime.getCurrentScript().getParameter({
8 |   name: 'custscript_inv_adj_account'
9 | });
```

Get the number of lines on the sublist

This Suitelet uses the number of lines in the sublist, which is specified by a script parameter.

Add the following code within the else if block that is processing the POST request:

```
1 | const sublistLength = scriptContext.request.getLineCount({
2 |   group: 'so_lines_sublist'
3 | });
```

Initialize variable arrays to store sales order data

This Suitelet stores data in two arrays: one for the sublist items and one for the sales order lines.

Add the following code within the else if block that is processing the POST request:

```
1 | let arrItemsInfo = [];
2 | let arrSOLines = [];
```

Load the saved search

This Suitelet loads the inventory number search specified as a parameter in the POST request.

Add the following code within the else if block that is processing the POST request:

```
1 | const objSavedSearch = search.load({
2 |   id: inventoryNumbersSearch
3 | });
```

Process sublist data

This Suitelet uses a for loop to process all the sublist data in the POST request. This processing includes the following:

- Get sublist data from the POST request
- Get serial numbers from the POST request
- Save the data from the POST request

Add the following code within the else if block that is processing the POST request:

```
1 | for (let i = 0; i < sublistLength; i++) {
2 | }
```

Get sublist data from the POST request

This Suitelet receives sublist data as a parameter in the POST request. This data includes the item ID, the substitute item ID, quantity, and serial numbers, and the sales order line.

Add the following code at the top of the for loop:

```
1 | let originalItem = scriptContext.request.getSublistValue({
2 |   group: 'so_lines_sublist',
3 |   name: 'item_id',
4 |   line: i
5 | });
6 | let substituteItem = scriptContext.request.getSublistValue({
7 |   group: 'so_lines_sublist',
8 |   name: 'item_substitute_id',
9 |   line: i
10 | });
11 | let substituteItemQty = scriptContext.request.getSublistValue({
12 |   group: 'so_lines_sublist',
13 |   name: 'item_substitute_qty',
14 |   line: i
15 | });
16 | let serialNumberField = scriptContext.request.getSublistValue({
17 |   group: 'so_lines_sublist',
18 |   name: 'item_substitute_serial_nums',
19 |   line: i
20 | });
21 | let salesOrderLine = scriptContext.request.getSublistValue({
22 |   group: 'so_lines_sublist',
23 |   name: 'sales_order_line',
24 |   line: i
25 | });
26 | arrSOLines.push(salesOrderLine);
```

Get serial numbers from the POST request

This Suitelet receives serial numbers as a parameter in the POST request. The serial numbers are included as one CSV string that needs to be split into individual serial numbers.

Add the following code within the for loop:

```
1 | let arrSerialNumbersIDs = [];
```

```

2 | if (!isEmpty(serialNumberField)) {
3 |   let serialNumbersSplit = serialNumberField.split(',');
4 |   arrSerialNumbersIDs = getSerialNumbersIDs(serialNumbersSplit, objSavedSearch, arrSerialNumbersIDs);
5 | }

```

Save the data from the POST request

This Suitelet saves all the data from the POST request into an array.

Add the following code within the for loop:

```

1 | let objectItemInfo = {};
2 | objectItemInfo.originalItem = originalItem;
3 | objectItemInfo.substituteItem = substituteItem;
4 | objectItemInfo.substituteItemQty = substituteItemQty;
5 | objectItemInfo.arrSerialNumbersIDs = arrSerialNumbersIDs;
6 | arrItemsInfo.push(objectItemInfo);

```

Log remaining governance units

This Suitelet logs how many governance units are left at this point in the Suitelet execution.

Add the following code after the end of the for loop, within the else if block that is processing the POST request.

```

1 | const remainingUnitsBeforeCreation = runtime.getCurrentScript().getRemainingUsage();
2 | log.debug({
3 |   title: stLogTitle,
4 |   details: 'remainingUnitsBeforeCreation: ' + remainingUnitsBeforeCreation
5 | });

```

Create inventory adjustments

This Suitelet creates inventory adjustment records when a substitute item is used to fulfill a sales order. One record removes the substitute items from inventory, and one record adds the original items back into inventory.

Add the following code within the else if block that is processing the POST request:

```

1 | createInventoryAdjustments(subsidiary, location, account, arrItemsInfo);

```

Update and save the sales order

This Suitelet uses a for loop to process each line on the sales order. After the order is processed, it's saved.

Add the following code within the else if block that is processing the POST request:

```

1 | const salesOrderRec = record.load({
2 |   type: record.Type.SALES_ORDER,
3 |   id: recordID,
4 |   isDynamic: true
5 | });
6 | for (let i = 0; i < arrSOLines.length; i++) {
7 |   salesOrderRec.selectLine({
8 |     sublistId: 'item',
9 |     line: i
10 |   });
11 |   salesOrderRec.setCurrentSublistValue({
12 |     sublistId: 'item',

```

```

13     fieldId: 'custcol_line_processed_substitute',
14     value: true
15   });
16   salesOrderRec.commitLine({
17     sublistId: 'item'
18   });
19 }
20 const salesOrderUpdated = salesOrderRec.save({
21   enableSourcing: true,
22   ignoreMandatoryFields: true
23 });

```

Log remaining governance units

This Suitelet logs how many governance units are left at this point in the Suitelet execution.

Add the following code within the `else if` block that is processing the POST request:

```

1 log.audit({
2   title: stLogTitle,
3   details: 'Sales Order Updated: ' + salesOrderUpdated
4 });
5 const remainingUnitsEND = runtime.getCurrentScript().getRemainingUsage();
6 log.debug({
7   title: stLogTitle,
8   details: 'remainingUnitsEND: ' + remainingUnitsEND
9 });

```

Create the addSublistAndFields function

This Suitelet defines an `addSublistAndFields` function to complete the Item Substitution form. In this function, you'll add code for the following:

- [Add a try-catch block](#)
- [Add sales order fields to the Suitelet form](#)
- [Add substitute item sublist data and the sales order line to the Suitelet form](#)
- [Populate the sublist and fields on the Suitelet form](#)

Add the following code after the end of the `onRequest` function:

```

1 function addSublistAndFields(form, recordID, maxLines) {
2 }

```

Add a try-catch block

This function uses a global variable to set the log message title and uses a try-catch block to add the sublists and fields to the Item Substitution form. If an error occurs, the script logs the error to the console.

Add the following code at the top of the `addSublistAndFields` function:

```

1 const stLogTitle = 'addSublistAndFields';
2 try {
3 } catch(error) {
4   log.error({
5     title: stLogTitle,
6     details: error
7   });
8 }

```


Add sales order fields to the Suitelet form

The Suitelet form includes hidden fields for the sales order ID, subsidiary, and location.

Add the following code at the top of the try block of the addSublistAndFields function:

```

1  const salesOrderID = form.addField({
2    id: 'sales_order_id',
3    type: ui.FieldType.TEXT,
4    label: 'Sales Order ID'
5  });
6  const salesOrderSubsidiary = form.addField({
7    id: 'sales_order_subsidiary',
8    type: ui.FieldType.TEXT,
9    label: 'Sales Order Subsidiary'
10 });
11 const salesOrderLocation = form.addField({
12   id: 'sales_order_location',
13   type: ui.FieldType.TEXT,
14   label: 'Sales Order Location'
15 });
16 salesOrderID.updateDisplayType({
17   displayType: ui.FieldDisplayType.HIDDEN
18 });
19 salesOrderSubsidiary.updateDisplayType({
20   displayType: ui.FieldDisplayType.HIDDEN
21 });
22 salesOrderLocation.updateDisplayType({
23   displayType: ui.FieldDisplayType.HIDDEN
24 });

```

Add substitute item sublist data and the sales order line to the Suitelet form

The Suitelet form includes a sublist for substitute items. This sublist has fields for the item, item ID, quantity, substitute, substitute ID, substitute quantity, substitute serial numbers, and the sales order line. The user can enter data into the item substitute quantity and serial number fields.

Add the following code within the try block of the addSublistAndFields function:

```

1  const substituteItemsSublist = form.addSublist({
2    id: 'so_lines_sublist',
3    type: ui.SublistType.LIST,
4    tab: 'headertab',
5    label: 'Substitute Items'
6  });
7  const itemName = substituteItemsSublist.addField({
8    id: 'item_name',
9    type: ui.FieldType.TEXT,
10   label: 'Item'
11 });
12 const itemID = substituteItemsSublist.addField({
13   id: 'item_id',
14   type: ui.FieldType.TEXT,
15   label: 'Item ID'
16 });
17 const itemQty = substituteItemsSublist.addField({
18   id: 'item_qty',
19   type: ui.FieldType.FLOAT,
20   label: 'Item Qty'
21 });
22 const itemSubstitute = substituteItemsSublist.addField({
23   id: 'item_substitute',
24   type: ui.FieldType.TEXT,
25   label: 'Item Substitute'
26 });
27 const itemSubstituteID = substituteItemsSublist.addField({
28   id: 'item_substitute_id',
29   type: ui.FieldType.TEXT,

```

```

30     label: 'Item Substitute ID'
31   });
32   const itemSubstituteQty = substituteItemsSublist.addField({
33     id: 'item_substitute_qty',
34     type: ui.FieldType.FLOAT,
35     label: 'Substitute Quantity'
36   });
37   itemSubstituteQty.updateDisplayType({
38     displayType: ui.FieldDisplayType.ENTRY
39   });
40   const itemSubstituteSerialNumbers = substituteItemsSublist.addField({
41     id: 'item_substitute_serial_nums',
42     type: ui.FieldType.TEXT,
43     label: 'Substitute Serial Numbers'
44   });
45   itemSubstituteSerialNumbers.updateDisplayType({
46     displayType: ui.FieldDisplayType.ENTRY
47   });
48   const salesOrderLine = substituteItemsSublist.addField({
49     id: 'sales_order_line',
50     type: ui.FieldType.TEXT,
51     label: 'Sales Order Line'
52   });

```

Populate the sublist and fields on the Suitelet form

This Suitelet fills in the sublist and fields on the Item Substitution form.

Add the following code into the try block:

```

1 | populateSublistAndFields(salesOrderSubsidiary, salesOrderLocation, salesOrderID, substituteItemsSublist, recordID, maxLines);

```

Create the populateSublistAndFields function

This Suitelet defines a populateSublistAndFields function to fill in the sublists and fields on the Suitelet form. Within this function, you'll add code for the following:

- Add a try-catch block
- Load the sales order
- Set default values
- Set variables to iterate through the items on the sales order
- Populate fields and sublists

Add the following code after the end of the addSublistAndFields function:

```

1 | function populateSublistAndFields(subsidiaryField, locationField, salesOrderIDField, sublist, recordID, maxLines) {
2 | }

```

Add a try-catch block

This function uses a global variable to set the log message title and fills in the Item Substitution form using a try-catch block. If an error occurs, the script logs the error to the console.

Add the following code at the top of the populateSublistAndFields function:

```

1 | const stLogTitle = 'populateSublistAndFields';
2 | try {
3 | } catch(error) {
4 |   log.error({
5 |     title: stLogTitle,

```

```

6         details: error
7     });
8 }

```

Load the sales order

This function loads the sales order to fill in the fields and sublists, including substitute items.

Add the following code at the top of the try block of the populateSublistAndFields function:

```

1  const soRecord = record.load({
2      type: record.Type.SALES_ORDER,
3      id: recordID,
4      isDynamic: true
5  });

```

Set default values

This function sets default values for the sales order ID, the subsidiary, and the location.

Add the following code into the try block of the populateSublistAndFields function:

```

1  salesOrderIDField.defaultValue = recordID;
2  subsidiaryField.defaultValue = soRecord.getValue('subsidiary');
3  locationField.defaultValue = soRecord.getValue('location');

```

Set variables to iterate through the items on the sales order

This function uses variables to iterate through the items on the sales order.

Add the following code into the try block of the populateSublistAndFields function:

```

1  const countLines = soRecord.getLineCount({
2      sublistId: 'item'
3  });
4  let line = 0;

```

Populate fields and sublists

This function fills in the following fields and sublists: Item ID, Item Quantity, Substitute Item, Substitute Item ID, and sales order line.

Add the following code into the try block of the populateSublistAndFields function:

```

1  for (let i = 0; i < countLines; i++) {
2      if (line < maxLines) {
3          soRecord.selectline({
4              sublistId: 'item',
5              line: i
6          });
7          let lineAlreadyProcessed = soRecord.getCurrentSublistValue({
8              sublistId: 'item',
9              fieldId: 'custcol_line_processed_substitute'
10         });
11         if (lineAlreadyProcessed === 'F' || lineAlreadyProcessed === false) {
12             let itemQty = soRecord.getCurrentSublistValue({
13                 sublistId: 'item',
14                 fieldId: 'quantityavailable'
15             });
16             let orderQty = soRecord.getCurrentSublistValue({
17                 sublistId: 'item',
18                 fieldId: 'quantity'

```

```

19     });
20     if (itemQty < orderQty) {
21         let item = soRecord.getCurrentSublistText({
22             sublistId: 'item',
23             fieldId: 'item'
24         });
25         sublist.setSublistValue({
26             id: 'item_name',
27             line: line,
28             value: item
29         });
30         let itemID = soRecord.getCurrentSublistValue({
31             sublistId: 'item',
32             fieldId: 'item'
33         });
34         sublist.setSublistValue({
35             id: 'item_id',
36             line: line,
37             value: itemID
38         });
39         sublist.setSublistValue({
40             id: 'item_qty',
41             line: line,
42             value: itemQty
43         });
44         let itemSubstitute = soRecord.getCurrentSublistValue({
45             sublistId: 'item',
46             fieldId: 'custcol_item_substitute'
47         });
48         if (!isEmpty(itemSubstitute)) {
49             sublist.setSublistValue({
50                 id: 'item_substitute',
51                 line: line,
52                 value: itemSubstitute
53             });
54             let itemSubstituteID = soRecord.getCurrentSublistValue({
55                 sublistId: 'item',
56                 fieldId: 'custcol_sub_item_id'
57             });
58             sublist.setSublistValue({
59                 id: 'item_substitute_id',
60                 line: line,
61                 value: itemSubstituteID
62             });
63         }
64         sublist.setSublistValue({
65             id: 'sales_order_line',
66             line: line,
67             value: i
68         });
69         line++;
70     }
71 }
72 }
73 }

```

Create the getSerialNumbersIDs function

This Suitelet defines a `getSerialNumbersIDs` function to split the string of serial numbers from the POST request into individual values. Then it creates a search filter using those numbers. Within this function, you'll add code for the following:

- Add a try-catch block
- Initialize search variables
- Create the search filter
- Run the saved search
- Save and return the IDs of the serial numbers

Add the following code after the end of the `populateSublistAndFields` function:

```
1 function getSerialNumbersIDs(serialNumbersSplit, objSavedSearch, arrSerialNumbersIDs) {
2 }
```

Add a try-catch block

This function uses a global variable to set the log message title and retrieves the IDs of the serial numbers for the substitute items using a try-catch block. If an error occurs, the script logs the error to the console.

Add the following code at the top of the `getSerialNumbersIDs` function:

```
1 const stLogTitle = 'getSerialNumbersIDs';
2 try {
3 } catch(error) {
4     log.error({
5         title: stLogTitle,
6         details: error
7     });
8 }
```

Initialize search variables

This function uses an array variable and an index to store the search filter.

Add the following code at the top of the try block of the `getSerialNumbersIDs` function:

```
1 let namesFilter = [];
2 let index = 0;
```

Create the search filter

This function uses a search filter to find inventory numbers that match the serial number IDs of the substitute items.

Add the following code to the try block of the `getSerialNumbersIDS` function:

```
1 for (let i = 0; i < serialNumbersSplit.length; i++) {
2     namesFilter.push(['inventorynumber', 'IS', serialNumbersSplit[i]]);
3     if (index !== (serialNumbersSplit.length - 1)) {
4         namesFilter.push('OR');
5     }
6     index++;
7 }
```

Run the saved search

This function runs a search based on the inventory number and search filters.

Add the following code after the `for` block within the try block:

```
1 const results = extendedSearch('inventorynumber', objSavedSearch, namesFilter);
```

Save and return the IDs of the serial numbers

This function splits the search results into an ID and an inventory number, then saves and returns them.

Add the following code into the try block of the `getSerialNumbersIDs` function:

```

1 for (let i = 0; i < results.length; i++) {
2   let obj = new Object();
3   obj.internalID = results[i].id;
4   obj.serialName = results[i].getValue('inventorynumber');
5   arrSerialNumbersIDs.push(obj);
6 }
7 return arrSerialNumbersIDs;

```

Create the extendedSearch function

This Suitelet defines an extendedSearch function to search for sales order and item data. Within this function, you will add code for the following:

- [Verify function parameters](#)
- [Initialize search variables](#)
- [Build the saved search](#)
- [Run the saved search](#)
- [Process and return the search results](#)

Add the following code after the end of the getSerialNumbersIDs function:

```

1 function extendedSearch(stRecordType, objSavedSearch, arrSearchFilter, arrSearchColumn) {
2 }

```

Verify function parameters

This function expects up to four parameters: record type, saved search, search filter and search column. You have to provide either the record type or saved search.

Add the following code at the top of the extendedSearch function:

```

1 if (stRecordType === null && objSavedSearch === null) {
2   error.create({
3     name: 'SSS_MISSING_REQD_ARGUMENT',
4     message: 'search: Missing a required argument. Either stRecordType or objSavedSearch should be provided.',
5     notifyOff: false
6   });
7 }

```

Initialize search variables

This function uses two variables to store the search results and set the maximum number of results allowed.

Add the following code after the if statement block:

```

1 let arrReturnSearchResults = new Array();
2 const maxResults = 1000;

```

Build the saved search

This function builds on the saved search you pass in, or creates a new one.

Add the following code:

```

1 if (objSavedSearch !== null) {
2   objSavedSearch.filters = [];

```

```

3   if (arrSearchFilter !== null) {
4       if (arrSearchFilter[0] instanceof Array || (typeof arrSearchFilter[0] === 'string')) {
5           objSavedSearch.filterExpression = objSavedSearch.filterExpression.concat(arrSearchFilter);
6       } else {
7           objSavedSearch.filters = objSavedSearch.filters.concat(arrSearchFilter);
8       }
9   }
10  if (arrSearchColumn !== null) {
11      objSavedSearch.columns = objSavedSearch.columns.concat(arrSearchColumn);
12  }
13  } else {
14      let objSavedSearch = search.create({
15          type: stRecordType
16      });
17      if (arrSearchFilter !== null) {
18          if (arrSearchFilter[0] instanceof Array || (typeof arrSearchFilter[0] === 'string')) {
19              objSavedSearch.filterExpression = arrSearchFilter;
20          } else {
21              objSavedSearch.filters = arrSearchFilter;
22          }
23      }
24      if (arrSearchColumn !== null) {
25          objSavedSearch.columns = arrSearchColumn;
26      }
27  }

```

Run the saved search

This function runs the search after it's built.

Add the following code:

```

1 | const objResultset = objSavedSearch.run();

```

Process and return the search results

This function concatenates the search results before returning them.

Add the following code:

```

1 | let intSearchIndex = 0;
2 | let arrResultSlice = null;
3 | do {
4 |     arrResultSlice = objResultset.getRange(intSearchIndex, intSearchIndex + maxResults);
5 |     if (arrResultSlice === null) {
6 |         break;
7 |     }
8 |     arrReturnSearchResults = arrReturnSearchResults.concat(arrResultSlice);
9 |     intSearchIndex = arrReturnSearchResults.length;
10  } while (arrResultSlice.length >= maxResults);
11  return arrReturnSearchResults;

```

Create the createInventoryAdjustments function

This Suitelet defines a createInventoryAdjustments function to create inventory adjustment records when the user select substitute items on a sales order and saves it. Within this function, you'll add code for the following:

- Add a try-catch block
- Create an inventory adjustment (deletion) record
- Create an inventory adjustment (addition) record

Add the following code after the end of the extendedSearch function:

```

1 function createInventoryAdjustments(subsidiary, location, account, arrItemsInfo) {
2 }

```

Add a try-catch block

This function uses a global variable to set the log message title and adds inventory adjustment records using a try-catch block. If an error occurs, the script logs the error to the console.

Add the following code at the top of the createInventoryAdjustments function:

```

1 const stLogTitle = 'createInventoryAdjustments';
2 try {
3 } catch(error) {
4     log.error({
5         title: stLogTitle,
6         details: error
7     });
8 }

```

Create an inventory adjustment (deletion) record

This function creates an inventory adjustment record to reduce the quantity of the substitute item when the user selects one for the sales order. You'll write code to do the following:

- Set the sublist fields and save
- Save the record and log a message

Add the following code at the top of the try block of the createInventoryAdjustments function:

```

1 let inventoryAdjustmentDeletion = record.create({
2     type: record.Type.INVENTORY_ADJUSTMENT,
3     isDynamic: true
4 });
5 inventoryAdjustmentDeletion.setValue({
6     fieldId: 'subsidiary',
7     value: subsidiary
8 });
9 inventoryAdjustmentDeletion.setValue({
10    fieldId: 'adjlocation',
11    value: location
12 });
13 inventoryAdjustmentDeletion.setValue({
14    fieldId: 'account',
15    value: account
16 });

```

Set the sublist fields and save

This function iterates through all items on the sublist to set values for the substitute item on the inventory adjustment record.

Add the following code within the try block of the createInventoryAdjustments function:

```

1 for (let j = 0; j < arrItemsInfo.length; j++) { let objectItemInfo = arrItemsInfo[j]; inventoryAdjustmentDeletion.selectNew
  Line({ sublistId: 'inventory' }); inventoryAdjustmentDeletion.setCurrentSublistValue({ sublistId: 'inventory', fieldId: 'item', val
  ue: objectItemInfo.substituteItem }); inventoryAdjustmentDeletion.setCurrentSublistValue({ sublistId: 'inventory', fieldId: 'lo
  cation', value: location }); inventoryAdjustmentDeletion.setCurrentSublistValue({ sublistId: 'inventory', fieldId: 'adjustqty
  by', value: (objectItemInfo.substituteItemQty * (-1)) }); let itemTypeLookup = search.lookupFields({ type: search.Type.ITEM, id: ob
  jectItemInfo.substituteItem, columns: 'recordtype' }); if (!isEmpty(itemTypeLookup)) { let itemType = itemTypeLookup.record
  type; if (itemType === 'serializedinventoryitem') { if (objectItemInfo.arrSerialNumbersIDs.length > 0) {
2      let inventoryDetailIA = inventoryAdjustmentDeletion.getCurrentSublistSubrecord({
3          sublistId: 'inventory',

```



```

4         fieldId: 'inventorydetail'
5     });
6     for (let index = 0; index < objectItemInfo.arrSerialNumbersIDs.length; index++) {
7         inventoryDetailIA.selectNewLine({
8             sublistId: 'inventoryassignment',
9         });
10        let line = inventoryDetailIA.getCurrentSublistIndex({
11            sublistId: 'inventoryassignment'
12        });
13        inventoryDetailIA.setCurrentSublistValue({
14            sublistId: 'inventoryassignment',
15            fieldId: 'issueinventorynumber',
16            value: JSON.parse(objectItemInfo.arrSerialNumbersIDs[index]).internalID
17        });
18        inventoryDetailIA.commitLine({
19            sublistId: 'inventoryassignment'
20        });
21    }
22    }
23    }
24    }
25    inventoryAdjustmentDeletion.commitLine({
26        sublistId: 'inventory'
27    });
28    }

```

Save the record and log a message

This function saves the inventory adjustment record and logs an audit message.

Add the following code within the try block of the createInventoryAdjustments function:

```

1 const invAdjIDDeletion = inventoryAdjustmentDeletion.save({
2     enableSourcing: true,
3     ignoreMandatoryFields: true
4 });
5 log.audit({
6     title: 'Inventory Adjustment deletion: ',
7     details: invAdjIDDeletion
8 });

```

Create an inventory adjustment (addition) record

This function creates an inventory adjustment record to increase the quantity of the original item when the user selected a substitute item for the sales order. You'll write code to do the following:

- Set the sublist fields and save
- Save the record and log a message

Add the following code within the try block of the createInventoryAdjustments function:

```

1 let inventoryAdjustmentAddition = record.create({
2     type: record.Type.INVENTORY_ADJUSTMENT,
3     isDynamic: true
4 });
5 inventoryAdjustmentAddition.setValue({
6     fieldId: 'subsidiary',
7     value: subsidiary
8 });
9 inventoryAdjustmentAddition.setValue({
10    fieldId: 'adjlocation',
11    value: location
12 });
13 inventoryAdjustmentAddition.setValue({
14    fieldId: 'account',
15    value: account

```

```
16 | });
```

Set the sublist fields and save

This function iterates through all the items on the sublist to set values for the original item on the inventory adjustment record.

Add the following code into the try block of the createInventoryAdjustments function:

```
1 | for (let k = 0; k < arrItemsInfo.length; k++) { let objectItemInfoAdd = arrItemsInfo[k]; inventoryAdjustmentAddition.selectNew
  | Line({ sublistId: 'inventory' }); inventoryAdjustmentAddition.setCurrentSublistValue({ sublistId: 'inventory', fieldId: 'item', val
  | ue: objectItemInfoAdd.originalItem }); inventoryAdjustmentAddition.setCurrentSublistValue({ sublistId: 'inventory', fieldId: 'loca
  | tion', value: location }); inventoryAdjustmentAddition.setCurrentSublistValue({ sublistId: 'inventory', fieldId: 'adjustqtyby', val
  | ue: objectItemInfoAdd.substituteItemQty }); let itemTypeLookup = search.lookupFields({ type: search.Type.ITEM, id: objectItemIn
  | foAdd.originalItem, columns: 'recordtype' }); if (!isEmpty(itemTypeLookup)) { let itemType = itemTypeLookup.recordtype; if (item
  | Type === 'serializedinventoryitem') { if (objectItemInfoAdd.arrSerialNumbersIDs.length > 0) {
  |     let inventoryDetailIAAddition = inventoryAdjustmentAddition.getCurrentSublistSubrecord({
  |         sublistId: 'inventory',
  |         fieldId: 'inventorydetail'
  |     });
  |     for (let index1 = 0; index1 < objectItemInfoAdd.arrSerialNumbersIDs.length; index1++) {
  |         inventoryDetailIAAddition.selectNewLine({
  |             sublistId: 'inventoryassignment',
  |         });
  |         inventoryDetailIAAddition.setCurrentSublistValue({
  |             sublistId: 'inventoryassignment',
  |             fieldId: 'receiptinventorynumber',
  |             value: objectItemInfoAdd.arrSerialNumbersIDs[index1].serialName
  |         });
  |         inventoryDetailIAAddition.commitLine({
  |             sublistId: 'inventoryassignment'
  |         });
  |     }
  | }
  | }
  | }
  | inventoryAdjustmentAddition.commitLine({
  |     sublistId: 'inventory'
  | });
  | }
```

Save the record and log a message

This function saves the inventory adjustment record and logs an audit message.

Add the following code into the try block of the createInventoryAdjustments function, following the for block:

```
1 | const invAdjIDAddition = inventoryAdjustmentAddition.save({
  |     enableSourcing: true,
  |     ignoreMandatoryFields: true
  | });
  | log.audit({
  |     title: 'Inventory Adjustment addition: ',
  |     details: invAdjIDAddition
  | });
```

Create the isEmpty function

This Suitelet uses a support function to determine if a value is empty, null, or undefined.

Add the following code after the end of the createInventoryAdjustments function:

```
1 | function isEmpty(stValue) {
  |     return stValue === '' || stValue === null || stValue === undefined;
```

```
3 | }
```

Create the return statement

This script associates the onRequest function with the onRequest client script entry point.

Add the following code immediately above the closing `});` in your script:

```
1 | return {
2 |   onRequest: onRequest
3 | };
```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you are happy with your script, save it as a .js file (for example, `sl_itemSubstitution.js`).

Step 5: Write the Validate Item Substitution Script

This script validates the fields on the Suitelet form.

Script Summary

The following table summarizes this script:

Script: Validate Item Substitution	
Script Type	SuiteScript 2.x Client Script Type
Modules Used	■ N/record Module ■ N/search Module ■ N/runtime Module ■ N/ui/dialog Module ■ N/currentRecord Module – This module is available to all scripts as a provided context object. You don't need to explicitly load this module as a dependency in your define or require statement, however, you can if you want. This tutorial doesn't explicitly load this module. ■ N/log Module – This module is available to all scripts as a global object. However, you should explicitly load this module to avoid conflicts with other objects that may be named 'log'.
Entry Points	■ <code>validateLine(scriptContext)</code>

For more information about script types and entry points, see the help topic [SuiteScript 2.x Script Types](#).

The Complete Script

This tutorial includes the complete script along with individual steps you can follow to build the script in logical sections. The complete script is provided below so that you can copy it, paste it into your text editor, and save it as a .js file (for example, `cs_validate_itemSubstitution.js`).

If you would rather create this script step by step, follow the steps in [Build the Script](#).

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#)

Note

This sample uses SuiteScript 2.1.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/runtime', 'N/ui/dialog', 'N/log'], (record, search, runtime, dialog, log) => {
8      let itemSearch;
9      let substAvailQtyEmptyMsg;
10     function validateLine(scriptContext) {
11         try {
12             if (isEmpty(itemSearch) && isEmpty(substAvailQtyEmptyMsg)) {
13                 itemSearch = runtime.getCurrentScript().getParameter({
14                     name: 'custscript_item_id_from_name_ss'
15                 });
16                 substAvailQtyEmptyMsg = runtime.getCurrentScript().getParameter({
17                     name: 'custscript_substit_available_qty_empty'
18                 });
19             }
20             const currentRecord = scriptContext.currentRecord;
21             const salesOrderLocation = currentRecord.getValue({
22                 fieldId: 'location'
23             });
24             const quantityAvailable = currentRecord.getCurrentSublistValue({
25                 sublistId: 'item',
26                 fieldId: 'quantityavailable'
27             });
28             const quantityOrdered = currentRecord.getCurrentSublistValue({
29                 sublistId: 'item',
30                 fieldId: 'quantity'
31             });
32             if (quantityAvailable < quantityOrdered) {
33                 const itemSubstituteName = currentRecord.getCurrentSublistValue({
34                     sublistId: 'item',
35                     fieldId: 'custcol_item_substitute'
36                 });
37                 const itemID = getItemID(itemSearch, itemSubstituteName);
38                 if (!isEmpty(itemID)) {
39                     currentRecord.setCurrentSublistValue({
40                         sublistId: 'item',
41                         fieldId: 'custcol_sub_item_id',
42                         value: itemID
43                     });
44                     const itemTypeLookup = search.lookupFields({
45                         type: search.Type.ITEM,
46                         id: itemID,
47                         columns: 'recordtype'
48                     });
49                     if (!isEmpty(itemTypeLookup)) {
50                         const itemType = itemTypeLookup.recordtype;
51                         const itemRecord = record.load({
52                             type: itemType,
53                             id: itemID,
54                             isDynamic: true
55                         });
56                         const countLines = itemRecord.getLineCount({
57                             sublistId: 'locations'

```

```

58         });
59         for (let i = 0; i < countLines; i++) { itemRecord.selectLine({ sublistId: 'locations', line: i }); let line
Location = itemRecord.getCurrentSublistValue({ sublistId: 'locations', fieldId: 'location' }); if (lineLocation === salesOrder
Location) { let availableQty = itemRecord.getCurrentSublistValue({ sublistId: 'locations', fieldId: 'quantityavailable' }); if (!
isEmpty(availableQty)) { currentRecord.setCurrentSublistValue({ sublistId: 'item', fieldId: 'custcol_sub_avail_qty', val
ue: availableQty }); } else { dialog.alert({ title: "Alert", message: substAvailQtyEmptyMsg }); } } } } } catch(error) { con
sole.log(stLogTitle, error); } return true; } function getItemID(itemsSavedSearch, itemSubstituteName) { try { let itemID; con
st itemSearch = search.load({ id: itemsSavedSearch }); const nameFilter = search.createFilter({ name: 'itemid', operator: search.Op
erator.IS, values: itemSubstituteName }); itemSearch.filters.push(nameFilter); const itemResult = itemSearch.run().getRange({ s
tart: 0, end: 1 }); if (!isEmpty(itemResult) && itemResult.length > 0) {
60         itemID = itemResult[0].getValue('internalid');
61     }
62     return itemID;
63 } catch(error) {
64     console.log('getItemID', error);
65 }
66 }
67 function isEmpty(stValue) {
68     return stValue === '' || stValue === null || stValue === undefined;
69 }
70 return {
71     validateLine: validateLine
72 };
73 });

```

Build the Script

You can write the script using a step-by-step approach that includes the following:

- Start with required opening lines
- Create the entry point function
- Complete the validateLine function
- Create the getItemID function
- Create the isEmpty function
- Create the return statement
- Save your script file



Note: The code snippets included below don't include indentation. Refer to [The Complete Script](#) for suggested formatting.

Start with required opening lines

You need JSDoc comments and a define function at the top of the script file. The JSDoc comments in this script indicate that it's a SuiteScript 2.1 client script. The script uses five SuiteScript modules specified in the define statement:

- N/record – lets you work with NetSuite records
- N/search – lets you create and run on-demand or saved searches and analyze and iterate through the search results
- N/runtime – provides access to runtime settings for company, script, session, system, user, or version
- N/ui/dialog – lets you create a modal dialog that is displayed until a button on the dialog is pressed
- N/log – lets you log execution details

Start a new script file using any text editor and add the following JSDoc comments and define function at the top of the file:

Note: This tutorial script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). If you want to copy the script into the SuiteScript Debugger and test it, you must use the `require` function. For more information, see the help topic [SuiteScript Debugger](#).

```
1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/record', 'N/search', 'N/runtime', 'N/ui/dialog', 'N/log'], (record, search, runtime, dialog, log) => {
8  });
```

Create the entry point function

This script is triggered on the `validateLine` entry point when the Suitelet is called after the user clicks the Open Suitelet button on a sales order form. A try-catch block is used to log any errors that might occur during script execution. Most of the script code goes in the try block.

Add the following function definition and initial try-catch block statements at the top of the `define` function:

```
1  let itemSearch;
2  let substAvailQtyEmptyMsg;
3  function validateLine(scriptContext) {
4      try {
5      } catch(error) {
6          console.log(stLogTitle, error);
7      }
8      return true;
9  }
```

Complete the validateLine function

This client script defines a `validateLine` function to validate data entered on the sales order. This validation includes determining if there is sufficient quantity of an item at the location of the sales order. Within this function, you'll add code for the following:

- [Get the script parameters](#)
- [Get data from the sales order](#)
- [Process data from the sales order](#)

Get the script parameters

This function uses script parameters to specify the search and a user message, if they're defined.

Add the following code at the top of the try block:

```
1  if (isEmpty(itemSearch) && isEmpty(substAvailQtyEmptyMsg)) {
2      itemSearch = runtime.getCurrentScript().getParameter({
3          name: 'custscript_item_id_from_name_ss'
4      });
5      substAvailQtyEmptyMsg = runtime.getCurrentScript().getParameter({
6          name: 'custscript_substit_available_qty_empty'
7      });
8  }
```

Get data from the sales order

This function gets the data entered by the user on the sales order. This data includes the location, the available quantity, and the quantity ordered for the each item.

Add the following code after the code within the try block:

```

1  const currentRecord = scriptContext.currentRecord;
2  const salesOrderLocation = currentRecord.getValue({
3    fieldId: 'location'
4  });
5  const quantityAvailable = currentRecord.getCurrentSublistValue({
6    sublistId: 'item',
7    fieldId: 'quantityavailable'
8  });
9  const quantityOrdered = currentRecord.getCurrentSublistValue({
10   sublistId: 'item',
11   fieldId: 'quantity'
12 });

```

Process data from the sales order

This function retrieves the primary substitute item for any item that has insufficient quantity and then determines if there's enough of the substitute item. This is a big chunk of code, so look at the inline comments for a description of the processing.

Add the following code after the code within the try block:

```

1  if (quantityAvailable < quantityOrdered) {
2    const itemSubstituteName = currentRecord.getCurrentSublistValue({
3      sublistId: 'item',
4      fieldId: 'custcol_item_substitute'
5    });
6    const itemID = getItemID(itemSearch, itemSubstituteName);
7    if (!isEmpty(itemID)) {
8      currentRecord.setCurrentSublistValue({
9        sublistId: 'item',
10       fieldId: 'custcol_sub_item_id',
11       value: itemID
12     });
13     const itemTypeLookup = search.lookupFields({
14       type: search.Type.ITEM,
15       id: itemID,
16       columns: 'recordtype'
17     });
18     if (!isEmpty(itemTypeLookup)) {
19       const itemType = itemTypeLookup.recordtype;
20       const itemRecord = record.load({
21         type: itemType,
22         id: itemID,
23         isDynamic: true
24       });
25       const countLines = itemRecord.getLineCount({
26         sublistId: 'locations'
27       });
28       for (let i = 0; i < countLines; i++) {
29         itemRecord.selectLine({
30           sublistId: 'locations',
31           line: i
32         });
33         let lineLocation = itemRecord.getCurrentSublistValue({
34           sublistId: 'locations',
35           fieldId: 'location'
36         });
37         if (lineLocation === salesOrderLocation) {
38           let availableQty = itemRecord.getCurrentSublistValue({
39             sublistId: 'locations',
40             fieldId: 'quantityavailable'

```

```

41 |         });
42 |         if (!isEmpty(availableQty)) {
43 |             currentRecord.setCurrentSublistValue({
44 |                 sublistId: 'item',
45 |                 fieldId: 'custcol_sub_avail_qty',
46 |                 value: availableQty
47 |             });
48 |         } else {
49 |             dialog.alert({
50 |                 title: "Alert",
51 |                 message: substAvailQtyEmptyMsg
52 |             });
53 |         }
54 |     }
55 | }
56 | }
57 | }
58 | }

```

Create the getItemID function

This client script defines a getItemID function to get the ID of each substitute item on the sales order. Within this function, you'll add code for the following:

- Add a try-catch block
- Create a global variable
- Load, update, and run the saved search
- Check and return the search results

Add the following code after the end of the validateLine function:

```

1 | function getItemID(itemsSavedSearch, itemSubstituteName) {
2 | }

```

Add a try-catch block

This function performs a search using a try-catch block. If an error occurs, the script logs the error to the console.

Add the following code at the top of the getItemID function:

```

1 | try {
2 | } catch(error) {
3 |     console.log('getItemID', error);
4 | }

```

Create a global variable

This function uses a global variable to store the substitute item ID.

Add the following code at the top of the try block:

```

1 | let itemID;

```

Load, update, and run the saved search

This function runs a saved search to get the item IDs.

Add the following code within the try block:


```

1 | const itemSearch = search.load({
2 |   id: itemsSavedSearch
3 | });
4 | const nameFilter = search.createFilter({
5 |   name: 'itemid',
6 |   operator: search.Operator.IS,
7 |   values: itemSubstituteName
8 | });
9 | itemSearch.filters.push(nameFilter);
10 | const itemResult = itemSearch.run().getRange({
11 |   start: 0,
12 |   end: 1
13 | });

```

Check and return the search results

This function returns the search results, which is the substitute item ID.

Add the following code within the try block:

```

1 | if (!isEmpty(itemResult) && itemResult.length > 0) {
2 |   itemID = itemResult[0].getValue('internalid');
3 | }
4 | return itemID;

```

Create the isEmpty function

This client script uses a support function to determine if a value is empty, null, or undefined.

Add the following code after the end of the getItemID function:

```

1 | function isEmpty(stValue) {
2 |   return stValue === '' || stValue === null || stValue === undefined;
3 | }

```

Create the return statement

This script associates the validateLine function with the validateLine client script entry point.

Add the following code immediately above the closing `});` in your script:

```

1 | return {
2 |   validateLine: validateLine
3 | };

```

Save your script file

You need to save your script file so you can upload it to the NetSuite File Cabinet. Before you save your script file, you may want to adjust the indentation to improve the readability of the script. Refer to [The Complete Script](#) for suggested formatting.

When you're happy with your script, save it as a .js file (for example, `cs_validate_itemSubstitution.js`).

Step 6: Create the Script Records

After you create your scripts, you need to create script records for each one:

- Create the script record for the Add Fulfill with Substitutes Button user event script
- Create the script record for the Item Substitution Button Helper client script
- Create the script record for the Item Substitution Suitelet script
- Create the script record for the Item Substitution Suitelet script

For more information about creating script records, see the help topic [Creating a Script Record](#).

Create the script record for the Add Fulfill with Substitutes Button user event script

To create the script record for the Add Fulfill with Substitutes Button user event script:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Add Fulfill with Substitutes Button
ID	_ue_item_substitution NetSuite prefixes the ID with customscript .
Description	This script adds a custom button to the sales order record so the user can fulfill it with substitute items.

5. Set any other fields on the script record as required.
6. Click **Save**.

Create the script record for the Item Substitution Button Helper client script

To create the script record for the Item Substitution Button Helper client script:

1. Upload your script to the NetSuite File Cabinet.
2. Go to Customization > Scripting > Scripts > New.
3. Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
4. On the Script page, enter the following values:

Field	Value
Name	Item Substitution Button Helper
ID	_cs_item_substitution_helper NetSuite prefixes the ID with customscript .

Field	Value
Description	This script calls the Suitelet when the user clicks the Fulfill With Substitutes custom button, and when the user saves the Suitelet form.

- Set any other fields on the script record as required.
- Click **Save**.

Create the script record for the Item Substitution Suitelet script

To create the script record for the Item Substitution Suitelet script:

- Upload your script to the NetSuite File Cabinet.
- Go to Customization > Scripting > Scripts > New.
- Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
- On the Script page, enter the following values:

Field	Value
Name	Item Substitution Suitelet
ID	_sl_item_substitution NetSuite prefixes the ID with customscript .
Description	This script creates the form for users to select substitute items on a sales order.

- Set any other fields on the script record as required.
- Click **Save**.

Create the script record for the Validate Item Substitution client script

To create the script record for the Validate Item Substitution client script:

- Upload your script to the NetSuite File Cabinet.
- Go to Customization > Scripting > Scripts > New.
- Select your script from the **Script File** list and click **Create Script Record**. The Script page is displayed.
- On the Script page, enter the following values:

Field	Value
Name	Validate Item Substitution
ID	_cs_valid_item_substitution NetSuite prefixes the ID with customscript .
Description	This script validates the data the user enters in the Fulfill With Substitutes Suitelet form.

- Set any other fields on the script record as required.

6. Click **Save**.

Step 7: Deploy the Scripts

After you create the script record for each of the scripts, you can create script deployment records for them. A script deployment record determines how, when, and for whom the script runs.

For more information about script deployment records, see the help topic [Script Deployment](#).

- [Deploy the Add Fulfill with Substitutes Button user event script](#)
- [Deploy the Item Substitution Button Helper client script](#)
- [Deploy the Item Substitution Suitelet script](#)
- [Deploy the Validate Item Substitution client script](#)

Deploy the Add Fulfill with Substitutes Button user event script

To deploy the Add Fulfill with Substitutes Button user event script:

1. Complete the steps in [Step 6: Create the Script Records](#) for your user event script.
2. Go to Customization > Scripting > Scripts.
3. Find your user event script in the list and click Deployments. The Script Deployments page appears.
4. Click **New Deployment**. The Script Deployment page appears.
5. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_ue_item_substitution_btn NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status let the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's best to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

6. Click **Save**.

Deploy the Item Substitution Button Helper client script

To deploy the Item Substitution Button Helper client script:

1. Complete the steps in [Step 6: Create the Script Records](#) for your client script.

2. Go to Customization > Scripting > Scripts.
3. Find your client script in the list and click Deployments. The Script Deployments page appears.
4. Click **New Deployment**. The Script Deployment page appears.
5. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_cs_item_substitution_helper NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

6. Click **Save**.

Deploy the Item Substitution Suitelet script

To deploy the Item Substitution Suitelet script:

1. Complete the steps in [Step 6: Create the Script Records](#) for your Suitelet script.
2. Go to Customization > Scripting > Scripts.
3. Find your Suitelet script in the list and click Deployments. The Script Deployments page appears.
4. Click **New Deployment**. The Script Deployment page appears.
5. On the Script Deployment page, enter the following values:

Field	Value
Title	Item Substitution Suitelet
ID	_sl_item_substitution NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Execute As Role	Current Role It's best practice to have scripts run with the user's current role to avoid granting unwanted access.
Audience > Roles	Check Select All

6. Click **Save**.

Deploy the Validate Item Substitution client script

To deploy the Validate Item Substitution client script:

1. Complete the steps in [Step 6: Create the Script Records](#) for your client script.
2. Go to Customization > Scripting > Scripts.
3. Find your client script in the list and click Deployments. The Script Deployments page appears.
4. Click **New Deployment**. The Script Deployment page appears.
5. On the Script Deployment page, enter the following values:

Field	Value
Applies To	Sales Order
ID	_cs_validate_item_subst NetSuite prefixes the ID with custdeploy .
Status	Testing The Testing status lets the script owner test the script without affecting other users in the account.
Log Level	Debug The Debug level writes all log.debug statements and errors to the Execution Log subtab of the script deployment record.
Audience > Roles	Check Select All

6. Click **Save**.

Step 8: Create and Set the Script Parameters

This solution uses several script parameters in the user event script, the Suitelet, and the client script.

- [Create and set the script parameter for the Add Fulfill with Substitutes Button user event script](#)
- [Create and set the script parameters for the Item Substitution Suitelet script](#)
- [Create and set the script parameters for the Validate Item Substitution client script](#)

For more information about creating and setting script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

Create and set the script parameter for the Add Fulfill with Substitutes Button user event script

The user event script uses a script parameter to specify the location of the client script that processes the user click on the Fulfill with Substitutes button.

To create the script parameter:

1. Go to Customization > Scripting > Scripts.
2. Locate your script in the list and click Edit next to the script name.
3. On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.

- On the Script Field page, enter the following values:

Label	ID	Type	Description	Preference
Client Script Path	_client_script_path NetSuite prefixes the ID with custscript , so the script uses custscript_client_script_path .	Free-Form text	The path (location) to the client script.	Leave blank to set the parameter value on the script deployment.

- To save the script parameter, click **Save**. The Script page appears.
- To save the script record with the updated script parameter, click **Save**.

To set the script parameter:

- Go to Customization > Scripting > Script Deployments.
- Locate your script deployment in the list and click Edit next to the script deployment name.
- On the **Parameters** subtab, enter the client script location in the **Client Script Path** field. This is the location of the helper client script. For example, if you uploaded the script to the SuiteScripts folder in the File Cabinet, enter **SuiteScripts/cs_itemSubstution_helper.js**.
- Click **Save**.

Create and set the script parameters for the Item Substitution Suitelet script

The Suitelet uses script parameters for the following: location of the client script that populates the field on the Suitelet, maximum number of lines, and IDs for the custom search and inventory adjustment account.

To create the script parameter:

- Go to Customization > Scripting > Scripts.
- Locate your script in the list and click Edit next to the script name.
- On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.
- On the Script Field page, enter the following values. You can click **Save & New** to create subsequent parameters.

Script Parameter	Label	ID	Type	Description	Preference
Max Lines	Max Lines	_max_lines NetSuite prefixes the ID with custscript , so the script uses custscript_max_lines .	Integer Number	The maximum number of item lines on the form.	Leave blank to set the parameter value on the script deployment.
Client Script Path	Client Script Path for Populate Item	_sl_client_script_path NetSuite prefixes the ID with custscript , so	Free-Form Text	The path to the client script.	

Script Parameter	Label	ID	Type	Description	Preference
		the script uses custscript_sl_client_script_path .			
Inventory Numbers Search	Inventory Numbers Search	_inv_nums_search NetSuite prefixes the ID with custscript , so the script uses custscript_inv_nums_search .	Free-Form Text	The name of the inventory numbers search.	
Inventory Adjustment Account	Inventory Adjustment Account	_inv_adj_account NetSuite prefixes the ID with custscript , so the script uses custscript_inv_adj_account .	Free-Form Text	The account ID of the Inventory Adjustment account.	

5. To save the script parameters, click **Save**. The Script page appears.
6. To save the script record with the updated script parameters, click **Save**.

To set the script parameters:

1. Go to Customization > Scripting > Script Deployments.
2. Locate your script deployment in the list and click **Edit** next to the script deployment name.
3. On the **Parameters** subtab, enter the values for each script parameter as shown in the following table.

Script Parameter	Value
Max Lines (Script ID: custscript_max_lines)	Any reasonable value, such as 10.
Validate Client Script Path (Script ID: custscript_sl_client_script_path)	This is the location of the client script. For example, if you uploaded the script to the SuiteScripts folder in the File Cabinet, enter SuiteScripts/cs_itemSubstitution.js.
Inventory Numbers Search (Script ID: custscript_inv_nums_search)	customsearch_inventory_numbers
Inventory Adjustment Account (Script ID: custscript_inv_adj_account)	Set this value to your account ID for your Inventory Adjustment account.

4. Click **Save**.

Create and set the script parameters for the Validate Item Substitution client script

The client script uses script parameters to specify the saved search and to define an alert message.

To create the script parameters:

1. Go to Customization > Scripting > Script Deployments.

2. Locate your script in the list and click **Edit** next to the script name.
3. On the Script page, click the **Parameters** subtab and click **New Parameter**. The Script Field page appears.
4. On the Script Field page, enter the following values. You can click **Save & New** to create subsequent parameters.

Parameter	Label field	ID field	Type field	Description	Preference field
Item ID from Name Saved Search	Item ID from Name NetSuite prefixes the ID with custscript .	_item_id_from_name_ss	Free-Form Text	The name of the id from name search.	Leave blank to set the parameter value on the script deployment.
Substitute Available Qty Empty	Substitute Available Qty Empty NetSuite prefixes the ID with custscript .	_sublist_available_qty_empty	Free-Form Text	Alert message displayed when a substitute item has 0 quantity.	

5. To save the script parameters, click **Save**. The Script page appears.
6. To save the script record with the updated script parameters, click **Save**.

To set the script parameters:

1. Go to Customization > Scripts > Script Deployments.
2. Locate your script deployment in the list and click **Edit** next to the script deployment name.
3. On the **Parameters** subtab, enter the values for each script parameter as shown in the following table.

Script Parameter	Value
Item ID from Name Saved Search (Script ID: custscript_item_id_from_name_ss)	customsearch_itemid_from_name
Substitute Available Qty Empty (Script ID: custscript_substit_available_qty_empty)	Sorry, that item is not available.

4. Click **Save**.

Step 9: Test the Solution

After you create the script records and deploy your scripts, you can test your solution by testing each script and then testing the entire solution by creating a new sales order and selecting substitute items for it.

To test the Add Fulfill with Substitutes Button user event script:

1. Access an existing sales order in edit mode.
2. Make sure the Fulfill With Substitutes button appears on the Sales Order page.

To test the Item Substitution Button Helper client script:

1. Access an existing sales order in edit mode.
2. Click the **Fulfill With Substitute** button.
3. Make sure the Suitelet form appears.

To test the Item Substitution Suitelet script:

1. Access a sales order, in edit mode, that has the item sublist pre-populated with:
 - Line items from the sales order
 - Ordered quantity of the line items
 - Each line item's Primary Substitute item
2. Enter the following information in this sublist:
 - The number of Substitute Items to fulfill
 - The serial numbers of the Substitute item(s)
3. Click **Save**.
4. Make sure that there's an Inventory Adjustment record that moves inventory from the Substitute Item to the originally ordered item, using the serial numbers entered in the Suitelet.
5. Make sure that the inventory detail of the original item's subrecord is populated with the new entered serial numbers.
6. Make sure that the sales order was saved and you're re-directed to the Item Fulfillment record.

To test the Validate Item Substitution client script:

1. Add a line item to a sales order.
2. Make sure the **Substitute Available Qty** column fills in automatically.
3. Make sure the **Primary Substitute** and **Internal #** fields update by sourcing (not by script).
4. Make sure you are alerted if any of the fields pulled from the Item record are empty or null.
5. Make sure that you can commit the line can be committed and the fields can be left blank.

You should test each script first before testing the entire solution.

To test the entire solution:

1. You can use the following test conditions to test the entire solution:
 - Original item inventory is on hand and available to commit when being added to a sales order
 - Original item has 0 units available to commit and substitute items have enough units to commit when being added to a sales order
 - Original item has 0 units available to commit and substitute items do not have any units available to commit when being added to a sales order
 - Original item has 0 units available to commit and substitute items only has partial units available to commit when being added to a sales order
 - Original item has partial units available to commit and substitute items have enough units to commit
 - Original item has partial units available to commit and substitute items do not have any units available to commit
 - Original item has partial units available to commit and substitute items only has partial units available to commit

2. Create a sales order that meets the test condition.
3. Edit the sales order to add substitute items as needed for the test.
4. Save the sales order.
5. Verify the saved sales order and any inventory adjustment records that were created.